arXiv:2611.14194v1 [math.CO] 10 Nov 2026

YOU CAN HAVE IT ALL: SMALL SPARSE INDUCED-UNIVERSAL GRAPHS FOR MINOR-CLOSED CLASSES

Vida Dujmović[♀] *Cyril Gavoille*[♁] *Gwenaël Joret*[♂]
Piotr Micek[♃] *Pat Morin*[♄] *David R. Wood*[♅]

Abstract. We show that, for every fixed proper minor-closed graph class $\mathcal{G}$ and every positive integer n, there exists a graph U_n with $n^{1+o(1)}$ vertices and edges such that every n-vertex graph in $\mathcal{G}$ is isomorphic to an induced subgraph of U_n.

This result generalizes and unifies the results of Dujmović et al. (*J. ACM* 2021) on induced-universal graphs and the results of Esperet, Joret, and Morin (*J. London Math. Soc.* 2023) on sparse induced-universal graphs from planar graphs to all proper minor-closed graph classes. It simultaneously improves the previous bounds for the number of vertices in induced-universal graphs (previously $O(n^2)$ obtained as a corollary of a result of DeVos et al. (*J. Comb. Theory, Ser. B* 2004) and a result of Gavoille and Labourel (*ESA* 2007)) and the number of edges in subgraph-universal graphs (previously $O(n^{3/2})$ by Babai et al. (*Theory and Practice of Combinatorics* 1982)).

[♀]School of Computer Science and Electrical Engineering, University of Ottawa, Ottawa, Canada (vida.dujmovic@uottawa.ca). Research supported by NSERC and a University of Ottawa Research Chair.

[♁]LaBRI, University of Bordeaux, France (gavoille@labri.fr). This work is supported by the French ANR Projets ENEDISC (ANR-24-CE48-7768-01) and TEMPOGRAL (ANR-22-CE48-0001).

[♂]Département d'Informatique, Université libre de Bruxelles, Belgium (gwenael.joret@ulb.be). G. Joret is supported by the Belgian National Fund for Scientific Research (FNRS) and by the Australian Research Council.

[♃]Department of Theoretical Computer Science, Faculty of Mathematics and Computer Science, Jagiellonian University, Kraków, Poland (piotr.micek@uj.edu.pl). Research supported by the National Science Center of Poland under grant UMO-2023/05/Y/ST6/00079 within the WEAVE-UNISONO program.

[♄]School of Computer Science, Carleton University, Ottawa, Canada (morin@scs.carleton.ca). Research supported by NSERC.

[♅]School of Mathematics, Monash University, Melbourne, Australia (david.wood@monash.edu). Research supported by the Australian Research Council and by NSERC.

Contents

1 Introduction **1**

1.1 New Results 2

1.2 Relation to Recent Work 4

2 Building blocks **6**

2.1 Trees and Forests 6

2.2 Forest-Decompositions and Tree-Decompositions 7

2.3 Graph Product Structure 7

2.4 Weight Functions 7

2.5 On Multipart Labels 8

2.6 Shannon and Shannon–Fano–Elias Codes 9

2.7 Working with Tree Decompositions 11

3 Weighted Mixed Labelling Schemes **17**

3.1 Basic Properties 17

3.2 Orientable Labelling Schemes 19

3.3 Adding Apex Vertices 21

3.4 Tree-Decompositions: Combining via Clique-Sums 26

4 Proof of Theorems 1, 3 and 4 and Corollary 2 with Explicit Bounds **36**

5 Discussion **37**

A A Mixed Labelling Scheme for $\mathcal{R}_k$ **42**

1 Introduction

Let $\mathcal{G}$ be a graph class (i.e., a set of graphs closed under isomorphism). A graph U_n is *n-universal* for $\mathcal{G}$ if it "contains" every n-vertex graph in $\mathcal{G}$. Here, the word "contains" can take on at least two meanings. A graph U_n is *n-subgraph-universal* for $\mathcal{G}$ if every n-vertex graph in $\mathcal{G}$ is isomorphic to some subgraph of U_n. A graph U_n is *n-induced-universal* for $\mathcal{G}$ if every n-vertex graph in $\mathcal{G}$ is isomorphic to some *induced* subgraph of U_n. These two notions of universality have largely been treated as separate concepts evaluated by different quality measures. A notable exception is an early work by Chung [7] that we discuss below.

Clearly, an n-subgraph-universal graph must have at least n vertices, and the complete graph K_n contains every n-vertex graph as a subgraph. Thus, for every positive integer n, K_n is an n-vertex graph that is n-subgraph-universal for the set of all graphs. Consequently, research on subgraph-universal graphs has focused on minimizing the number of edges, especially when $\mathcal{G}$ is a sparse class of graphs, such as trees, planar graphs, or K_t-minor-free graphs for some fixed t. This line of research has a long and interesting history, dating back to the work of Chung, Graham, and Pippenger [8] in 1978. Notable results include subgraph-universal graphs with $O(n \log n)$ edges for trees and outerplanar graphs [7, 8, 27], $O(n)$ edges for bounded-degree trees [3], $O(kn \log(n/k+1))$ edges for graphs of treewidth at most k [27], and $n^{1+o(1)}$ edges for planar graphs [17].

In the induced-universal setting, the vertex-disjoint union of all n-vertex graphs in $\mathcal{G}$ is n-induced-universal, but this graph generally has an enormous number of vertices.[1] Thus, research on induced-universal graphs has focused on finding graphs with as few vertices as possible. The subject of induced-universal graphs also has a long history, especially in computer science where it appears under the notion of adjacency labelling schemes (discussed below). Notable results include induced-universal graphs with $O(n)$ vertices for trees [1], $n(\log(n/k))^{O(k)}$ vertices for graphs of treewidth at most k [18], and a series of results leading to induced-universal graphs for planar graphs having $n \cdot \exp(O(\sqrt{\log n}))$ vertices [5, 7, 11, 18, 19, 26, 33].

A 1990 work by Chung [7] gave several theorems about induced-universal graphs that simultaneously bound both the number of vertices and the number of edges. This work showed the existence of n-induced-universal graphs for forests with $O(n \log n)$ vertices and $O(n^2)$ edges. Using the fact that the edges of a planar graph can be partitioned into three forests, she obtained n-induced-universal graphs for planar graphs with $O((n \log n)^3)$ vertices and $O(n^6)$ edges. For the next three decades, however, most research either focused on finding subgraph-universal graphs with few edges or on finding induced-universal graphs with few vertices.

This dichotomy persisted until 2023, when Esperet et al. [17] showed that, for the class of planar graphs, there exists an n-induced-universal graph with $n^{1+o(1)}$ vertices *and edges*. Thus, their result simultaneously satisfies the stronger requirement of being induced-universal while (nearly) optimizing both quality measures. Their work is based

[1] More precisely, let $\mathcal{G}_n$ be the set of all n-vertex graphs in $\mathcal{G}$. The universal graph is constructed by choosing one representative graph G from each isomorphism class of $\mathcal{G}_n$, and then taking the vertex-disjoint union of all such graphs.

on a construction of n-induced-universal graphs with $n^{1+o(1)}$ vertices for planar graphs due to Dujmović et al. [11]. In the current paper, we extend these results to all proper minor-closed graph classes.

1.1 New Results

As mentioned above, induced-universal graphs are closely related to adjacency labelling schemes. The results in this paper and the techniques used to obtain them are best described using the language of adjacency labelling schemes, which we now define. Let $\mathcal{G}$ be a class of graphs and let $f : \mathbb{N}^+ \to \mathbb{R}^+$ be a function (where $\mathbb{N}^+$ and $\mathbb{R}^+$ denote the sets of positive integers and positive real numbers, respectively). We say that $\mathcal{G}$ admits an *$f(n)$-bit adjacency labelling scheme* if there exists a function $A : (\{0,1\}^*)^2 \to \{0,1\}$ (called an *adjacency tester*) such that for all positive integers n, for every n-vertex graph $G \in \mathcal{G}$, there exists a function $\ell : V(G) \to \{0,1\}^*$ such that $|\ell(v)| \leqslant f(n)$ for each vertex v in G (where $|\alpha|$ denotes the length of a binary string α), and such that for every two vertices u, v in G,

$$A(\ell(u), \ell(v)) = \begin{cases} 0 & \text{if } uv \notin E(G), \\ 1 & \text{if } uv \in E(G). \end{cases}$$

(Such a function ℓ is called an *adjacency labelling* of G for A.)

A graph H is a *minor* of a graph G if a graph isomorphic to H can be obtained from a subgraph of G by contracting edges. A class of graphs $\mathcal{G}$ is *minor-closed* if every minor of every graph in $\mathcal{G}$ is also in $\mathcal{G}$, and it is *proper* if it is not the class of all graphs. Examples of proper minor-closed graph classes include planar graphs, graphs of treewidth at most k, graphs embeddable in a fixed surface, linklessly embeddable graphs, knotlessly embeddable graphs, and, for every fixed graph X, the class of graphs with no X-minor. Note that every proper minor-closed class is contained in the class of K_t-minor-free graphs for some fixed t. In this paper, we prove the following result. (All logarithms are in base 2.)

Theorem 1. *Every proper minor-closed class of graphs admits a $(1 + o(1))\log n$-bit adjacency labelling scheme.*

Note that all the dependence on the fixed proper minor-closed class in Theorem 1 is in the $o(\log n)$ term. Furthermore, Theorem 1 is optimal up to the lower-order term; in our construction, this $o(\log n)$ term is $O\big((\log n)^{3/4}\big)$. The proof of Theorem 1 is constructive: for every fixed proper minor-closed class $\mathcal{G}$, there is a polynomial-time algorithm that takes a graph $G \in \mathcal{G}$ as input and constructs the labelling $\ell : V(G) \to \{0,1\}^*$. We prove Theorem 1 by introducing and studying a strengthening of adjacency labelling schemes called (weighted) mixed labelling schemes.

An adjacency labelling scheme for a class of graphs $\mathcal{G}$ that uses an adjacency tester A can be used to construct an n-induced-universal graph for $\mathcal{G}$ having $O(2^{f(n)})$ vertices, as follows: We take U_n to be the graph with vertex set $V(U_n) := \{0,1\}^{\leqslant f(n)}$ (the set of binary strings of length at most $f(n)$) and edge set $E(U_n) := \{\alpha\beta : \alpha, \beta \in V(U_n),\ \alpha \neq \beta,\ \text{and } A(\alpha,\beta) = 1\}$. For any n-vertex graph $G \in \mathcal{G}$, there exists an adjacency labelling $\ell_G : V(G) \to \{0,1\}^{\leqslant f(n)}$ of G for A. Then the subgraph $U_n[\{\ell_G(v) : v \in V(G)\}]$ of U_n induced by $\{\ell_G(v) : v \in V(G)\}$ is

isomorphic to G.[2] Thus, Theorem 1 implies the existence of induced-universal graphs with a near-linear number of vertices for n-vertex graphs belonging to a fixed proper minor-closed class of graphs.

Corollary 2. *For every proper minor-closed graph class $\mathcal{G}$, for every positive integer n, there exists a graph U_n with $n^{1+o(1)}$ vertices such that every n-vertex graph in $\mathcal{G}$ is isomorphic to an induced subgraph of U_n.*

Prior to the current work, the best result for adjacency labelling of X-minor-free graphs (even for $X = K_6$) required labels of length $(2 + o(1))\log n$, leading to n-induced-universal graphs with $n^{2+o(1)}$ vertices. (This result is obtained by combining a $(1+o(1))\log n$-bit adjacency labelling scheme for bounded-treewidth graphs due to Gavoille and Labourel [18] and a result of DeVos, Ding, Oporowski, Sanders, Reed, Seymour, and Vertigan [10] establishing that the edges of every X-minor-free graph can be partitioned into two sets, each of which defines a graph of bounded treewidth.)

The proof of Theorem 1 makes use of a variant of the Robertson-Seymour Graph Minor Structure Theorem, which describes any graph in a proper minor-closed family in terms of a tree-decomposition. To make this work, we resolve the problem of "combining adjacency-labellings along tree-decompositions" identified by Bonamy et al. [5] as the main difficulty in obtaining efficient adjacency-labelling schemes for proper minor-closed graph classes. We do this by introducing and studying a generalization of adjacency labelling schemes called mixed labelling schemes.

The adjacency labelling scheme we use to prove Theorem 1 actually satisfies some additional conditions that not only imply Corollary 2, but also ensure that the universal graph U_n is in fact sparse:

Theorem 3. *For every proper minor-closed graph class $\mathcal{G}$, for every positive integer n, there exists a graph U_n with $n^{1+o(1)}$ vertices and $n^{1+o(1)}$ edges such that every n-vertex graph in $\mathcal{G}$ is isomorphic to an induced subgraph of U_n.*

Prior to the current work, the best result for n-subgraph-universal graphs for H-minor-free graphs was a construction having $O(n^{3/2})$ vertices and edges due to Babai, Chung, Erdős, Graham, and Spencer [2], obtained using a (by now standard) application of $O(\sqrt{n})$ separators. Theorem 3 improves this quantitatively by reducing the number of vertices and edges to $n^{1+o(1)}$ and qualitatively by constructing induced-universal graphs instead of subgraph-universal graphs.

We establish Theorem 3 by introducing a notion of *orientable* (mixed or adjacency) labelling schemes and showing that any orientable adjacency labelling scheme yields an induced-universal graph that is sparse.

[2] A minor detail is that the labelling ℓ_G needs to be injective, for each $G \in \mathcal{G}$. This is the case for the adjacency labelling schemes developed in this paper. See [33, Section 2.1] for more details on the connection between adjacency labelling schemes and induced-universal graphs.

1.2 Relation to Recent Work

The current paper is the full version of the conference paper [12], which only establishes Theorem 1 and Corollary 2 using mixed labelling schemes. In order to establish Theorem 3, we define orientable mixed labelling schemes. The mixed labelling scheme in [12] is not orientable, so the mixed labelling scheme presented here is significantly different. In [12], the tree indexing a tree-decomposition is partitioned into subtrees with at most b vertices in each layer—a partition into *b-skinny subtrees* whose weights decrease by a factor of b between subtrees. This then requires a solution for graphs having b-skinny tree-decompositions and a solution for graphs having tree-decompositions of height $O(\log_b n)$ whose torsos themselves have b-skinny tree-decompositions. The mixed labelling scheme for graphs having b-skinny tree-decompositions in [12] is not orientable, and there seems to be no easy way to fix this.

The current paper avoids this two-layer approach and thus avoids having to deal specifically with b-skinny tree-decompositions. Instead, the proof of Lemma 18 (the technical core of the current paper) performs a partition into subtrees each having $O(b)$ nodes with the property that, within a single subtree, the nodes have weights that differ by a factor of at most $O(b)$.

Part of our proof requires that we devise an orientable mixed labelling scheme for a generalization of planar graphs. As in [17], we do this by modifying the adjacency labelling scheme in [11], which is based on the Planar Graph Product Structure Theorem [13]. The modifications we use are different from those in [17], lead to a simplification of the original adjacency labelling scheme from [11], and improve the $n^{o(1)}$ factor obtained in [17]. Since graph products can be used to describe a variety of minor-closed and non-minor-closed graph classes, we state our general result for graph products as a separate theorem. Let $\mathcal{R}_k$ denote the class of graphs G for which there exists a graph H of treewidth at most k and a path P such that G is isomorphic to a subgraph of the strong graph product $H \boxtimes P$.

Theorem 4. *For all integers $k, n \geqslant 1$, there exists a graph U_n^k with $n \cdot \exp(O(k\sqrt{\log n}))$ vertices and edges such that every n-vertex graph in $\mathcal{R}_k$ is isomorphic to an induced subgraph of U_n^k.*

The proof of Theorem 4 combines the adjacency labelling scheme of Dujmović et al. [11] and its improvement by Gawrychowski and Janczewski [19] with logarithmic-height binary-tree decompositions discovered by several groups in the late 1990s and early 2000s in the context of parallel algorithms [4], reachability queries [6], and space-efficient algorithms [25].

Organization of the Paper The remainder of the paper is organized as follows:

- Section 2 reviews some basic definitions and supporting results including treewidth, tree- and forest-decompositions, strong products, and prefix-free codes. This section also introduces the Graph Minor Product Structure Theorem (Theorem 5) of Dujmović et al. [13] and then finishes by developing some tools for manipulating the tree-decompositions that come out of this theorem.
- Section 3 begins by introducing weighted mixed labelling schemes and definitions

of orientability for both adjacency labelling and mixed labelling schemes. Mixed labelling schemes are a generalization of adjacency labelling schemes that also satisfy some additional requirements. In order to solve the problem of combining schemes along tree-decompositions, we need these schemes to support vertex-weighted graphs. In order to obtain sparse induced-universal graphs, we also need these schemes to be orientable. As a result of all these requirements, the complete definition of an efficient orientable weighted mixed labelling scheme is somewhat involved.

After these definitions, this section proves a relatively easy result (Lemma 17) that applies to any graph family $\mathcal{G}^{+a}$ whose members can be derived from a graph G in a graph family $\mathcal{G}$ via the addition of at most a apex vertices. Specifically, it shows that if $\mathcal{G}$ admits an efficient orientable weighted mixed labelling scheme then $\mathcal{G}^{+a}$ also admits an efficient orientable weighted mixed labelling scheme. Although this result and its proof are not difficult, the proof is not particularly short; verifying that all aspects of the definition are satisfied takes some time. This is a useful warm-up for what comes next.

Next is the main technical core of the paper, Lemma 18, which establishes that the class $\mathcal{G}'$ of graphs having tree-decompositions whose torsos come from a graph family $\mathcal{G}$ that admits an efficient orientable weighted mixed labelling scheme admits an efficient orientable adjacency labelling scheme. Any such class $\mathcal{G}'$ therefore has n-induced-universal graphs with $n^{1+o(1)}$ vertices and edges.

- Section 4 combines the ingredients from the previous two sections with an efficient orientable weighted mixed labelling scheme for the class $\mathcal{R}_k$ of product-structured graphs (Lemma 19) in order to prove the main results of the paper, Theorems 1 and 3 and Corollary 2. Briefly: for any proper minor-closed graph family $\mathcal{G}$, Theorem 5 states that each graph in $\mathcal{G}$ has a tree-decomposition whose torsos belong to $\mathcal{R}_k^{+a}$, for some fixed k and a. By Lemmas 17 and 19, $\mathcal{R}_k^{+a}$ admits an efficient orientable weighted mixed labelling scheme. By Lemma 18, $\mathcal{G}$ admits an efficient orientable adjacency labelling scheme. The resulting adjacency labelling scheme implies Theorem 1 and Corollary 2 and the fact that this scheme is orientable implies Theorem 3.

- Appendix A establishes Lemma 19, the existence of an efficient orientable weighted mixed labelling scheme for $\mathcal{R}_k$. In addition to its application in proving Theorems 1 and 3, Lemma 19 immediately implies Theorem 4. This result is left until the appendix because it requires a complete review and simplification of the adjacency labelling scheme in Dujmović et al. [11] and its improvement by Gawrychowski and Janczewski [19].

  At the end of this appendix, we point out that this simplification leads to n-induced-universal graphs for the class of graphs of treewidth at most k having $n(\log n)^{O(k)}$ vertices and edges. This improves the previous best known bound (from [17]) of $n(\log n)^{O(k^2)}$.

- Section 5 concludes with a brief discussion.

2 Building blocks

Let $\mathbb{N}$ be the set of all nonnegative integers, let $\mathbb{N}^+$ be the set of all positive integers, let $\mathbb{R}$ be the set of all real numbers, let $\mathbb{R}^+$ be the set of all positive real numbers, and let $\mathbb{R}^{\geqslant 0}$ be the set of all nonnegative real numbers. For $p \in \mathbb{N}$, let $[p] := \{1, \ldots, p\}$ (with $[0] := \emptyset$). We denote by $\{0,1\}^*$ the set of all finite binary strings, by $|\alpha|$ the length (number of bits) of a binary string $\alpha \in \{0,1\}^*$, by ε the empty string (the unique binary string of length 0), and, for $x \in \mathbb{R}^{\geqslant 0}$, by $\{0,1\}^{\leqslant x}$ the set of binary strings of length at most x.

In this paper, each graph G we consider is a simple graph with vertex set denoted by $V(G)$ and edge set denoted by $E(G)$. For an edge of G with endpoints v and w, we use the shorthand notation vw instead of $\{v,w\}$ for the edge, with occasional exceptions. For a graph G and a vertex v of G, $N_G(v) := \{w \in V(G) : vw \in E(G)\}$ is the *(open) neighbourhood* of v in G; for a subset $X \subseteq V(G)$, $N_G(X) := (\bigcup_{v \in X} N_G(v)) \setminus X$. For any set S, $G[S]$ is the graph with vertex set $V(G[S]) := S \cap V(G)$ and edge set $E(G[S]) := \{vw \in E(G) : v, w \in S\}$, which we call the subgraph of G *induced* by $V(G) \cap S$. For any set S, $G - S := G[V(G) \setminus S]$; for a single vertex v, we write $G - v$ as shorthand for $G - \{v\}$. For any $R \subseteq E(G)$, $G - R$ is the graph with vertex set $V(G)$ and edge set $E(G) \setminus R$. A non-empty subset K of vertices of G is a *clique* in G if $vw \in E(G)$ for each $\{v,w\} \in \binom{K}{2}$.

A *graph class* is a set of graphs closed under isomorphism. A graph class $\mathcal{G}$ is *monotone* if, for every $G \in \mathcal{G}$, every subgraph of G is also in $\mathcal{G}$. For a graph class $\mathcal{G}$ and a nonnegative integer a, define $\mathcal{G}^{+a}$ as the class of graphs G such that $G - X \in \mathcal{G}$ for some $X \subseteq V(G)$ with $|X| \leqslant a$ (the vertices in X are sometimes called *apex vertices*). Note that if $\mathcal{G}$ is monotone, then $\mathcal{G}^{+a}$ is also monotone.

2.1 Trees and Forests

For a rooted tree T, we denote the root of T by $r(T)$. For a node x of T, define $P_T(x)$ to be the path in T from $r(T)$ to x. A *forest* F is a graph (possibly disconnected) each of whose components is a tree. If each component of a forest F is a rooted tree, then F is a *rooted* forest. For a node x in a rooted forest F, let $P_F(x) := P_T(x)$, where T is the component of F that contains x.

Let F be a rooted forest. For each $x \in V(F)$, the *F-depth* of x, denoted $\mathrm{depth}_F(x)$, is the number of edges in $P_F(x)$. The *height* of F, denoted $\mathrm{height}(F) := \max_{x \in V(F)} \mathrm{depth}_F(x)$, is the maximum F-depth of a node in F. For each $y \in V(F)$ and each $x \in V(P_F(y))$ (including y), the node x is an *F-ancestor* of y and y is an *F-descendant* of x. For each $x \in V(F)$, F_x denotes the subtree of F induced by all F-descendants of x. For each edge xy of F, where x is an F-ancestor of y, we say that x is the *F-parent* of y, denoted $\mathrm{p}_F(y)$, and y is an *F-child* of x. We say that a set $S \subseteq V(F)$ is an *F-antichain* if there are no distinct $x, y \in S$ such that x is an F-ancestor of y. (When there is no danger of ambiguity, we may drop the leading F- from F-depth, F-ancestor, F-descendant, F-parent, and F-child, and write $\mathrm{p}(y)$ for $\mathrm{p}_F(y)$.) We treat each subgraph $F' \subseteq F$ as a rooted forest in which each component T of F' is rooted at the node $r(T)$ of T having minimum F-depth.

For a rooted tree T and any non-empty set $S \subseteq V(T)$, the *lowest common T-ancestor* of S, denoted $\mathrm{lca}_T(S)$, is the node in $\bigcap_{x \in S} V(P_T(x))$ of maximum T-depth (for two nodes

$x, y \in V(T)$, we also write $\mathrm{lca}_T(x,y)$ for $\mathrm{lca}_T(\{x,y\})$).

2.2 Forest-Decompositions and Tree-Decompositions

It will be convenient to work briefly with forest-decompositions, which are the natural generalization of tree-decompositions, allowing the graph indexing the bags to be a forest. A *forest-decomposition* of a graph G is a pair $\mathcal{F} := (F, (B_x \mid x \in V(F)))$, where F is a forest and $(B_x \mid x \in V(F))$ is a collection of subsets of $V(G)$ indexed by the nodes of F such that (i) for each edge vw of G, there exists $x \in V(F)$ with $v, w \in B_x$; and (ii) for each vertex v of G, $F[\{x \in V(F) \mid v \in B_x\}]$ is non-empty and connected. The sets B_x are called the *bags* of $\mathcal{F}$. When F is a tree, $\mathcal{F}$ is a *tree-decomposition*. The *width* of a tree-decomposition $(T, (B_x \mid x \in V(T)))$ is $\max_{x \in V(T)} |B_x| - 1$. The *treewidth* of a graph G, denoted $\mathrm{tw}(G)$, is the minimum width of a tree-decomposition of G.

A forest-decomposition is *rooted* if it is indexed by a rooted forest. For each edge xy of F, $B_x \cap B_y$ is called an *adhesion* of $\mathcal{F}$, and we say that this adhesion is incident to x and y. For each $x \in V(F)$, the graph $G\langle \mathcal{F}, B_x \rangle$ obtained from $G[B_x]$ by adding edges so that each adhesion incident to x becomes a clique is called the *torso* of $\mathcal{F}$ at x. For a root node $x \in V(F)$, the *parent adhesion* of x is the empty set. For a non-root node $y \in V(F)$, the *parent adhesion* of y is $B_{\mathrm{p}_F(y)} \cap B_y$. When $\mathcal{F}$ is rooted, the *home* (or *home node*) of a vertex $v \in V(G)$ is the unique node $x \in V(F)$ such that $v \in B_x$ and v is not in the parent adhesion of x. The *adhesion-width* of $\mathcal{F}$ is the size of its largest adhesion (or zero if $\mathcal{F}$ has no edges).

2.3 Graph Product Structure

The *strong product* of graphs A and B, denoted by $A \boxtimes B$, is the graph with vertex set $V(A) \times V(B)$, where distinct vertices $(v,x), (w,y) \in V(A) \times V(B)$ are adjacent if $v = w$ and $xy \in E(B)$, or $x = y$ and $vw \in E(A)$, or $vw \in E(A)$ and $xy \in E(B)$. For an integer $k \geqslant 0$, let $\mathcal{R}_k$ be the class of graphs isomorphic to a subgraph of $H \boxtimes P$ for some graph H of treewidth at most k and some path P. Our starting point is the following variant of the Graph Minor Structure Theorem:

Theorem 5 (Graph Minor Product Structure Theorem [13])**.** *For every proper minor-closed class $\mathcal{G}$, there exist integers $k, a \geqslant 0$ such that every graph in $\mathcal{G}$ has a tree-decomposition in which every torso is in $\mathcal{R}_k^{+a}$.*

2.4 Weight Functions

For a non-empty set S, a *pseudoweight function* over S is a function $\omega : S \to \mathbb{R}^{\geqslant 0}$. A *weight function* is a pseudoweight function that assigns strictly positive weights (i.e., $\omega(x) > 0$ for each $x \in S$). For $X \subseteq S$, we use the shorthand $\omega(X) := \sum_{x \in X} \omega(x)$. When $S := V(G)$ is the vertex-set of a graph G and H is a subgraph of G, we use the shorthand $\omega(H) := \omega(V(H))$.

Let $\omega : S \to \mathbb{R}^+$ be a weight function. For each $x \in S$, the *(Kraft) ideal codeword length* for x is $\log \omega(S) - \log \omega(x)$. It will be convenient to bound the total weight as well as the Kraft ideal codeword lengths. A weight function $\omega : S \to \mathbb{N}^+$ is *nice* if $\omega(S) \leqslant 2|S|$. In particular, a nice weight function satisfies

$$\log \omega(S) - \log \omega(x) \leqslant \log |S| + 1$$

for every $x \in S$.

Observation 6. *For every weight function $\omega : S \to \mathbb{R}^+$, there exists a nice weight function $\omega' : S \to \mathbb{N}^+$ such that $\log \omega'(S) - \log \omega'(x) \leqslant \min\{\log |S|,\ \log \omega(S) - \log \omega(x)\} + 1$, for each $x \in S$. If ω is positive integer-valued, then $\omega'(x) \leqslant \omega(x)$ for each $x \in S$, and hence $\omega'(S) \leqslant \omega(S)$.*

Proof. Let $m := |S|$ and $W := \omega(S)$. For each $x \in S$, define

$$\omega'(x) := \left\lceil \frac{m\omega(x)}{W} \right\rceil \ .$$

Then $\omega'(x) \geqslant 1$ for each $x \in S$ and

$$m \leqslant \omega'(S) < \sum_{x \in S} \left(\frac{m\omega(x)}{W} + 1 \right) = 2m \ ,$$

so ω' is nice. Since $\omega'(x) \geqslant 1$ and $\omega'(x) \geqslant m\omega(x)/W$, we obtain

$$\frac{\omega'(S)}{\omega'(x)} \leqslant \min\left\{ 2m,\ \frac{2W}{\omega(x)} \right\} \ .$$

Taking logarithms proves the claimed bound. Finally, if ω is positive integer-valued, then $W \geqslant m$, so $m\omega(x)/W \leqslant \omega(x)$. Since $\omega(x)$ is an integer, $\omega'(x) = \lceil m\omega(x)/W \rceil \leqslant \omega(x)$ for each $x \in S$. □

2.5 On Multipart Labels

In many cases, a label (a bitstring) consists of a sequence of parts (also bitstrings) of varying lengths, $s_1, \ldots, s_p$, where each s_i may be empty. The naive concatenation of $s_1, \ldots, s_p$ has length

$$\textstyle\sum_{i=1}^{p} |s_i| \ .$$

This ignores the important issue that, in order for a decoder to extract the individual parts $s_1, \ldots, s_p$, it must know the number of parts p and (at least $p-1$ of) the lengths $|s_1|, \ldots, |s_p|$. We handle this by encoding the value of p and the lengths $|s_1|, \ldots, |s_p|$ and prepending these to the concatenation. This is most easily done using Elias' γ-code $\gamma : \mathbb{N}^+ \to \{0,1\}^*$ for positive integers, in which each $x \in \mathbb{N}^+$ is encoded by a self-delimiting binary string $\gamma(x)$ of length $|\gamma(x)| = 2\lfloor \log x \rfloor + 1$ [16].[3] In our application, each part s_i may have zero length and p may even be equal to zero, so we encode each $x \in \mathbb{N}$ using $\gamma(x+1)$, which has length $2\lfloor \log(x+1) \rfloor + 1$. In this way, we obtain a self-delimiting encoding of $s_1, \ldots, s_p$ as

$$\langle s_1, \ldots, s_p \rangle := \gamma(p+1) \circ \gamma(|s_1|+1) \circ \cdots \circ \gamma(|s_p|+1) \circ s_1 \circ \cdots \circ s_p \ ,$$

where $\circ$ denotes string concatenation (and $\langle\rangle := \gamma(1)$ when $p = 0$). This encoding has length

$$|\langle s_1, \ldots, s_p \rangle| = \textstyle\sum_{i=1}^{p} |s_i| + O(\hat{\lg} p) + O(\sum_{i=1}^{p} \hat{\lg} |s_i|) \ , \tag{1}$$

[3] For each $x \in \mathbb{N}^+$, $\gamma(x)$ consists of $\lfloor \log x \rfloor$ 0 bits, followed by the binary representation of x (which begins with a 1 bit and has length $\lfloor \log x \rfloor + 1$).

where $\hat{\lg} x := \log \max\{2, x\}$ (so that $\hat{\lg} x$ is defined and $\hat{\lg} x \geqslant 1$ for each $x \in \mathbb{N}$). The bits of $\langle s_1, \ldots, s_p \rangle$ that come from $s_1, \ldots, s_p$ are called the *payload* bits and the remaining bits are called the *bookkeeping* bits of $\langle s_1, \ldots, s_p \rangle$. In our setting it will always be the case that $p \in o(\log n)$ and we will use Observation 6 to ensure that $|s_i| \in O(\log n)$ for each $i \in [p]$. Thus, the number of bookkeeping bits in $\langle s_1, \ldots, s_p \rangle$ is $O(1 + p \log\log n)$.

In some cases, we include one or more non-negative integers as parts of a multipart label. When we include $d \in \mathbb{N}$ as part of a label, we include its $\lceil \log(d+1) \rceil$-bit binary representation $\mathrm{bin}(d)$ (so $\mathrm{bin}(0) = \varepsilon$).

2.6 Shannon and Shannon–Fano–Elias Codes

A rooted tree T is a *binary tree* if each node of T has at most two children among which at most one is the *left child* and at most one is the *right child*. A binary tree T is *full* if for every node x of T, either x has two children or x has no children. A full binary tree T is *complete* if all the leaves of T have the same depth in T. Let T be a binary tree. There is a natural ordering, called *left-to-right ordering*, of the leaves of T, such that for every node x, if x has two children, then all the leaf-descendants of the left child of x precede all the leaf-descendants of the right child of x.

The following result is a variant of Alphabetic Binary Trees. Similar results (with the constant 3 replaced by 2) are well-known and there are several proofs [20, 30, 32]. We provide a proof of the weaker statement here because it illustrates a simple but powerful technique—*simulating weight by multiplicity*—that we will use again in Appendix A.

Lemma 7. *Let S be a non-empty linearly ordered finite set and let $\omega : S \to \mathbb{N}^+$ be a weight function. Then there exists a full binary tree T such that S is the set of leaves of T, the linear order on S agrees with the left-to-right order of the leaves in T, and for each $x \in S$,*

$$\mathrm{depth}_T(x) \leqslant \log \omega(S) - \log \omega(x) + 3 \ .$$

Proof. Let $h := \lceil \log \omega(S) \rceil$. Let Q be the complete binary tree of height h, so with 2^h leaves. For each $x \in S$, define the $\omega(x)$-element set

$$S_\omega(x) := \{(x,1), (x,2), \ldots, (x, \omega(x))\}$$

and let $S_\omega := \bigcup_{x \in S} S_\omega(x)$. Then $|S_\omega| = \omega(S)$. Treat S_ω as a linearly ordered set by using lexicographic order and rename the leftmost $\omega(S)$ leaves of Q, in order, with elements of S_ω.

Now for each $x \in S$, we will fix a node $r(x)$ in Q such that (1) all the leaves of $Q_{r(x)}$ lie in $S_\omega(x)$; and (2) $Q_{r(x)}$ contains at least $\omega(x)/4$ leaves.

Let $x \in S$. If $\omega(x) = 1$ then define $r(x) := (x,1)$, which is a leaf of Q. Clearly, (1) and (2) hold. Now suppose that $\omega(x) \geqslant 2$. Consider the lowest common Q-ancestor y of $S_\omega(x)$. Thus, y is an internal node of Q and each child of y has a leaf-descendant in $S_\omega(x)$. Since $S_\omega(x)$ is a set of consecutive leaves in Q, all leaves of the left child y' of y in Q that are in $S_\omega(x)$ are rightmost in $Q_{y'}$ and all leaves of the right child y'' of y in Q that are in $S_\omega(x)$ are leftmost in $Q_{y''}$. Let z be the child of y in Q such that Q_z has at least $\omega(x)/2$ leaves in

$S_\omega(x)$ and let S_z be the set of all such leaves. Thus, $|S_z| \geqslant \omega(x)/2$. Since the argument is completely symmetric in these cases, assume that z is the left child of y. Consider a path in Q_z starting from z and always taking an edge to the right child of the current node until we finish at a leaf. Let v be the first vertex of that path such that Q_v has all leaves in S_z. If $v = z$ then we set $r(x) := v$. Clearly (1) and (2) hold. Suppose that $v \neq z$. Let u be the parent of v. Since the set of leaves of Q_u form an interval of rightmost leaves of Q_z and since Q_u contains a leaf not in S_z, we conclude that Q_u contains all nodes in S_z as leaves. Therefore, Q_v contains at least $|S_z|/2$ vertices of S_z as leaves. We set $r(x) := v$ and again (1) and (2) hold.

Note that the set $\{r(x) \mid x \in S\}$ is a Q-antichain. Indeed, if $x, y \in S$, $x \neq y$ and $r(x)$ is a Q-ancestor of $r(y)$, then the leaves of $Q_{r(y)}$ are contained in the leaves of $Q_{r(x)}$ which by (1) implies that $Q_{r(x)}$ contains all nodes in $S_\omega(y)$ as leaves. However, again by (1) all the leaves of $Q_{r(x)}$ lie in $S_\omega(x)$ which is disjoint from $S_\omega(y)$, a contradiction.

Let $x \in S$. Since $Q_{r(x)}$ contains at least $\omega(x)/4$ leaves, $\operatorname{height}(Q_{r(x)}) \geqslant \log(\omega(x)/4) = \log \omega(x) - 2$. Finally,

$$\begin{aligned}\operatorname{depth}_Q(r(x)) &\leqslant \operatorname{height}(Q) - \operatorname{height}(Q_{r(x)}) \\ &\leqslant \lceil \log \omega(S) \rceil - (\log \omega(x) - 2) \\ &\leqslant \log \omega(S) - \log \omega(x) + 3 \ .\end{aligned}$$

Let $T_0 := \bigcup_{x \in S} P_Q(r(x))$. Since $\{r(x) \mid x \in S\}$ is a Q-antichain, we conclude that $\{r(x) \mid x \in S\}$ is the set of all leaves of T_0. Obtain T from T_0 by repeatedly contracting an edge incident to any internal node that has only one child. Then T is a full binary tree with leaf set $\{r(x) \mid x \in S\}$, and $\operatorname{depth}_T(r(x)) \leqslant \operatorname{depth}_{T_0}(r(x)) = \operatorname{depth}_Q(r(x))$ for each $x \in S$. Identifying each leaf $r(x)$ of T with x for each $x \in S$ gives the desired tree. □

A *code* for a set S is an injective function $\rho : S \to \{0,1\}^*$. A code $\rho : S \to \{0,1\}^*$ is *prefix-free* if for all distinct $x, y \in S$, $\rho(x)$ is not a prefix of $\rho(y)$. In coding theory, Lemma 7 is the basis of the Shannon–Fano–Elias coding scheme, as we now explain. By assigning each edge e of a binary tree T a 0 or 1 depending on whether e joins a parent to its left or right child, we obtain an encoding of each root-to-leaf path in T as a binary string, which gives a prefix-free code for the leaves of T. Doing this with the tree from Lemma 7, we obtain the following.

Corollary 8. *There exists a function $F : (\{0,1\}^*)^2 \to \{0,1\}^*$ such that, for every $n \in \mathbb{N}^+$, every set $S := \{x_1, \ldots, x_n\}$, and every weight function $\omega : S \to \mathbb{R}^+$, there exists a prefix-free code $\rho : S \to \{0,1\}^*$ for S and functions $\overleftarrow{\pi} : S \to \{0,1\}^*$, $\overrightarrow{\pi} : S \to \{0,1\}^*$ such that:*

(a) $|\rho(x_i)| \leqslant \log \omega(S) - \log \omega(x_i) + O(1)$ *for each* $i \in [n]$;
(b) $|\overleftarrow{\pi}(x_i)| + |\overrightarrow{\pi}(x_i)| \in O(\log \log n)$ *for each* $i \in [n]$;
(c) $F(\rho(x_i), \overleftarrow{\pi}(x_i)) = \rho(x_{i-1})$ *for each* $i \in \{2, \ldots, n\}$; *and*
(d) $F(\rho(x_i), \overrightarrow{\pi}(x_i)) = \rho(x_{i+1})$ *for each* $i \in \{1, \ldots, n-1\}$.

Proof. Let $\omega' : S \to \mathbb{N}^+$ be the nice weight function obtained by applying Observation 6 to ω. Then $\log \omega'(S) \leqslant \log n + O(1)$ and $\log \omega'(x_i) \geqslant 0$ for each $i \in [n]$. The prefix-free code ρ

is obtained by applying Lemma 7 to S (ordered by $x_1 < \cdots < x_n$) with weight function ω' to obtain a full binary tree T and then encoding each root-to-leaf path in T with a bitstring as described above. This gives a code ρ that satisfies (a).

Set $\overleftarrow{\pi}(x_1) := \varepsilon$. Fix $i \in \{2,\dots,n\}$ and let $y_0, y_1, \dots, y_\ell$ be the path from $y_0 := x_i$ to $y_\ell := x_{i-1}$ in T. Let j be such that $y_j = \mathrm{lca}_T(x_i, x_{i-1})$. Since T is a full binary tree, y_{j+1} is the left child of y_j and y_k is the right child of y_{k-1} for each $k \in \{j+2,\dots,\ell\}$. Thus, $\rho(x_{i-1})$ is the prefix of $\rho(x_i)$ of length $\mathrm{depth}_T(y_j)$, followed by a 0 bit, followed by a string of 1 bits of length $\ell - j - 1$. We set $\overleftarrow{\pi}(x_i) := \langle 0, \mathrm{bin}(j), \mathrm{bin}(\ell - j - 1)\rangle$. Since $j < \ell$ and $\ell \leqslant 2\,\mathrm{height}(T) \in O(\log n)$, $\overleftarrow{\pi}(x_i)$ has length $O(\log\log n)$. Symmetrically, set $\overrightarrow{\pi}(x_n) := \varepsilon$ and, for each $i \in \{1,\dots,n-1\}$, define $\overrightarrow{\pi}(x_i) := \langle 1, \mathrm{bin}(j), \mathrm{bin}(\ell - j - 1)\rangle$ using the path $y_0, y_1, \dots, y_\ell$ from $y_0 := x_i$ to $y_\ell := x_{i+1}$ in T, where $y_j = \mathrm{lca}_T(x_i, x_{i+1})$.

The operation of F is now obvious. Given the arguments $\rho(x_i)$ and $\langle a, \mathrm{bin}(b), \mathrm{bin}(c)\rangle$, F checks the value of $a \in \{0,1\}$. If $a = 0$, then it removes the suffix of length b from $\rho(x_i)$ and appends the string 01^c to obtain $\rho(x_{i-1})$. If $a = 1$, then F removes the suffix of length b from $\rho(x_i)$ and appends the string 10^c to obtain $\rho(x_{i+1})$. □

2.7 Working with Tree Decompositions

In this section, we develop some tools for manipulating tree-decompositions like those obtained from Theorem 5.

Lemma 9. *Let $b > 1$, let T be a rooted tree, and let $\omega : V(T) \to \mathbb{N}^+$ be a weight function. Then there exists $Y \subseteq E(T)$ such that*

(a) *each root-to-leaf path in T contains at most $\log_b \omega(T)$ edges in Y; and*
(b) *$\omega(T_y) \leqslant b \cdot \omega(T_x)$ for any two vertices x, y in the same component of $T - Y$.*

Proof. The proof is by induction on $|V(T)|$. Let $X_0 := \{x \in V(T) : \omega(T_x) \geqslant \omega(T)/b\}$. Since the root r of T satisfies $\omega(T_r) = \omega(T) > \omega(T)/b$ (as $b > 1$) and every ancestor of a vertex in X_0 is also in X_0, the induced subgraph $T_0 := T[X_0]$ is a connected subtree of T containing r. If $X_0 = V(T)$, then setting $Y := \emptyset$ satisfies the lemma: each root-to-leaf path contains $0 \leqslant \log_b \omega(T)$ edges of Y (since $\omega(T) \geqslant 1$), and for any $x, y \in V(T)$, $\omega(T_y) \leqslant \omega(T) \leqslant b \cdot \omega(T_x)$ by the definition of X_0.

Otherwise, let $F_0 := T - X_0$ and let $T_1, \dots, T_s$ be the connected components of F_0. By the definition of X_0, for any $x, y \in V(T_0)$ we have $\omega(T_y) \leqslant \omega(T) \leqslant b \cdot \omega(T_x)$. Furthermore, for each $i \in [s]$, the component T_i is the subtree T_{y_i} rooted at the child $y_i \notin X_0$ of some vertex $x_i \in V(T_0)$, so $\omega(T_i) < \omega(T)/b$. Since $r \in X_0$, each component T_i satisfies $|V(T_i)| < |V(T)|$. Applying induction to each T_i with weight function $\omega|_{V(T_i)}$ yields an edge set $Y_i \subseteq E(T_i)$.

Let $Y_0 := \{x_i y_i \mid i \in [s]\}$ and let $Y := Y_0 \cup \bigcup_{i=1}^{s} Y_i$. Each component C of $T - Y$ is either T_0 or a component of $T_i - Y_i$ for some $i \in [s]$. In the former case, $\omega(T_y) \leqslant b \cdot \omega(T_x)$ for all $x, y \in V(T_0)$ as established above. In the latter case, since $(T_i)_z = T_z$ for every $z \in V(T_i)$, the inductive hypothesis implies $\omega(T_y) \leqslant b \cdot \omega(T_x)$ for all $x, y \in V(C)$. This establishes condition (b).

Finally, let P be a root-to-leaf path in T. If P is contained entirely in T_0, then $E(P) \cap Y = \emptyset$, so $|E(P) \cap Y| = 0 \leqslant \log_b \omega(T)$. Otherwise, P leaves T_0 via a unique edge $x_i y_i \in Y_0$ into

some component T_i, and the suffix $P_i := P \cap T_i$ is a root-to-leaf path in T_i. By induction, $|E(P_i) \cap Y_i| \leqslant \log_b \omega(T_i)$. Therefore,

$$|E(P) \cap Y| = |E(P) \cap Y_0| + |E(P_i) \cap Y_i| \leqslant 1 + \log_b \omega(T_i) < 1 + \log_b(\omega(T)/b) = \log_b \omega(T) \ . \quad \square$$

The following lemma is a standard weighted separator theorem for trees; we include its short proof for completeness.

Lemma 10. *Let T be a tree and let $\omega : V(T) \to \mathbb{R}^{\geqslant 0}$ be a pseudoweight function. Then, for any $b \in \mathbb{N}^+$, there exists $S \subseteq V(T)$ with $|S| \leqslant b-1$ such that $\omega(C) \leqslant \omega(T)/b$ for each component C of $T - S$.*

Proof. The proof is by induction on b. If $b = 1$, then $S := \emptyset$ satisfies the requirements of the lemma since T is the only component of $T - S$. Now assume $b \geqslant 2$. Root T at an arbitrary vertex. The set $\{v \in V(T) : \omega(T_v) \geqslant \omega(T)/b\}$ contains the root of T and is therefore non-empty. Let x be a vertex in this set of maximum T-depth. Then $\omega(T_x) \geqslant \omega(T)/b$, while $\omega(T_y) < \omega(T)/b$ for each child y of x (if $\omega(T) = 0$, x is a leaf and this holds vacuously).

The components of $T - x$ consist of the subtrees T_y for each child y of x (each having weight $\omega(T_y) < \omega(T)/b$), and possibly the component $T_0 := T - V(T_x)$ containing the parent $\mathrm{p}(x)$ of x. If $\omega(T_0) \leqslant \omega(T)/b$ (or if T_0 does not exist), then $S := \{x\}$ satisfies the lemma since $|S| = 1 \leqslant b-1$ and every component of $T - x$ has weight at most $\omega(T)/b$.

Otherwise, $\omega(T_0) > \omega(T)/b$. Since $\omega(T_x) \geqslant \omega(T)/b$, we have $\omega(T_0) = \omega(T) - \omega(T_x) \leqslant (1 - 1/b)\omega(T) = \frac{b-1}{b}\omega(T)$. Applying induction to the tree T_0 with pseudoweight function $\omega|_{V(T_0)}$ and parameter $b-1$ yields a set $S_0 \subseteq V(T_0)$ of size at most $b-2$ such that each component C of $T_0 - S_0$ satisfies $\omega(C) \leqslant \frac{\omega(T_0)}{b-1} \leqslant \frac{\omega(T)}{b}$.

We define $S := S_0 \cup \{x\}$, so $|S| \leqslant (b-2)+1 = b-1$. Each component of $T - S$ is either a component of $T_0 - S_0$ (which has weight at most $\omega(T)/b$ by the inductive hypothesis) or one of the subtrees T_y for a child y of x (which has weight $\omega(T_y) < \omega(T)/b$). Thus, S satisfies the requirements of the lemma. $\square$

Lemma 11. *Let G be an n-vertex graph, with $n \in \mathbb{N}^+$, and let $\mathcal{T}$ be a tree-decomposition of G. Then, for any integer $b \geqslant 2$, there exists a rooted tree-decomposition $\mathcal{Q} := (Q, (C_x \mid x \in V(Q)))$ of G and $Y \subseteq E(Q)$ with the following properties:*

(a) *each root-to-leaf path in Q contains at most $\log_b n$ edges of Y;*
(b) *each connected component of $Q - Y$ has at most $4b-1$ vertices;*
(c) *each adhesion of $\mathcal{Q}$ is contained in the union of at most two adhesions of $\mathcal{T}$; and*
(d) *for each $x \in V(Q)$, at least one of the following holds:*
   - *C_x is contained in the union of at most three adhesions of $\mathcal{T}$; or*
   - *$G\langle \mathcal{Q}, C_x \rangle - A$ is a subgraph of some torso of $\mathcal{T}$, where $A = \emptyset$ or A is an adhesion of $\mathcal{T}$.*

Proof. The proof is by induction, with a generalized inductive hypothesis. An inductive instance $(G, \mathcal{T}, \alpha, \beta)$ involves a graph G, a tree-decomposition $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ of G, and two (not necessarily distinct) leaves α, β of T such that each of B_α and B_β equals the adhesion of $\mathcal{T}$ incident to α and β respectively, and $|V(G) \setminus (B_\alpha \cup B_\beta)| \geqslant 1$. We will produce

a rooted tree-decomposition $\mathcal{Q} := (Q, (C_x \mid x \in V(Q)))$ of G and $Y \subseteq E(Q)$ that satisfies all the requirements of the lemma as well as the following conditions, the first of which is a strengthening of (a):

(e) each root-to-leaf path in Q contains at most $\log_b(n - |B_\alpha \cup B_\beta|)$ edges of Y;
(f) $\alpha \in V(Q)$ and α is the root of Q; and
(g) $C_\alpha = B_\alpha \cup B_\beta$.

The induction is on the quantity $n - |B_\alpha \cup B_\beta| = |V(G) \setminus (B_\alpha \cup B_\beta)|$. To see that this suffices to establish the lemma for an n-vertex graph G, we can attach two new leaves α and β to T to obtain T', set $B_\alpha := B_\beta := \emptyset$ (so their incident adhesions are empty), and apply the refined result to $(G, (T', (B_x \mid x \in V(T'))), \alpha, \beta)$ to obtain the desired rooted tree-decomposition $\mathcal{Q}$ and edge set Y.

We now proceed with the inductive proof. Root T at α so that $\mathcal{T}$ is a rooted tree-decomposition of G. For each $x \in V(T) \setminus \{\alpha\}$, let A_x be the parent adhesion of x in $\mathcal{T}$, and set $A_\alpha := \emptyset$. Define the pseudoweight function $\omega : V(T) \to \mathbb{N}$ by $\omega(x) := |B_x \setminus (A_x \cup B_\alpha \cup B_\beta)|$. Since each vertex in $V(G) \setminus (B_\alpha \cup B_\beta)$ belongs to $B_x \setminus A_x$ for exactly one node $x \in V(T)$, we have $\omega(T) = n - |B_\alpha \cup B_\beta|$.

Apply Lemma 10 to the tree T with pseudoweight function ω to obtain a set S_0 of at most $b-1$ nodes of T such that $\omega(C) \leqslant \omega(T)/b = (n - |B_\alpha \cup B_\beta|)/b$ for each component C of $T - S_0$. Let T' be the vertex-minimal subtree of T that spans $S_0 \cup \{\alpha, \beta\}$. Let S_1 be the set of nodes of T' that have degree at least 3 in T'. Then $|S_1| \leqslant |S_0 \cup \{\alpha, \beta\}| - 2 \leqslant b - 1$. Let $S := S_0 \cup S_1 \cup \{\alpha, \beta\}$, so $|S| \leqslant 2b$. Let $C_1, \ldots, C_s$ be the connected components of $T - S$. For each $i \in [s]$, let $N_i := N_T(V(C_i))$. Since every branching node of T' belongs to $S_1 \subseteq S$, no component C_i can have three or more neighbors in S, so $|N_i| \in \{1, 2\}$ for each $i \in [s]$. For each $i \in [s]$, write $N_i := \{a_i, b_i\}$ (where $a_i = b_i$ if $|N_i| = 1$).

For each $i \in [s]$, let $G_i := G[\bigcup_{x \in V(C_i)} B_x]$ and let $m_i := |V(G_i) \setminus (B_{a_i} \cup B_{b_i})|$. Observe that $m_i \leqslant \omega(C_i) \leqslant \omega(T)/b$ for each $i \in [s]$. For each $i \in [s]$, let $\{x_i\} := N_T(a_i) \cap V(C_i)$ and $\{y_i\} := N_T(b_i) \cap V(C_i)$ so that x_i and y_i are the unique neighbours in C_i of a_i and b_i, respectively (possibly $x_i = y_i$). Since $\mathcal{T}$ is a tree-decomposition, $B_{a_i} \cap V(G_i) = B_{a_i} \cap B_{x_i}$ and $B_{b_i} \cap V(G_i) = B_{b_i} \cap B_{y_i}$.

We construct a tree Q_0 as follows: $V(Q_0)$ contains all nodes of S. For each $i \in [s]$ such that $|N_i| = 2$, we add a node v_i and edges $a_i v_i$ and $b_i v_i$ to Q_0. For each edge $xy \in E(T)$ with $x, y \in S$, we add the edge xy to Q_0. Since each component C_i with $|N_i| = 2$ corresponds to a path in T' between two nodes in S, Q_0 is a tree with $V(Q_0) \supseteq S$ and $|V(Q_0)| \leqslant 2|S| - 1 \leqslant 4b - 1$. We root Q_0 at α.

For each $x \in S$, set $C_x := B_x$. For each $i \in [s]$ with $|N_i| = 2$, set $C_{v_i} := (B_{a_i} \cup B_{b_i}) \cap V(G_i)$. To ensure property (g), we add the vertices in B_β to C_x for each x on the path from β to α in Q_0. This completes the definition of the bags C_x for all $x \in V(Q_0)$. Then $\mathcal{Q}_0 := (Q_0, (C_x \mid x \in V(Q_0)))$ is a tree-decomposition of $G_S := G[\bigcup_{x \in S} B_x]$.

If $m_i = 0$, then $V(G_i) \subseteq B_{a_i}$ when $|N_i| = 1$, and $V(G_i) \subseteq C_{v_i}$ when $|N_i| = 2$. Thus all vertices and edges of G_i are covered by a single bag of $\mathcal{Q}_0$, and no recursive call is needed for this component. For each $i \in [s]$ such that $m_i > 0$, we construct a tree-decomposition of G_i as follows.

- If $|N_i| = 1$ (so $N_i = \{a_i\}$) then $x_i \in V(C_i)$ is adjacent to a_i in T. Let T_i be the tree obtained from C_i by adding a new leaf $\alpha_i = \beta_i$ adjacent to x_i, and set $B_{\alpha_i} := B_{a_i} \cap B_{x_i}$. Then $\mathcal{T}_i := (T_i, (B_x \mid x \in V(T_i)))$ is a tree-decomposition of G_i. Apply induction to the instance $(G_i, \mathcal{T}_i, \alpha_i, \beta_i)$.
- If $|N_i| = 2$ then $x_i, y_i \in V(C_i)$ are the unique neighbours of a_i and b_i in C_i (possibly $x_i = y_i$). Let T_i be the tree obtained from C_i by adding new leaves α_i adjacent to x_i and β_i adjacent to y_i, and set $B_{\alpha_i} := B_{a_i} \cap B_{x_i}$ and $B_{\beta_i} := B_{b_i} \cap B_{y_i}$. Then $\mathcal{T}_i := (T_i, (B_x \mid x \in V(T_i)))$ is a tree-decomposition of G_i. Apply induction to the instance $(G_i, \mathcal{T}_i, \alpha_i, \beta_i)$.

In either case, induction yields a rooted tree-decomposition $\mathcal{Q}_i := (Q_i, (C_x \mid x \in V(Q_i)))$ of G_i with root α_i, and an edge set $Y_i \subseteq E(Q_i)$. By (g), the root bag satisfies $C_{\alpha_i} = B_{\alpha_i} \cup B_{\beta_i}$.

We now construct the tree Q from Q_0 and $\{Q_i \mid m_i \geqslant 1\}$. For each $i \in [s]$ such that $m_i > 0$ and $|N_i| = 1$, let $e_i := a_i\alpha_i$. For each $i \in [s]$ such that $m_i > 0$ and $|N_i| = 2$, let $e_i := v_i\alpha_i$. Then $V(Q) := V(Q_0) \cup \bigcup_{i:m_i \geqslant 1} V(Q_i)$ and $E(Q) := E(Q_0) \cup \bigcup_{i:m_i \geqslant 1} (E(Q_i) \cup \{e_i\})$. Define $Y := \bigcup_{i:m_i \geqslant 1} (Y_i \cup \{e_i\})$. Since $V(G_S) \cap V(G_i) = B_{\alpha_i} \cup B_{\beta_i} = C_{\alpha_i}$ and $C_{\alpha_i} \subseteq C_{a_i}$ (when $|N_i| = 1$) or $C_{\alpha_i} \subseteq C_{v_i}$ (when $|N_i| = 2$), $\mathcal{Q} := (Q, (C_x \mid x \in V(Q)))$ is a valid rooted tree-decomposition of G with root α, satisfying (f) and (g).

Because each edge e_i belongs to Y, the connected components of $Q - Y$ are precisely Q_0 and the connected components of $Q_i - Y_i$ for each i with $m_i \geqslant 1$. Since $|V(Q_0)| \leqslant 4b - 1$ and each component of $Q_i - Y_i$ has at most $4b - 1$ vertices by induction, property (b) holds.

Any root-to-leaf path in Q consists of a path in Q_0 followed by at most one edge e_i and a root-to-leaf path in Q_i for some i with $m_i \geqslant 1$. A path contained in Q_0 uses no edges of Y. Otherwise, since $E(Q_0) \cap Y = \emptyset$, the number of edges of Y on the path is at most:

$$1 + \log_b(|V(G_i)| - |B_{\alpha_i} \cup B_{\beta_i}|) \leqslant 1 + \log_b \omega(C_i) \leqslant 1 + \log_b((n - |B_\alpha \cup B_\beta|)/b) = \log_b(n - |B_\alpha \cup B_\beta|) \ .$$

Thus, property (e) holds.

Next, we verify property (c). For an edge $xy \in E(Q_0)$ with $x, y \in S$, the adhesion $C_x \cap C_y$ is contained in $(B_x \cap B_y) \cup B_\beta$, which is the union of at most two adhesions of $\mathcal{T}$. For an edge a_iv_i (or b_iv_i), the adhesion is contained in $(B_{a_i} \cap B_{x_i}) \cup B_\beta$, also the union of at most two adhesions of $\mathcal{T}$. For each edge e_i, the adhesion is contained in $C_{\alpha_i} = B_{\alpha_i} \cup B_{\beta_i}$, which is the union of two adhesions of $\mathcal{T}$. Finally, for an edge of Q_i, the adhesion is contained in the union of at most two adhesions of $\mathcal{T}_i$ by induction, and each adhesion of $\mathcal{T}_i$ is an adhesion of $\mathcal{T}$. Thus, property (c) holds.

Finally, we verify property (d). First we consider the torsos of $\mathcal{Q}$ at vertices $x \in S$. Let x be a vertex in S, so $x \in V(T) \cap V(Q_0)$ and $C_x \subseteq B_x \cup B_\beta$. We claim that $G\langle \mathcal{Q}, C_x \rangle - B_\beta$ is a subgraph of $G\langle \mathcal{T}, B_x \rangle - B_\beta$. Let xy be an edge of Q incident to x. There are three possibilities:

- If $y \in S$ then $C_x \subseteq B_x \cup B_\beta$ and $C_y \subseteq B_y \cup B_\beta$. Then $C_x \cap C_y \setminus B_\beta = B_x \cap B_y \setminus B_\beta$ is a clique in both $G\langle \mathcal{Q}, C_x \rangle - B_\beta$ and $G\langle \mathcal{T}, B_x \rangle - B_\beta$.
- If $y = v_i$ for some $i \in [s]$ with $|N_i| = 2$, then $x \in \{a_i, b_i\}$. Without loss of generality, suppose $x = a_i$. Then $C_x \cap C_y \setminus B_\beta = B_{a_i} \cap V(G_i) \setminus B_\beta$ is contained in the original adhesion $B_{a_i} \cap B_{x_i}$ incident to x. Thus $C_x \cap C_y \setminus B_\beta$ is a clique in both $G\langle \mathcal{Q}, C_x \rangle - B_\beta$ and $G\langle \mathcal{T}, B_x \rangle - B_\beta$.

- If $xy = e_i = a_i\alpha_i$ for some $i \in [s]$ with $|N_i| = 1$ and $m_i > 0$, then $C_x \cap C_y \setminus B_\beta = B_{a_i} \cap V(G_i) \setminus B_\beta$ is contained in the original adhesion $B_{a_i} \cap B_{x_i}$ incident to x. Thus $C_x \cap C_y \setminus B_\beta$ is a clique in both $G\langle \mathcal{Q}, C_x\rangle - B_\beta$ and $G\langle \mathcal{T}, B_x\rangle - B_\beta$.

Thus, for each $x \in S$, $G\langle \mathcal{Q}, C_x\rangle - B_\beta$ is a subgraph of $G\langle \mathcal{T}, B_x\rangle - B_\beta$, so x satisfies the second alternative of property (d), since B_β is an adhesion of $\mathcal{T}$.

For each $x \in V(Q_0) \setminus S$, $x = v_i$ for some $i \in [s]$ with $|N_i| = 2$. Then $C_x = C_{v_i} \subseteq (B_{a_i} \cap B_{x_i}) \cup (B_{b_i} \cap B_{y_i}) \cup B_\beta$ is contained in the union of at most three adhesions of $\mathcal{T}$ so x satisfies the first alternative of property (d).

Lastly, for each $x \in V(Q) \setminus V(Q_0)$, there are two possibilities:

- $x = \alpha_i$ for some $i \in [s]$ with $m_i > 0$. By (g), $C_{\alpha_i} = B_{\alpha_i} \cup B_{\beta_i}$. Each of B_{α_i} and B_{β_i} is an adhesion of $\mathcal{T}$, so x satisfies the first alternative of property (d).
- x is a non-root node of Q_i for some $i \in [s]$ with $m_i > 0$. By induction, x satisfies (d) with respect to G_i, $\mathcal{T}_i$, and $\mathcal{Q}_i$. By definition, $G_i\langle \mathcal{T}_i, B_y\rangle = G\langle \mathcal{T}, B_y\rangle$ for each $y \in V(C_i)$, and every adhesion of $\mathcal{T}_i$ is an adhesion of $\mathcal{T}$, and $G_i\langle \mathcal{Q}_i, C_x\rangle = G\langle \mathcal{Q}, C_x\rangle$. Therefore, x also satisfies (d) with respect to G, $\mathcal{T}$, and $\mathcal{Q}$. □

Let $\mathcal{F} := (F, (B_x \mid x \in V(F)))$ be a rooted forest-decomposition of a graph G and, for each $x \in V(F)$, let A_x denote the parent adhesion of x. We say that $\mathcal{F}$ is *tidy* if (1) $B_x \setminus A_x \neq \emptyset$ for each $x \in V(F)$; and (2) $A_y \setminus A_x \neq \emptyset$ for each non-root $y \in V(F)$ with $x = \mathrm{p}_F(y)$.

Lemma 12. *Let G_0 be an n-vertex graph, with $n \in \mathbb{N}^+$, and let $\mathcal{T}$ be a tree-decomposition of G_0. Then, for any integer $b \geqslant 2$, there exists a graph G with an isolated vertex v_0 such that $G - v_0 = G_0$, a tidy rooted tree-decomposition $\mathcal{Q} := (Q, (C_x \mid x \in V(Q)))$ of G rooted at $x_0 \in V(Q)$, and a subset of edges $Y \subseteq E(Q)$ with the following properties:*

(a) $C_{x_0} = \{v_0\}$;
(b) *each root-to-leaf path in Q contains at most $2\log_b(n+1)+1$ edges of Y;*
(c) *each connected component of $Q - Y$ has at most $4b-1$ vertices;*
(d) *letting A_x denote the parent adhesion of x in $\mathcal{Q}$ (with $A_{x_0} := \emptyset$), for each connected component C of $Q - Y$ and each $x, y \in V(C)$,*

$$\left|\bigcup_{z \in V(Q_x)} (C_z \setminus A_z)\right| \leqslant b \left|\bigcup_{z \in V(Q_y)} (C_z \setminus A_z)\right| \ ;$$

(e) *each adhesion of $\mathcal{Q}$ is either $\{v_0\}$ or is contained in the union of at most two adhesions of $\mathcal{T}$; in particular, if $\mathcal{T}$ has adhesion-width at most k, then $\mathcal{Q}$ has adhesion-width at most* $\max\{2k, 1\}$; *and*
(f) *for each $x \in V(Q)$, at least one of the following holds:*
   - *$C_x \setminus \{v_0\}$ is contained in the union of at most three adhesions of $\mathcal{T}$; or*
   - *$G\langle \mathcal{Q}, C_x\rangle - (A \cup \{v_0\})$ is a subgraph of some torso of $\mathcal{T}$, where $A = \emptyset$ or A is an adhesion of $\mathcal{T}$.*

Proof. Let $\mathcal{T}_0 := (T_0, (B_x \mid x \in V(T_0)))$ and $Y_0 \subseteq E(T_0)$ be the tree-decomposition of G_0 obtained by applying Lemma 11 to the tree-decomposition $\mathcal{T}$. Thus, each torso of $\mathcal{T}_0$ satisfies Lemma 11(d). First, we transform $\mathcal{T}_0$ and Y_0 into a tidy rooted forest-decomposition $\mathcal{F} = (F, (B_x \mid x \in V(F)))$ and $Y_1 \subseteq E(F)$ in five steps:

(1) Initially, set $F := T_0$ and $Y_1 := Y_0$.
(2) Remove every node x of F with $B_x = \emptyset$ (along with any incident edges in F and Y_1).
(3) Remove every edge xy of F (and Y_1) with $B_x \cap B_y = \emptyset$.
(4) Repeatedly replace an edge $yz \in E(F)$ with the edge xz if x is the F-parent of y, y is the F-parent of z, and $B_z \cap B_y \subseteq B_x$.
If the edge yz is in Y_1, replace yz with xz in Y_1.
If the edge xy is in Y_1, add the edge xz to Y_1 (and leave xy in Y_1).
(5) Repeatedly remove every leaf y of F with $B_y \subseteq B_{\mathrm{p}(y)}$ (along with the edge $\mathrm{p}(y)y$ from F and Y_1). (Note that any non-root node y with $B_y \subseteq B_{\mathrm{p}(y)}$ has $B_z \cap B_y \subseteq B_y \subseteq B_{\mathrm{p}(y)}$ for each child z of y, so y loses all its children in step (4) and is removed in step (5).)

Let $\mathcal{F} = (F, (B_x \mid x \in V(F)))$ be the resulting tidy forest-decomposition. (For each $x \in V(F)$, the bag B_x is well-defined since $V(F) \subseteq V(T_0)$.) Note that each torso of $\mathcal{F}$ is a subgraph of a torso of $\mathcal{T}_0$. In step (4), the new edge xz satisfies $B_x \cap B_z = B_x \cap (B_y \cap B_z) = B_y \cap B_z$, so the adhesion on xz is identical to the adhesion on yz. Thus every adhesion of $\mathcal{F}$ is an adhesion of $\mathcal{T}_0$. Furthermore, for any node $z \in V(F)$, the F-ancestors of z form a subset of the T_0-ancestors of z. Furthermore, if x is both an F-ancestor and T_0-ancestor of z then the path from x to z in F contains an edge in Y_1 if and only if the path from x to z in T_0 contains an edge in Y_0. Therefore, for any connected component C of $F - Y_1$, $V(C) \subseteq V(C_0)$ for some component C_0 of $T_0 - Y_0$. Thus $|V(C)| \leqslant |V(C_0)| \leqslant 4b - 1$ for each component C of $F - Y_1$. Let Q be the tree obtained from F by introducing a vertex x_0 and adding an edge x_0x' for each root x' of F. Let $Y_2 := Y_1 \cup \{x_0x' \mid x' \text{ is a root of } F\}$. Then each component of $Q - Y_2$ has at most $4b - 1$ nodes.

Let G be the graph obtained by adding an isolated vertex v_0 to G_0. Define $C_{x_0} := \{v_0\}$. For each child x' of x_0, define $C_{x'} := B_{x'} \cup \{v_0\}$. For each other vertex x in Q, define $C_x := B_x$. Then $\mathcal{Q} := (Q, (C_x \mid x \in V(Q)))$ is a tidy rooted tree-decomposition of G that satisfies (a) and (f). Moreover, for each root x' of F, the adhesion on x_0x' is $C_{x_0} \cap C_{x'} = \{v_0\}$, while for every other edge $xy \in E(Q)$, $C_x \cap C_y = B_x \cap B_y$ is an adhesion of $\mathcal{F}$ (and hence of $\mathcal{T}_0$). By Lemma 11(c), each adhesion of $\mathcal{T}_0$ is contained in the union of at most two adhesions of $\mathcal{T}$. Therefore, $\mathcal{Q}$ satisfies (e).

Let $A_{x_0} := \emptyset$ and, for each vertex $x \in V(Q) \setminus \{x_0\}$, let A_x be the parent adhesion of x in $\mathcal{Q}$. For each $x \in V(Q)$, define $\omega(x) := |C_x \setminus A_x|$. Since $\mathcal{Q}$ is tidy, $\omega(x) \geqslant 1$ for each $x \in V(Q)$, so $\omega : V(Q) \to \mathbb{N}^+$ is a weight function with total weight $\omega(Q) = \sum_{x \in V(Q)} |C_x \setminus A_x| = |V(G)| = n+1$. Let $Y_3 \subseteq E(Q)$ be the set of edges obtained by applying Lemma 9 to Q with the weight function ω. Then, for each component C of $Q - Y_3$ and each $x, y \in V(C)$, we have

$$\left|\bigcup_{z \in V(Q_x)} (C_z \setminus A_z)\right| = \omega(Q_x) \leqslant b \cdot \omega(Q_y) = b \left|\bigcup_{z \in V(Q_y)} (C_z \setminus A_z)\right| .$$

Let $Y := Y_2 \cup Y_3$. Since each component of $Q - Y$ is contained in a component of $Q - Y_2$ and in a component of $Q - Y_3$, the pair $(\mathcal{Q}, Y)$ satisfies (c) and (d). Each root-to-leaf path in Q contains at most $1 + \log_b n$ edges in Y_2 and at most $\log_b(n+1)$ edges in Y_3. Thus, each root-to-leaf path in Q contains at most $1 + \log_b n + \log_b(n+1) \leqslant 2\log_b(n+1) + 1$ edges in Y, so $(\mathcal{Q}, Y)$ satisfies (b). □

3 Weighted Mixed Labelling Schemes

Let $\mathcal{G}$ be a class of graphs. We say that $\tau := (G^+, G, \omega)$ is a *$\mathcal{G}$-admissible triple* if G^+ is a graph in $\mathcal{G}$ with at least one vertex, G is a spanning subgraph of G^+, and $\omega : V(G) \to \mathbb{R}^+$ is a weight function. Let $\mathcal{T}_\mathcal{G}$ be the set of all $\mathcal{G}$-admissible triples. For each $n \in \mathbb{N}$, let $\mathcal{T}_{\mathcal{G},\leqslant n} := \{(G^+, G, \omega) \in \mathcal{T}_\mathcal{G} : |V(G)| \leqslant n\}$.

Let $g_i : \mathbb{N} \to \mathbb{R}$ for each $i \in [3]$. Let $A : (\{0,1\}^*)^2 \to \{0,1\}$ (the *adjacency tester*) and $I : (\{0,1\}^*)^3 \to \{0,1\}$ (the *identity tester*). For a $\mathcal{G}$-admissible triple $\tau := (G^+, G, \omega)$, let $n := |V(G)|$. A pair of functions

- $\mu : V(G) \cup \{K \mid K \text{ is a clique in } G^+\} \to \{0,1\}^*$ (the *mixed labelling function*); and
- $\kappa : \{(K,u) \mid K \text{ is a clique in } G^+, u \in K\} \to \{0,1\}^*$ (the *local identifier function*)

is a *mixed labelling* of (G^+, G) for (A, I) if it satisfies:

(M1) for all pairs of distinct cliques K and L in G^+, $\mu(K) \neq \mu(L)$;
(M2) for all $u, v \in V(G)$, $A(\mu(u), \mu(v)) = 1$ if and only if $uv \in E(G)$;
(M3) for every clique K in G^+, every $u \in K$, and every $v \in V(G)$, $I(\mu(K), \kappa(K,u), \mu(v)) = 1$ if and only if $u = v$.

If (μ, κ) additionally satisfies the length bounds

(M4) for every $v \in V(G)$, $|\mu(v)| \leqslant \log \omega(G) - \log \omega(v) + g_1(n)$;
(M5) for every clique K in G^+, $|\mu(K)| \leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + g_3(n)$;
(M6) for every clique K in G^+ and every $u \in K$, $|\kappa(K,u)| \leqslant g_2(n)$,

then (μ, κ) is called a *weighted (g_1, g_2, g_3) mixed labelling* of (G^+, G, ω) for (A, I).

Here is a little intuition about the last three conditions. The function $g_1(n)$ bounds how much the length of a vertex label exceeds the ideal value $\log \omega(G) - \log \omega(v)$ that appears in Kraft's inequality [31]. The function $g_2(n)$ is the length of the longest local identifier. The function $g_3(n)$ bounds how much the length of a clique label exceeds the Kraft-like quantity $\log \omega(G) - \log \min_{v \in K} \omega(v)$.

To prove Theorem 3, it will be helpful to fix a particular mixed labelling of each $\mathcal{G}$-admissible triple. A *weighted (g_1, g_2, g_3) mixed labelling* of $\mathcal{G}$ for (A, I) is an indexed collection $\mathcal{S} := ((\mu_\tau, \kappa_\tau) : \tau \in \mathcal{T}_\mathcal{G})$ where (μ_τ, κ_τ) is a weighted (g_1, g_2, g_3) mixed labelling of τ for (A, I), for each $\tau \in \mathcal{T}_\mathcal{G}$. The triple $(\mathcal{S}, A, I)$ is called a *weighted (g_1, g_2, g_3) mixed labelling scheme* for $\mathcal{G}$. We say that $\mathcal{G}$ *admits* a weighted (g_1, g_2, g_3) mixed labelling scheme if such a triple $(\mathcal{S}, A, I)$ exists. Similar definitions apply to adjacency labelling schemes. An *$f(n)$-bit adjacency labelling* of $\mathcal{G}$ for A is an indexed collection $\mathcal{A} := (\ell_G : G \in \mathcal{G})$ where, for each $G \in \mathcal{G}$, ℓ_G is an adjacency labelling of G for A and $|\ell_G(v)| \leqslant f(|V(G)|)$ for each $v \in V(G)$. The pair $(\mathcal{A}, A)$ is an *$f(n)$-bit adjacency labelling scheme* for $\mathcal{G}$.

3.1 Basic Properties

Let $(\mathcal{S}, A, I)$ be a weighted (g_1, g_2, g_3) mixed labelling scheme for a graph class $\mathcal{G}$, where $\mathcal{S} = ((\mu_\tau, \kappa_\tau) : \tau \in \mathcal{T}_\mathcal{G})$. For any graph G, let $\omega_1 : V(G) \to \{1\}$ be the *uniform weight function* defined by $\omega_1(v) := 1$ for all $v \in V(G)$. Let $\mathcal{G}^-$ be the class of all spanning subgraphs of graphs in $\mathcal{G}$. For each non-empty $G \in \mathcal{G}^-$, choose a graph $G^+ \in \mathcal{G}$ such that G is a spanning subgraph of G^+, and let $\mathcal{A} := (\mu_{(G^+, G, \omega_1)}|_{V(G)} : G \in \mathcal{G}^-)$. Then $(\mathcal{A}, A)$ is a $(\log n + g_1(n))$-

bit adjacency labelling scheme for $\mathcal{G}^-$. Thus, we are interested in the case where $g_1(n) \in o(\log n)$. We say that a graph class $\mathcal{G}$ admits an *efficient* weighted mixed labelling scheme if $\mathcal{G}$ admits a weighted (g_1, g_2, g_3) mixed labelling scheme with $g_i(n) \in o(\log n)$ for each $i \in [3]$. So any graph class that admits an efficient weighted mixed labelling scheme admits a $(1 + o(1)) \log n$-bit adjacency labelling scheme.

An adjacency labelling ℓ of a graph G is *injective* if $\ell(u) \neq \ell(v)$ for all distinct $u, v \in V(G)$. An adjacency labelling scheme $(\mathcal{A}, A)$ for $\mathcal{G}$ is *injective* if ℓ_G is injective for each $G \in \mathcal{G}$. The following lemma shows that any adjacency labelling obtained from a mixed labelling scheme is injective, which justifies Corollary 2.

Lemma 13. *Let G^+ be a graph with at least one vertex, let G be a spanning subgraph of G^+, and let (μ, κ) be a mixed labelling of (G^+, G) for (A, I) where A is an adjacency tester and I is an identity tester. Then the restriction $\mu|_{V(G)}$ of μ to $V(G)$ is injective.*

Proof. Suppose, for the sake of contradiction, that there exist distinct $v, w \in V(G)$ such that $\mu(v) = \mu(w)$. Since $K := \{v\}$ is a clique in G^+, (M3) implies that $I(\mu(K), \kappa(K, v), \mu(v)) = 1$ and $I(\mu(K), \kappa(K, v), \mu(w)) = 0$. Since $\mu(v) = \mu(w)$, this immediately leads to the contradiction

$$1 = I(\mu(K), \kappa(K, v), \mu(v)) = I(\mu(K), \kappa(K, v), \mu(w)) = 0 \ . \qquad \square$$

It will actually be convenient to consider a slightly weaker notion of weighted mixed labelling schemes. A $\mathcal{G}$-admissible triple (G^+, G, ω) is *nice* if $\omega : V(G) \to \mathbb{N}^+$ is a nice weight function. A *weak weighted (g_1, g_2, g_3) mixed labelling scheme* $(\mathcal{S}, A, I)$ for a graph class $\mathcal{G}$ satisfies all the conditions of a weighted (g_1, g_2, g_3) mixed labelling scheme for $\mathcal{G}$, with the exception that $\mathcal{S}$ only contains mixed labellings for nice triples $(G^+, G, \omega) \in \mathcal{T}_{\mathcal{G}}$. We say that $\mathcal{G}$ *admits* a weak weighted (g_1, g_2, g_3) mixed labelling scheme if such a scheme exists. Weak mixed labelling schemes and weighted mixed labelling schemes are essentially equivalent, as shown by the following lemma. This will allow us to only consider nice weight functions in the proofs.

Lemma 14. *Let $\mathcal{G}$ be a class of graphs. Let $g_i : \mathbb{N} \to \mathbb{R}$ for each $i \in [3]$. If $\mathcal{G}$ admits a weak weighted (g_1, g_2, g_3) mixed labelling scheme, then $\mathcal{G}$ admits a weighted $(g_1 + 1, g_2, g_3 + 1)$ mixed labelling scheme.*

Proof. Let $(\mathcal{S}, A, I)$ be a weak weighted (g_1, g_2, g_3) mixed labelling scheme for $\mathcal{G}$. We will use the same pair (A, I) for our weighted $(g_1 + 1, g_2, g_3 + 1)$ mixed labelling scheme. Let $\tau = (G^+, G, \omega) \in \mathcal{T}_{\mathcal{G}}$ and let $n := |V(G)|$. Let ω' be the nice weight function defined by $\omega'(v) := \lceil n\omega(v)/\omega(G) \rceil$ for each $v \in V(G)$, as in the proof of Observation 6. Thus, for every $v \in V(G)$,

$$\log \omega'(G) - \log \omega'(v) \leqslant \min\{\log n, \log \omega(G) - \log \omega(v)\} + 1 \ .$$

Therefore, $\mathcal{S}$ contains a weighted (g_1, g_2, g_3) mixed labelling (μ, κ) of (G^+, G, ω') for (A, I). Conditions (M1)–(M3) depend only on G^+ and G, so they hold for (μ, κ). For every $v \in V(G)$, (M4) gives

$$|\mu(v)| \leqslant \log \omega'(G) - \log \omega'(v) + g_1(n) \leqslant \log \omega(G) - \log \omega(v) + g_1(n) + 1 \ .$$

For every clique K in G^+, if $x \in K$ minimizes $\omega'(x)$, then $\log \omega'(G) - \log \omega'(x) \leqslant \log \omega(G) - \log \omega(x) + 1 \leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + 1$, and thus by (M5),

$$|\mu(K)| \leqslant \log \omega'(G) - \log \min_{v \in K} \omega'(v) + g_3(n) \leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + g_3(n) + 1 \ .$$

Finally, (M6) gives $|\kappa(K,u)| \leqslant g_2(n)$ for every clique K in G^+ and every $u \in K$. It follows that (μ, κ) is a weighted (g_1+1, g_2, g_3+1) mixed labelling of (G^+, G, ω) for (A, I). Setting $(\mu_\tau, \kappa_\tau) := (\mu, \kappa)$ for each $\tau \in \mathcal{T}_\mathcal{G}$, the indexed collection $\mathcal{S}' := ((\mu_\tau, \kappa_\tau) : \tau \in \mathcal{T}_\mathcal{G})$ yields the desired weighted (g_1+1, g_2, g_3+1) mixed labelling scheme $(\mathcal{S}', A, I)$ for $\mathcal{G}$. □

If the ultimate goal is to obtain an adjacency labelling scheme, then an even weaker notion suffices. Recall that $\omega_1 : V(G) \to \{1\}$ denotes the uniform weight function. We say that a class of graphs $\mathcal{G}$ *admits an unweighted (g_1, g_2, g_3) mixed labelling scheme* $(\mathcal{S}, A, I)$ if $\mathcal{S}$ satisfies all the conditions of a weighted (g_1, g_2, g_3) mixed labelling scheme for $\mathcal{G}$, with the exception that $\mathcal{S}$ only contains mixed labellings for triples $(G^+, G, \omega_1) \in \mathcal{T}_\mathcal{G}$ in which ω_1 is the uniform weight function.

Observation 15. *If a graph class $\mathcal{G}$ admits an unweighted (g_1, g_2, g_3) mixed labelling scheme, then it admits an injective $f(n)$-bit adjacency labelling scheme where $f(n) = \log n + g_1(n)$. Furthermore, if the mixed labelling scheme is g_0-orientable (defined in Subsection 3.2, below), then the resulting adjacency labelling scheme is also g_0-orientable.*

Proof. Let $(\mathcal{S}, A, I)$ be an unweighted (g_1, g_2, g_3) mixed labelling scheme for $\mathcal{G}$. For each non-empty graph $G \in \mathcal{G}$, let $\tau := (G, G, \omega_1)$, where $\omega_1 : V(G) \to \{1\}$ is the uniform weight function, and let (μ_τ, κ_τ) be the mixed labelling of τ in $\mathcal{S}$. By (M2), $\ell_G := \mu_\tau|_{V(G)}$ is an adjacency labelling of G for A. For $n := |V(G)|$, (M4) implies that for each $v \in V(G)$,

$$|\ell_G(v)| \leqslant \log \omega_1(G) - \log \omega_1(v) + g_1(n) = \log n + g_1(n) \ .$$

By Lemma 13, ℓ_G is injective. Thus, $(\mathcal{A}, A)$ is an injective $(\log n + g_1(n))$-bit adjacency labelling scheme for $\mathcal{G}$, where $\mathcal{A} := (\ell_G : G \in \mathcal{G})$. The orientability claim is verified in Subsection 3.2. □

3.2 Orientable Labelling Schemes

In order to establish Theorem 3, which promises sparse universal graphs, we need two additional definitions. Let $(\mathcal{S}, A, I)$ be a weighted (g_1, g_2, g_3) mixed labelling scheme for a graph class $\mathcal{G}$, where $\mathcal{S} = ((\mu_\tau, \kappa_\tau) : \tau \in \mathcal{T}_\mathcal{G})$. For each $n \in \mathbb{N}^+$, let

$$\mathcal{T}'_{\mathcal{G},\leqslant n} := \Big\{(G^+, G, \omega) \in \mathcal{T}_{\mathcal{G},\leqslant n} : \omega : V(G) \to \mathbb{N}^+,\ \omega(G) \leqslant 2n\Big\}.$$

The triples in $\mathcal{T}'_{\mathcal{G},\leqslant n}$ need not be nice: their total weight is bounded by $2n$, which may be much larger than their number of vertices. The *n-universal graph* U_n defined by $(\mathcal{S}, A)$ is the graph with vertex set

$$V(U_n) := \Big\{\mu_\tau(v) : \tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G},\leqslant n},\ v \in V(G)\Big\}$$

and edge set

$$E(U_n) := \left\{\{\mu_\tau(v), \mu_\tau(v')\} : \tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G},\leqslant n},\ vv' \in E(G)\right\} \ .$$

These definitions imply that, for each $\tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G},\leqslant n}$, the graph G is isomorphic to the induced subgraph $U_n[\{\mu_\tau(v) : v \in V(G)\}]$. Taking uniform weights, we see that U_n contains every graph $G \in \mathcal{G}$ with at most n vertices as an induced subgraph. (In fact, U_n contains every subgraph of every at-most-n-vertex graph in $\mathcal{G}$, even if $\mathcal{G}$ itself is not monotone.) Note that U_n is finite. Indeed, for each $\tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G},\leqslant n}$ and each $v \in V(G)$, we have $\omega(v) \geqslant 1$ and $\omega(G) \leqslant 2n$, so

$$|\mu_\tau(v)| \leqslant \log \omega(G) - \log \omega(v) + g_1(|V(G)|) \leqslant \log n + 1 + \max_{1\leqslant m\leqslant n} g_1(m) \ .$$

If g_1 is non-decreasing, this bound is at most $\log n + 1 + g_1(n)$.

Since the graph U_n defined above depends only on the vertex labellings in $\mathcal{S}$ (and not on clique labels or local identifiers), the same definition applies directly to any injective adjacency labelling $\mathcal{A} := (\ell_G : G \in \mathcal{G})$ of $\mathcal{G}$ for A. In this case, for each $n \in \mathbb{N}^+$ and each $G \in \mathcal{G}$ with at most n vertices, the n-universal graph U_n defined by $(\mathcal{A}, A)$ has an induced subgraph $U_n[\{\ell_G(v) : v \in V(G)\}]$ that is isomorphic to G.

Let $g_0 : \mathbb{N} \to \mathbb{R}^{\geqslant 0}$. We say that $(\mathcal{S}, A)$ (or $(\mathcal{A}, A)$) is *g_0-orientable* if, for each $n \in \mathbb{N}^+$, the n-universal graph U_n defined by $(\mathcal{S}, A)$ (or $(\mathcal{A}, A)$) has an orientation in which each vertex has outdegree at most $g_0(n)$. A weighted mixed labelling scheme $(\mathcal{S}, A, I)$ is *g_0-orientable* if $(\mathcal{S}, A)$ is g_0-orientable. This definition immediately implies that the n-universal graph U_n has at most $|V(U_n)| \cdot g_0(n)$ edges. We say that a graph class $\mathcal{G}$ *admits a g_0-orientable weighted mixed labelling scheme* if such a scheme exists (and similarly for adjacency labelling schemes). Observe that if $(\mathcal{S}, A, I)$ is g_0-orientable, then the injective adjacency labelling scheme $(\mathcal{A}, A)$ obtained from $\mathcal{S}$ in Observation 15 is also g_0-orientable. Indeed, for each $G \in \mathcal{G}$ with $|V(G)| \leqslant n$, the triple $\tau = (G, G, \omega_1)$ belongs to $\mathcal{T}'_{\mathcal{G},\leqslant n}$, so the n-universal graph of $(\mathcal{A}, A)$ is an induced subgraph of the n-universal graph of $(\mathcal{S}, A)$, and therefore inherits an orientation of maximum outdegree at most $g_0(n)$.

To control the number of edges when combining mixed labellings, we also need to bound the number of vertex labels associated with a fixed clique label and local identifier. This motivates the following strengthening of orientability. We say that $(\mathcal{S}, A, I)$ is *strongly g_0-orientable* if $(\mathcal{S}, A)$ is g_0-orientable and, for each $n \in \mathbb{N}^+$, each triple $\tau_0 := (G_0^+, G_0, \omega_0) \in \mathcal{T}'_{\mathcal{G},\leqslant n}$, each clique K_0 in G_0^+, and each $u_0 \in K_0$, there are at most $g_0(n)$ binary strings β for which there exists a triple $\tau := (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G},\leqslant n}$, a clique K in G^+, and a vertex $u \in K$ such that

(a) $\mu_\tau(K) = \mu_{\tau_0}(K_0)$;
(b) $\kappa_\tau(K, u) = \kappa_{\tau_0}(K_0, u_0)$; and
(c) $\mu_\tau(u) = \beta$.

We say that $\mathcal{G}$ *admits a strongly g_0-orientable weighted (g_1, g_2, g_3) mixed labelling scheme* if such a scheme exists.

Less formally, fix a clique label ζ and a local identifier γ that occur together in a triple in $\mathcal{T}'_{\mathcal{G},\leqslant n}$. Strong g_0-orientability guarantees that at most $g_0(n)$ distinct vertex labels β occur

with this same pair: there must be a triple $\tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G},\leqslant n}$, a clique K in G^+, and a vertex $u \in K$ such that $\mu_\tau(K) = \zeta$, $\kappa_\tau(K,u) = \gamma$, and $\mu_\tau(u) = \beta$. Any such occurrence implies that $I(\zeta,\gamma,\beta) = 1$.

Our construction will construct an adjacency labelling scheme $(\mathcal{A}', A')$ for a graph class $\mathcal{G}'$. The graphs in $\mathcal{G}'$ will contain subgraphs that are in some class $\mathcal{G}$ that admits a weighted mixed labelling scheme $(\mathcal{S}, A, I)$. In our constructions, a vertex label $\ell_{G'}(v)$ may contain ζ and γ, while the label of a neighbour w contains a corresponding vertex label β. If every edge handled in this way has a witness as above, then fixing $\ell_{G'}(v)$ leaves at most $g_0(n)$ choices for β. If, for each such β, there are at most $M(n)$ possible complete labels $\ell_{G'}(w)$, then orienting these edges from $\ell_{G'}(v)$ towards $\ell_{G'}(w)$ contributes at most $g_0(n) \cdot M(n)$ to the outdegree of $\ell_{G'}(v)$. Here $M(n)$ accounts for the fact that β is only *part* of the vertex label $\ell_{G'}(w)$. There may be up to $M(n)$ vertex labels $\ell_{G'}(w)$ that include β.

The definitions of U_n, g_0-orientability, and strong g_0-orientability also apply to weak weighted mixed labelling schemes, with all triples restricted to nice triples in $\mathcal{T}'_{\mathcal{G},\leqslant n}$.

Observation 16. *The construction in Lemma 14 preserves g_0-orientability and strong g_0-orientability.*

Proof. Let $(\mathcal{S}, A, I)$ be a weak weighted mixed labelling scheme and let $(\mathcal{S}', A, I)$ be the weighted mixed labelling scheme constructed in Lemma 14. Fix $n \in \mathbb{N}^+$. For every $\tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G},\leqslant n}$, the construction assigns to τ the labelling in $\mathcal{S}$ of $\tau' = (G^+, G, \omega')$, where $\omega'(v) = \lceil |V(G)| \omega(v)/\omega(G) \rceil$. By Observation 6, τ' is nice and $\omega'(G) \leqslant \omega(G) \leqslant 2n$, so $\tau' \in \mathcal{T}'_{\mathcal{G},\leqslant n}$. Consequently, the n-universal graph defined by $(\mathcal{S}', A)$ is a subgraph of the one defined by $(\mathcal{S}, A)$, and any orientation of the latter restricts to an orientation of the former with no larger outdegrees. Similarly, every occurrence of a clique label, local identifier, and vertex label in $\mathcal{S}'$ on a triple in $\mathcal{T}'_{\mathcal{G},\leqslant n}$ also occurs in $\mathcal{S}$ on a nice triple in the same set. Thus, the bound required for strong g_0-orientability is preserved as well. □

3.3 Adding Apex Vertices

The next lemma shows how a mixed labelling scheme can accommodate the addition of apex vertices. Recall that $\mathcal{G}^{+a}$ is the class of graphs G such that $G - X \in \mathcal{G}$ for some $X \subseteq V(G)$ with $|X| \leqslant a$.

Lemma 17. *Let $\mathcal{G}$ be a class of graphs that admits a strongly g_0-orientable weighted (g_1, g_2, g_3) mixed labelling scheme, where $g_0 : \mathbb{N} \to \mathbb{R}^{\geqslant 0}$ is a function, and $g_1, g_2, g_3 : \mathbb{N} \to \mathbb{R}^{\geqslant 0}$ are non-decreasing functions with $g_1(n), g_3(n) \in O(\log n)$. Let $a \in \mathbb{N}^+$. Then $\mathcal{G}^{+a}$ admits a strongly g'_0-orientable weighted (g'_1, g'_2, g'_3) mixed labelling scheme, where*

$$\begin{aligned}
g'_0(n) &= (2g_0(n) + a)2^a \ , \\
g'_1(n) &= g_1(n) + O(a) + O(\log\log n) \ , \\
g'_2(n) &= g_2(n) + O(\log a) \ , \\
g'_3(n) &= g_3(n) + O(a) + O(\log\log n) \ .
\end{aligned}$$

Proof. Let $(\mathcal{S}, A, I)$ be a strongly g_0-orientable weighted (g_1, g_2, g_3) mixed labelling scheme for $\mathcal{G}$, where $\mathcal{S} := ((\mu_\tau^-, \kappa_\tau^-) : \tau \in \mathcal{T}_\mathcal{G})$. For each $n \in \mathbb{N}^+$, let U_n be the n-universal graph defined by $(\mathcal{S}, A)$.

By Lemma 14 and Observation 16, it suffices to construct a strongly g_0'-orientable weak weighted mixed labelling scheme with the claimed asymptotic label-length bounds. The additive constants needed to extend it to arbitrary weights are absorbed into the $O(a)$ terms in g_1' and g_3'. We will describe testers A' and I' and construct a mixed labelling (μ_τ, κ_τ) for each nice triple $\tau \in \mathcal{T}_{\mathcal{G}^{+a}}$. Let $\mathcal{S}'$ be the indexed collection of these labellings over all nice triples. To fix a specific labelling (μ_τ, κ_τ) for each nice $\mathcal{G}^{+a}$-admissible triple $\tau = (G^+, G, \omega)$, we first choose, for each $G^+ \in \mathcal{G}^{+a}$, a specific set $X(G^+) \subseteq V(G^+)$ with $|X(G^+)| \leqslant a$ such that $G^+ - X(G^+) \in \mathcal{G}$.

Let $\tau := (G^+, G, \omega) \in \mathcal{T}_{\mathcal{G}^{+a}}$ be a triple with a nice weight function ω, let $X := X(G^+)$, and let $n := |V(G)|$. Since ω is nice, $\omega(G) \leqslant 2n$ and $\log \omega(G) - \log \omega(v) \leqslant \log n + 1$ for each $v \in V(G)$. We will describe the weighted mixed labelling $(\mu, \kappa) := (\mu_\tau, \kappa_\tau)$ of τ for (A', I').

The subgraph label μ^-: If $G - X$ is non-empty, then by the definition of X, $\tau^- := (G^+ - X, G - X, \omega|_{V(G-X)})$ is a $\mathcal{G}$-admissible triple with $|V(G-X)| \leqslant n$. Let $(\mu^-, \kappa^-) := (\mu_{\tau^-}^-, \kappa_{\tau^-}^-)$. Thus, (μ^-, κ^-) is a weighted (g_1, g_2, g_3) mixed labelling of $(G^+ - X, G - X, \omega|_{V(G-X)})$ for (A, I). For each $v \in V(G-X)$, since g_1 is non-decreasing,

$$\begin{aligned} |\mu^-(v)| &\leqslant \log \omega(G-X) - \log \omega(v) + g_1(|V(G-X)|) \\ &\leqslant \log \omega(G) - \log \omega(v) + g_1(n) \ , \end{aligned} \tag{2}$$

and for each clique K in $G^+ - X$, since g_3 is non-decreasing,

$$\begin{aligned} |\mu^-(K)| &\leqslant \log \omega(G-X) - \log \min_{v \in K} \omega(v) + g_3(|V(G-X)|) \\ &\leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + g_3(n) \ . \end{aligned} \tag{3}$$

If $G - X$ is empty, no labelling from $\mathcal{S}$ is needed; take μ^- and κ^- to have empty domains. Extend the domain of μ^- to include the vertices in X by defining $\mu^-(v) := \varepsilon$ for each $v \in X$.

The apex identifier $\sigma(u)$: Fix a linear ordering $u_1, \ldots, u_b$ of the vertices in X. By definition, $b \leqslant a$. For each $u \in V(G)$, define

$$\sigma(u) := \begin{cases} \lceil \log(b+1) \rceil \text{-bit binary representation of } i & \text{if } u \in X \text{ and } u = u_i, \\ \lceil \log(b+1) \rceil \text{ zero bits} & \text{if } u \in V(G) \setminus X. \end{cases}$$

The vertex label $\mu(v)$: For each $v \in V(G)$, let $c(v)$ be a b-bit string whose i-th bit indicates if $vu_i \in E(G)$. For each $v \in V(G)$, define

$$\mu(v) := \langle \sigma(v), c(v), \mu^-(v) \rangle \ .$$

For $v \notin X$, (2) bounds $|\mu^-(v)|$; for $v \in X$, $|\mu^-(v)| = 0$. Therefore, for each $v \in V(G)$,

$$|\sigma(v)| + b + |\mu^-(v)| \leqslant \lceil \log(b+1) \rceil + b + \log \omega(G) - \log \omega(v) + g_1(n)$$

$$
\begin{aligned}
&\leqslant \log \omega(G) - \log \omega(v) + g_1(n) + 2a \\
&\leqslant \log n + g_1(n) + 2a + 1 && \text{since } \omega \text{ is nice} \\
&\in O(a + \log n) \ ,
\end{aligned}
$$

and therefore

$$
\begin{aligned}
|\mu(v)| &\leqslant |\sigma(v)| + b + |\mu^-(v)| + O(\hat{\lg}|\sigma(v)| + \hat{\lg} b + \hat{\lg}|\mu^-(v)|) && \text{by (1)} \\
&\leqslant \log \omega(G) - \log \omega(v) + g_1(n) + O(a + \log\log n) \ .
\end{aligned}
$$

Adjacency testing: We now describe the adjacency tester A'. Given the labels $\mu(v) = \langle \sigma(v), c(v), \mu^-(v) \rangle$ and $\mu(w) = \langle \sigma(w), c(w), \mu^-(w) \rangle$ for two vertices v and w of G, A' works as follows. First, by checking the values of $\sigma(v)$ and $\sigma(w)$, the tester checks if v and/or w lie in X. If $\sigma(v)$ encodes an index $i \in [b]$, we return the i-th bit of $c(w)$ (which indicates whether $wu_i = wv \in E(G)$). Otherwise, if $\sigma(w)$ encodes an index $j \in [b]$, we return the j-th bit of $c(v)$ (which indicates whether $vu_j = vw \in E(G)$). If neither v nor w is in X, then $A'(\mu(v), \mu(w)) := A(\mu^-(v), \mu^-(w))$.

The clique label $\mu(K)$: For each clique K in G^+, let $c(K)$ be a b-bit string whose i-th bit indicates if $u_i \in K$. For each clique K in G^+, define

$$
\mu(K) := \begin{cases} \langle c(K) \rangle & \text{if } K \subseteq X, \\ \langle c(K), \mu^-(K \setminus X) \rangle & \text{otherwise.} \end{cases}
$$

To see that μ is injective when restricted to cliques, consider two distinct cliques K and L in G^+. If $K \cap X \neq L \cap X$, then $c(K) \neq c(L)$, so $\mu(K) \neq \mu(L)$ because their first parts differ. Otherwise, $K \setminus X \neq L \setminus X$. If one is empty, say $K \subseteq X$, then $\mu(K)$ has one part while $\mu(L)$ has two parts, so $\mu(K) \neq \mu(L)$. If both are non-empty, then $\mu^-(K \setminus X) \neq \mu^-(L \setminus X)$ since μ^- is injective on cliques of $G^+ - X$, which implies $\mu(K) \neq \mu(L)$.

Next, we bound the length of the clique labels. First consider a clique $K \not\subseteq X$. Since $K \setminus X \subseteq K$, we have $\min_{v \in K \setminus X} \omega(v) \geqslant \min_{v \in K} \omega(v)$, so by (3),

$$
|\mu^-(K \setminus X)| + b \leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + g_3(n - |X|) + a \ ,
$$

and therefore

$$
\begin{aligned}
|\mu(K)| &\leqslant |\mu^-(K \setminus X)| + b + O(\hat{\lg}|\mu^-(K \setminus X)| + \hat{\lg} b) && \text{by (1)} \\
&\leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + g_3(n) + O(a + \log\log n) \\
&\leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + g_3'(n) \ .
\end{aligned}
$$

If $K \subseteq X$, then $|\mu(K)| \leqslant b + O(\hat{\lg} b) \leqslant a + O(\log a) \leqslant g_3'(n) \leqslant \log \omega(G) - \log \min_{v \in K} \omega(v) + g_3'(n)$ since $\omega(G) \geqslant \min_{v \in K} \omega(v) \geqslant 1$.

The local identifier $\kappa(K,u)$: For each clique K in G^+ and each $u \in K$, define

$$\kappa(K,u) := \begin{cases} 0 \circ \kappa^-(K \setminus X, u) & \text{if } u \notin X, \\ 1 \circ \sigma(u) & \text{if } u \in X. \end{cases}$$

Note that $\kappa(K,u)$ is a concatenation, so it does not incur the encoding cost from (1). Thus,

$$|\kappa(K,u)| \leqslant 1 + \max\{g_2(n), \lceil \log(a+1) \rceil\} \leqslant g_2(n) + O(\log a) \leqslant g_2'(n) \ .$$

Identity testing: We now describe the identity tester I'. Given $\mu(K)$, $\kappa(K,u)$, and $\mu(v)$ for some clique K in G^+, $u \in K$, and $v \in V(G)$, we compute $I'(\mu(K), \kappa(K,u), \mu(v))$ as follows. First, by examining $\sigma(v)$ and the first bit of $\kappa(K,u)$, the tester determines whether $v \in X$ and whether $u \in X$. If $u, v \in X$, then $I'(\mu(K), \kappa(K,u), \mu(v)) = 1$ if $\kappa(K,u) = 1 \circ \sigma(v)$, and $I'(\mu(K), \kappa(K,u), \mu(v)) = 0$ otherwise. If $u, v \notin X$, then since $u \in K$ we have that $K \not\subseteq X$, which means $\mu(K)$ has a second part $\mu^-(K \setminus X)$. The tester extracts this second part and sets $I'(\mu(K), \kappa(K,u), \mu(v)) := I(\mu^-(K \setminus X), \kappa^-(K \setminus X, u), \mu^-(v))$. Otherwise, one of u or v is in X and the other is not, so $u \neq v$ and we set $I'(\mu(K), \kappa(K,u), \mu(v)) := 0$.

This completes the description of the adjacency tester A', the identity tester I', and the weighted (g_1', g_2', g_3') mixed labelling $(\mu_\tau, \kappa_\tau) := (\mu, \kappa)$ of $\tau = (G^+, G, \omega)$ for (A', I'). Since τ is an arbitrary nice $\mathcal{G}^{+a}$-admissible triple, this completes the construction of the weak weighted mixed labelling scheme $(\mathcal{S}', A', I')$.

g_0'-orientability: We now show that $(\mathcal{S}', A')$ is g_0'-orientable. Fix $n \in \mathbb{N}^+$. Fix an orientation of U_n with maximum outdegree at most $g_0(n)$. We must show that the n-universal graph U_n' defined by $(\mathcal{S}', A')$ has an orientation in which each vertex has outdegree at most $g_0'(n)$. We now explain how to choose the orientation for an arbitrary edge $\{\beta, \beta'\}$ of U_n'. By the definition of U_n', there exists a nice triple $\tau := (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G}^{+a}, \leqslant n}$ and an edge $vv' \in E(G)$ such that the mixed labelling $(\mu, \kappa) := (\mu_\tau, \kappa_\tau)$ of (G^+, G, ω) for (A', I') has $\mu(v) = \beta$ and $\mu(v') = \beta'$. Let $X := X(G^+)$, so $G^+ - X \in \mathcal{G}$. If $G - X$ is non-empty, its restricted weight function remains positive integer-valued, $|V(G-X)| \leqslant n$, and $\omega(G-X) \leqslant \omega(G) \leqslant 2n$. Thus, $\tau^- \in \mathcal{T}'_{\mathcal{G}, \leqslant n}$, even though τ^- need not be nice. By the definition of the adjacency tester A', exactly one of the following two cases applies:

1. If neither v nor v' is in X, then $A'(\beta, \beta') = A(\mu^-(v), \mu^-(v')) = 1$. The triple τ^- witnesses this edge in U_n. Use the fixed orientation of $\{\mu^-(v), \mu^-(v')\}$ in U_n. If $\{\mu^-(v), \mu^-(v')\}$ is oriented towards $\mu^-(v)$, then we orient $\{\beta, \beta'\}$ towards $\beta = \mu(v)$. If $\{\mu^-(v), \mu^-(v')\}$ is oriented towards $\mu^-(v')$, then we orient $\{\beta, \beta'\}$ towards $\beta' = \mu(v')$.

   Fix a vertex β in U_n'. We now argue that this branch of the adjacency testing procedure orients at most $g_0(n) \cdot 2^a$ edges away from β, over all choices of β'. The bitstring β determines the label $\mu(v) = \langle \sigma(v), c(v), \mu^-(v) \rangle$. Importantly, $c(v)$ is a bit-vector of length exactly $|X|$, so β strictly fixes $|X|$. Each edge oriented away from β by this branch of the adjacency testing procedure corresponds to an edge oriented away from $\mu^-(v)$ in U_n. Since β includes $\mu^-(v)$, there are at most $g_0(n)$ choices of $\mu^-(v')$ that result in an edge being oriented away from β in this way. Since $v' \notin X$, $\sigma(v')$ is a sequence of 0s of length $\lceil \log(|X|+1) \rceil$, and $c(v')$ is a bit-vector of length $|X|$, leaving

$2^{|X|}$ choices for $c(v')$. Thus, the number of possible strings $\beta' = \langle \sigma(v'), c(v'), \mu^-(v') \rangle$ for which the edge $\{\beta, \beta'\}$ is oriented away from β by this branch of the adjacency testing procedure is at most $g_0(n)2^{|X|} \leqslant g_0(n)2^a$.

2. Otherwise, at least one of v or v' is in X. If both $v, v' \in X$, then we orient $\{\beta, \beta'\}$ towards the lexicographically larger string. If $\{v, v'\} \cap X = \{v\}$, then we orient $\{\beta, \beta'\}$ towards β. If $\{v, v'\} \cap X = \{v'\}$, then we orient $\{\beta, \beta'\}$ towards β'.

   Fix a vertex β in U'_n. We now argue that this branch of the adjacency testing procedure orients at most $a2^a$ edges away from β. This corresponds to a case in which $v' \in X$, so $\beta' = \langle \sigma(v'), c(v'), \varepsilon \rangle$. There are at most $|X|$ choices for $\sigma(v')$ and $2^{|X|}$ choices for $c(v')$. Again, $|X|$ is fixed by the length of $c(v)$ in β, so the number of strings β' oriented away from β is at most $|X| \cdot 2^{|X|} \leqslant a2^a$.

Therefore, for any fixed $\beta \in V(U'_n)$, the outdegree of β in this orientation of U'_n is at most $g_0(n)2^a + a2^a = (g_0(n) + a)2^a \leqslant g'_0(n)$.

Strong g'_0-orientability: We now verify that $(\mathcal{S}', A', I')$ is strongly g'_0-orientable. Let $\tau_0 := (G_0^+, G_0, \omega_0) \in \mathcal{T}'_{\mathcal{G}^{+a}, \leqslant n}$ be a nice triple, and let $(\mu_{\tau_0}, \kappa_{\tau_0}) \in \mathcal{S}'$ be the mixed labelling of τ_0 for (A', I'). Let K_0 be a clique in G_0^+ and let $u_0 \in K_0$. Let $\zeta := \mu_{\tau_0}(K_0)$ and let $\gamma := \kappa_{\tau_0}(K_0, u_0)$. We need to show that there are at most $g'_0(n)$ strings $\beta \in V(U'_n)$ such that there exists a nice triple $\tau := (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{G}^{+a}, \leqslant n}$, some clique K in G^+, and some $u \in K$ such that $\mu_\tau(K) = \zeta$, $\kappa_\tau(K, u) = \gamma$, and $\mu_\tau(u) = \beta$.

Let $\tau := (G^+, G, \omega)$, K, u, and β be as described above and let $X := X(G^+)$. Let $(\mu, \kappa) := (\mu_\tau, \kappa_\tau)$. Since $\zeta = \mu(K)$, either $\zeta = \langle c(K) \rangle$ or $\zeta = \langle c(K), \mu^-(K \setminus X) \rangle$. In both cases, $c(K)$ is a bit-vector of length $|X|$, so ζ fixes $|X|$.

- If $u \in X$, then $\beta = \mu(u) = \langle \sigma(u), c(u), \varepsilon \rangle$. Since $\gamma = \kappa(K, u) = 1 \circ \sigma(u)$, the string γ determines $\sigma(u)$. There are $2^{|X|}$ choices for $c(u)$. Because $|X|$ is fixed by ζ, there are at most $2^{|X|} \leqslant 2^a$ such strings β.
- If $u \in V(G - X)$, then $\beta = \langle \sigma(u), c(u), \mu^-_{\tau^-}(u) \rangle$, and $I(\mu^-_{\tau^-}(K \setminus X), \kappa^-_{\tau^-}(K \setminus X, u), \mu^-_{\tau^-}(u)) = 1$. The string $\mu^-_{\tau^-}(K \setminus X)$ is extracted from ζ and the string $\kappa^-_{\tau^-}(K \setminus X, u)$ is extracted from γ. Here $\tau^- \in \mathcal{T}'_{\mathcal{G}, \leqslant n}$, since $|V(G - X)| \leqslant n$ and $\omega(G - X) \leqslant \omega(G) \leqslant 2n$. The triple τ^-, the clique $K \setminus X$, and the vertex u witness the occurrence of these three strings in $\mathcal{S}$. Since $(\mathcal{S}, A, I)$ is strongly g_0-orientable, there are at most $g_0(n)$ choices for $\mu^-_{\tau^-}(u)$ given ζ and γ. Since $u \notin X$, $\sigma(u)$ is a string of $\lceil \log(|X| + 1) \rceil$ zeros. The bit-vector $c(u)$ is an arbitrary binary string of length $|X|$. Because $|X|$ is fixed by ζ, there are at most $g_0(n) \cdot 2^{|X|} \leqslant g_0(n)2^a$ choices for $\beta = \mu_\tau(u)$ arising from such witnesses.

Therefore, the number of possible choices for β is at most $2^a + g_0(n)2^a \leqslant (2g_0(n) + a)2^a = g'_0(n)$.

Finally, extend $(\mathcal{S}', A', I')$ to all $\mathcal{G}^{+a}$-admissible triples by assigning to (G^+, G, ω) its labelling for the normalized weights $\widehat{\omega}(v) := \lceil |V(G)| \omega(v)/\omega(G) \rceil$. By Lemma 14, the vertex and clique label-length bounds increase by at most 1, which is absorbed into the stated $O(a)$ terms. By Observation 16, the resulting full weighted scheme remains strongly g'_0-orientable. □

3.4 Tree-Decompositions: Combining via Clique-Sums

Lemma 18. *Let $\mathcal{G}$ be a monotone class of graphs that admits a strongly g_0-orientable weighted (g_1, g_2, g_3) mixed labelling scheme where $g_i : \mathbb{N} \to \mathbb{R}^{\geqslant 0}$ is a non-decreasing function for each $i \in \{0,1,2,3\}$ and $g_i(n) \in O(\log n)$ for each $i \in \{1,2,3\}$. Let $b : \mathbb{N} \to \{2,3,\dots\}$ and $k : \mathbb{N} \to \mathbb{N}^+$ be non-decreasing functions such that $b(n) \leqslant n/2$ and $k(n) \leqslant \log n$ for all sufficiently large integers n. Let $\mathcal{G}'$ be the graph class that consists of, for each $n \in \mathbb{N}$, every n-vertex graph G that has a rooted tree-decomposition of adhesion-width at most $k(n)$ whose torsos belong to $\mathcal{G}$. Then $\mathcal{G}'$ admits an injective g_0'-orientable $f(n)$-bit adjacency labelling scheme with*

$$\begin{aligned} g_0'(n) &= g_0(n) \cdot \exp(O(k(n)(g_3(n) + g_2(n) + k(n) + \log b(n) + \log\log n))) \ , \\ f(n) &= \log n + g_1(n) + O((g_3(n) + k(n) + \log\log n) \cdot \log_{b(n)} n) \\ &\quad + O(k(n) \cdot (g_3(n) + g_2(n) + k(n) + \log b(n) + \log\log n)) \ . \end{aligned}$$

Proof. Let g_0, g_1, g_2, g_3, $\mathcal{G}$, $\mathcal{G}'$, k, and b be as given in the statement of the lemma. For any fixed constant n_0, graphs on at most n_0 vertices have labels of length $O(1)$ and U_n' has $O(1)$ vertices and edges, so the bounds hold trivially for $n < n_0$. Thus, we may assume that n is sufficiently large so that $b(n) \leqslant n/2$ and $k(n) \leqslant \log n$. We will define an adjacency tester A' and an adjacency labelling $\mathcal{A}' := (\ell_{G_0} : G_0 \in \mathcal{G}')$ of $\mathcal{G}'$ for A'. Since it is impractical to describe the operation of A' without first describing the format of the labels given as arguments to A', we begin by describing the construction of an adjacency labelling $\ell_{G_0} : V(G_0) \to \{0,1\}^*$ for an arbitrary graph $G_0 \in \mathcal{G}'$.

Let G_0 be an n-vertex graph in $\mathcal{G}'$ and let $\mathcal{T}_0$ be a rooted tree-decomposition of G_0 witnessing that $G_0 \in \mathcal{G}'$, so $\mathcal{T}_0$ has adhesion-width at most $k := k(n)$ and each torso of $\mathcal{T}_0$ belongs to $\mathcal{G}$. Let $b := b(n)$. Let $G, \mathcal{T} := (T, (B_x \mid x \in V(T)))$, and $Y \subseteq E(T)$ be the result of applying Lemma 12 to the tree-decomposition $\mathcal{T}_0$ of G_0, so that G has an isolated vertex v_0 with $G - v_0 = G_0$, $B_{x_0} = \{v_0\}$, and $\mathcal{T}$ is a tidy rooted tree-decomposition of G with root x_0.

Define $A_{x_0} := \emptyset$ and, for each $x \in V(T) \setminus \{x_0\}$, let A_x be the parent adhesion of x in $\mathcal{T}$. For each $x \in V(T)$, $G\langle \mathcal{T}, B_x\rangle - A_x$ is either a 1-vertex graph (if $x = x_0$), can be made into a subgraph of a torso of $\mathcal{T}_0$ by deleting at most one adhesion of $\mathcal{T}_0$ (so at most k vertices), or its vertex set is contained in the union of at most 3 adhesions of $\mathcal{T}_0$ (so it has at most $3k$ vertices). Since $\mathcal{G}$ is monotone, each subgraph of a graph in $\mathcal{G}$ is in $\mathcal{G}$. In each case, therefore, $G\langle \mathcal{T}, B_x\rangle - A_x \in \mathcal{G}^{+3k}$.

For each $v \in V(G)$, let $\omega(v) := 1$ and note that $\omega : V(G) \to \{1\}$ is a nice weight function. For each $x \in V(T)$, let $\omega(x) := \omega(B_x \setminus A_x)$. Then $\omega(T) = \sum_{x \in V(T)} \omega(x) = \omega(G) = n + 1$, since $\{B_x \setminus A_x \mid x \in V(T)\}$ is a partition of $V(G)$. For each $x \in V(T)$, let $C_x := N_{T_x}(x)$ be the set of T-children of x. For each $x \in V(T)$, and each clique K in $G\langle \mathcal{T}, B_x\rangle - A_x$, let $C_K := \{y \in C_x \mid A_y \setminus A_x = K\}$. Since $\mathcal{T}$ is tidy, $A_y \setminus A_x \neq \emptyset$ for each $y \in C_x$. Therefore, each $y \in C_x$ is contained in C_K for exactly one clique $K = A_y \setminus A_x$ in $G\langle \mathcal{T}, B_x\rangle - A_x$. We say that K is *light* if $\sum_{y \in C_K} \omega(T_y) < \omega(T_x)/b$, and is *non-light* otherwise.

Let $F := T - Y$. Properties (a), (b), (c), (d), (e), and (f) of Lemma 12 translate to the following properties of $\mathcal{T}$, Y, and F:

(P1) $\mathcal{T}$ has adhesion-width at most $2k$;

(P2) for each $x \in V(T)$, $G\langle \mathcal{T}, B_x\rangle - A_x \in \mathcal{G}^{+3k}$;
(P3) each component of F has at most $4b-1$ nodes;
(P4) for any two nodes x, y in the same component of F, $|\log \omega(T_x) - \log \omega(T_y)| \leqslant \log b$;
(P5) for each $x \in V(F)$, each light clique K in $G\langle \mathcal{T}, B_x\rangle - A_x$, and each $y \in C_K$, $xy \in Y$; and
(P6) each root-to-leaf path P in T contains at most $2\log_b n + O(1)$ edges in Y.

(Property (P5) holds because if $xy \notin Y$, then x and y lie in the same component of F, so by (d) we would have $\omega(T_y) \geqslant \omega(T_x)/b$, which contradicts the definition of a light clique since $\omega(T_y) \leqslant \sum_{z \in C_K} \omega(T_z) < \omega(T_x)/b$.)

By Lemma 17, $\mathcal{G}^{+3k}$ admits a strongly h_0-orientable weighted (h_1, h_2, h_3) mixed labelling scheme $\mathcal{S} := ((\mu_\tau, \kappa_\tau) : \tau \in \mathcal{T}_{\mathcal{G}^{+3k}})$ with

$$h_0(n) = (2g_0(n) + 3k)2^{3k} \ ,$$
$$h_1(n) = g_1(n) + O(k(n) + \log\log n) \ , \tag{4}$$
$$h_2(n) = g_2(n) + O(\log k(n) + \log\log n) \ , \tag{5}$$
$$h_3(n) = g_3(n) + O(k(n) + \log\log n) \ . \tag{6}$$

For each $n \in \mathbb{N}^+$, let U_n be the n-universal graph defined by $(\mathcal{S}, A)$.

The weight function δ: We now define a weight function $\delta : V(G) \to \mathbb{R}^+$ and, for each $x \in V(T)$, we define $\delta(x) = \delta(B_x \setminus A_x)$. This weight function is designed so that, for any node $x \in V(T)$, $\frac{1}{4b}\omega(T_x) \leqslant \delta(x) \leqslant 3k\omega(T_x)$. In fact, the upper bound will have an even stronger property: For each component C of F, $\delta(C) \leqslant 6k\omega(T_{r(C)})$.

For each $x \in V(T)$, let $C_x^- := \{y \in C_x \mid xy \in Y\}$ and let $C_x^+ := C_x \setminus C_x^-$. For each $x \in V(T)$ and each clique K in $G\langle \mathcal{T}, B_x\rangle - A_x$, let $C_K^- := C_K \cap C_x^-$. For each $x \in V(T)$ and each $v \in B_x \setminus A_x$, let $C_v := \{y \in C_x \mid v \in A_y\}$, let $C_v^- := C_v \cap C_x^-$ and let $C_v^+ := C_v \cap C_x^+$.

For each $x \in V(T)$ and each $v \in B_x \setminus A_x$, define

$$\delta(v) := \omega(v) + \sum_{y \in C_v^-} \omega(T_y) + \tfrac{1}{4b} \cdot \sum_{y \in C_v^+} \omega(T_y) \ .$$

For each $x \in V(T)$, let

$$\delta(x) := \delta(B_x \setminus A_x) \ .$$

The definition of $\delta(v)$ is chosen so that $\delta(x)$ approximates $\omega(T_x)$ for each $x \in V(T)$, in a manner we now make precise. For each $x \in V(T)$, $y \in C_x$, and $v \in B_x \setminus A_x$, $y \in C_v$ if and only if $v \in A_y \setminus A_x$. By property (P1) of $\mathcal{T}$, we have $|A_y| \leqslant 2k \leqslant 3k$ for each $y \in V(T)$, so each $y \in V(T)$ belongs to at most $3k$ sets C_v. Thus,

$$\delta(x) \leqslant \sum_{v \in B_x \setminus A_x} \left(\omega(v) + \sum_{y \in C_v} \omega(T_y) \right) \leqslant \omega(x) + \sum_{y \in C_x} 3k \cdot \omega(T_y) \leqslant 3k \cdot \omega(T_x) \ . \tag{7}$$

Since $\mathcal{T}$ is a tidy tree-decomposition, $A_y \setminus A_x \neq \emptyset$ for each $y \in C_x$, and thus every $y \in C_x$ appears in C_v for at least one $v \in B_x \setminus A_x$. Therefore

$$\delta(x) = \sum_{v \in B_x \setminus A_x} \delta(v) \geqslant \omega(x) + \sum_{y \in C_x^-} \omega(T_y) + \tfrac{1}{4b} \sum_{y \in C_x^+} \omega(T_y) \geqslant \tfrac{1}{4b}\omega(T_x) \tag{8}$$

for each $x \in V(T)$.

Beyond this, the definition of δ is chosen so that $\delta(C)$ approximates $\omega(T_{r(C)})$ for each component C of F:

$$\begin{aligned}
\delta(C) &= \sum_{x\in V(C)} \sum_{v\in B_x\setminus A_x} \left(\omega(v) + \sum_{y\in C_v^-} \omega(T_y) + \tfrac{1}{4b} \sum_{y\in C_v^+} \omega(T_y) \right) \\
&\leqslant \sum_{x\in V(C)} \left(\omega(x) + \sum_{y\in C_x^-} 3k\cdot\omega(T_y) + \tfrac{1}{4b}\cdot \sum_{y\in C_x^+} 3k\cdot\omega(T_y) \right) && \text{(since } |A_y| \leqslant 3k\text{)} \\
&\leqslant 3k \sum_{x\in V(C)} \left(\omega(x) + \sum_{y\in C_x^-} \omega(T_y) \right) + \tfrac{3k}{4b} \sum_{x\in V(C)} \sum_{y\in C_x^+} \omega(T_y) && \text{(since } k \geqslant 1\text{)} \\
&\leqslant 3k \sum_{x\in V(C)} \left(\omega(x) + \sum_{y\in C_x^-} \omega(T_y) \right) + \tfrac{3k}{4b} \sum_{x\in V(C)} \omega(T_x) && \text{(since } T_x \supset \textstyle\bigcup_{y\in C_x^+} T_y\text{)} \\
&\leqslant 3k \sum_{x\in V(C)} \left(\omega(x) + \sum_{y\in C_x^-} \omega(T_y) \right) + \tfrac{3k}{4b} \sum_{x\in V(C)} \omega(T_{r(C)}) && \text{(since } T_{r(C)} \supseteq T_x\text{)} \\
&\leqslant 3k \sum_{x\in V(C)} \left(\omega(x) + \sum_{y\in C_x^-} \omega(T_y) \right) + 3k\cdot\omega(T_{r(C)}) && \text{(since } |V(C)| < 4b\text{, by (P3))} \\
&\leqslant 6k\cdot\omega(T_{r(C)}) \ . && (9)
\end{aligned}$$

where the last inequality holds because the sets in $\{V(C)\} \cup \{V(T_y) \mid x \in V(C), y \in C_x^-\}$ are pairwise disjoint subsets of $V(T_{r(C)})$. The final inequality is most conveniently written as

$$\omega(T_{r(C)}) \geqslant \delta(C)/(6k) \ . \tag{10}$$

The inner-tree codes ρ_C for components C of F: For each component C of F, let $\rho_C : V(C) \to \{0,1\}^*$ be the prefix-free code obtained by applying Corollary 8 to the set $S = V(C)$ with weight function $\delta|_S$ obtained by restricting δ to S. Then, for each $x \in V(C)$:

$$\begin{aligned}
|\rho_C(x)| &\leqslant \log\delta(C) - \log\delta(x) + O(1) && (11)\\
&\leqslant \log(6k\cdot\omega(T_{r(C)})) - \log\delta(x) + O(1) && \text{(by (10))}\\
&= \log\omega(T_{r(C)}) - \log\delta(x) + O(\log\log n) && \text{(since } k \in O(\log n)\text{)}\\
&\leqslant \log\omega(T_{r(C)}) - \log(\omega(T_x)/(4b)) + O(\log\log n) && \text{(by (8))}\\
&\leqslant \log\omega(T_{r(C)}) - \log(\omega(T_{r(C)})/(4b^2)) + O(\log\log n) && \text{(by property (P4) of } F\text{)}\\
&\leqslant 2\log b + O(\log\log n). && (12)
\end{aligned}$$

The inner-node labelling μ_x: The next part of our labelling involves a mixed labelling (μ_x, κ_x) of $\tau_x := (G\langle\mathcal{T}, B_x\rangle - A_x, G[B_x] - A_x, \widehat{\delta_x})$ for each $x \in V(T)$. Here $\widehat{\delta_x}$ is a nice weight function on $B_x \setminus A_x$ that is closely related to $\delta|_{B_x\setminus A_x}$. The only reason for the (distracting)

introduction of $\widehat{\delta_x}$ is that later, we will make use of the fact that $(\mathcal{S}, A, I)$ is strongly h_0-orientable. In particular, the n-universal graph U_n defined by $(\mathcal{S}, A)$ has an orientation of maximum outdegree $h_0(n)$. For this to be useful, we require that the label $\mu_x(v)$ assigned to each vertex $v \in B_x \setminus A_x$ correspond to a vertex in U_n, and this is only guaranteed if $\tau_x \in \mathcal{T}'_{\mathcal{G}^{+3k},\leqslant n}$. This is why we require $\widehat{\delta_x}$ to be nice.

For each $x \in V(T)$, let $m_x := |B_x \setminus A_x|$ and define the weight function $\widehat{\delta_x} : B_x \setminus A_x \to \mathbb{N}^+$ by

$$\widehat{\delta_x}(v) := \left\lceil \frac{m_x \delta(v)}{\delta(x)} \right\rceil .$$

As in the proof of Observation 6, $\widehat{\delta_x}$ is nice, $\widehat{\delta_x}(B_x \setminus A_x) < 2m_x$, and

$$\log \widehat{\delta_x}(B_x \setminus A_x) - \log \widehat{\delta_x}(v) \leqslant \log \delta(x) - \log \delta(v) + 1$$

for each $v \in B_x \setminus A_x$. Let $\tau_x := (G\langle \mathcal{T}, B_x\rangle - A_x, G[B_x] - A_x, \widehat{\delta_x})$ and let $(\mu_x, \kappa_x) := (\mu_{\tau_x}, \kappa_{\tau_x})$. The root bag has $m_{x_0} = 1$, and every other set $B_x \setminus A_x$ excludes v_0, so $m_x \leqslant n$ for each $x \in V(T)$. Thus, $\tau_x \in \mathcal{T}'_{\mathcal{G}^{+3k},\leqslant n}$. Increasing the $O(\log\log n)$ terms in h_1 and h_3 by 1 preserves Equations (4) and (6), so we obtain, for each $x \in V(T)$,

$$|\mu_x(v)| \leqslant \log \delta(x) - \log \delta(v) + h_1(n) \tag{13}$$

for each $v \in B_x \setminus A_x$ and

$$|\mu_x(K)| \leqslant \log \delta(x) - \log(\min_{u \in K} \delta(u)) + h_3(n) \tag{14}$$

for each clique K in $G\langle \mathcal{T}, B_x\rangle - A_x$.

The definition of δ has been carefully chosen so that every non-light clique receives a short label: If K is a non-light clique then

$$\begin{aligned}
\min_{u \in K} \delta(u) &\geqslant \min_{u \in K} \left(\sum_{y \in C_u} \tfrac{1}{4b} \omega(T_y) \right) && \text{(by definition of } \delta) \\
&\geqslant \tfrac{1}{4b} \sum_{y \in C_K} \omega(T_y) && \text{(since } C_K \subseteq C_u \text{ for all } u \in K) \\
&\geqslant \omega(T_x)/(4b^2) && \text{(since } K \text{ is not light).}
\end{aligned} \tag{15}$$

Therefore, for any non-light clique K in $G\langle \mathcal{T}, B_x\rangle - A_x$,

$$\begin{aligned}
|\mu_x(K)| &\leqslant h_3(n) + \log \delta(x) - \log(\min_{u \in K} \delta(u)) \\
&\leqslant h_3(n) + \log \delta(x) - \log(\omega(T_x)/(4b^2)) && \text{(by (15))} \\
&\leqslant h_3(n) + \log(3k \cdot \omega(T_x)) - \log(\omega(T_x)/(4b^2)) \\
&\leqslant h_3(n) + 2\log b + O(\log\log n) \ .
\end{aligned} \tag{16}$$

The inter-tree codes ρ_K: For each $x \in V(T)$ and each clique K in $G\langle \mathcal{T}, B_x\rangle - A_x$ such that $C_K^- \neq \emptyset$, let $\rho_K : C_K^- \to \{0,1\}^*$ be obtained by applying Corollary 8 to the set $S = C_K^-$ with weight defined by $y \mapsto \omega(T_y)$. Then, for each $y \in C_K^-$,

$$\begin{aligned} |\rho_K(y)| &\leqslant \log\left(\textstyle\sum_{y' \in C_K^-} \omega(T_{y'})\right) - \log \omega(T_y) + O(1) \\ &\leqslant \log(\min_{u \in K} \delta(u)) - \log \omega(T_y) + O(1) \ , \end{aligned} \tag{17}$$

since $y' \in C_u^-$ for each $u \in K$.

The outer-tree codes $\tau(C,D)$: Let C and D be two components of F such that some edge $xy \in Y$ joins them, with $x \in V(C)$, $y \in V(D)$ and $x = \mathrm{p}_T(y)$. Thus $y = r(D)$ is the root of component D. Then we define

$$\tau(C,D) := \langle \rho_C(x), \mu_x(A_y \setminus A_x), \rho_{A_y \setminus A_x}(y) \rangle \ .$$

Then

$$\begin{aligned} |\tau(C,D)| &= |\rho_C(x)| + |\mu_x(A_y \setminus A_x)| + |\rho_{A_y \setminus A_x}(y)| + O(\log\log n) \\ &\leqslant \log \delta(C) - \log \delta(x) + |\mu_x(A_y \setminus A_x)| + |\rho_{A_y \setminus A_x}(y)| + O(\log\log n) && \text{(by (11))} \\ &\leqslant \log \delta(C) - \log\left(\min_{u \in A_y \setminus A_x} \delta(u)\right) + h_3(n) + |\rho_{A_y \setminus A_x}(y)| + O(\log\log n) && \text{(by (14))} \\ &\leqslant \log \delta(C) - \log \omega(T_y) + h_3(n) + O(\log\log n) && \text{(by (17))} \\ &\leqslant \log \delta(C) - \log(\delta(D)/(6k)) + h_3(n) + O(\log\log n) && \text{(by (10))} \\ &\leqslant \log \delta(C) - \log \delta(D) + h_3(n) + O(\log\log n) && \text{(since } k \leqslant \log n\text{).} \end{aligned} \tag{18}$$

For a fixed component C of F, $\tau(C,\cdot)$ is a function that maps each component D of F whose root is adjacent to a node of C onto a binary string. For each component C of F, the mapping $\tau(C,\cdot)$ is injective, because of the following: $\rho_C : V(C) \to \{0,1\}^*$ is injective by definition; μ_x is injective over the cliques in $G\langle \mathcal{T}, B_x\rangle - A_x$ for each $x \in V(C)$; and $\rho_K : C_K^- \to \{0,1\}^*$ is injective for each clique K in $G\langle \mathcal{T}, B_x\rangle - A_x$.

The node id $\mathrm{id}(x)$: Refer to Figure 1. Let $x \in V(T)$ and let $x_1y_1, \ldots, x_sy_s$ be the sequence of edges in Y encountered along the path from the root x_0 of T to x. Let $x_{s+1} := x$ and let $y_0 := x_0$ be the root of T. Then $x_iy_i \in Y$ for each $i \in \{1,\ldots,s\}$, and y_i is the root of some component C_i of F for each $i \in \{0,\ldots,s\}$. Define

$$\begin{aligned} \mathrm{id}(C_s) &:= \langle \tau(C_0,C_1), \tau(C_1,C_2), \ldots, \tau(C_{s-1},C_s) \rangle \ , \\ \mathrm{id}(x) &:= \langle \tau(C_0,C_1), \tau(C_1,C_2), \ldots, \tau(C_{s-1},C_s), \rho_{C_s}(x) \rangle \ . \end{aligned}$$

Then

$$\begin{aligned} |\mathrm{id}(C_s)| &\leqslant \sum_{i=0}^{s-1} (|\tau(C_i,C_{i+1})| + O(\log\log n)) \\ &\leqslant \sum_{i=0}^{s-1} (\log \delta(C_i) - \log \delta(C_{i+1})) + s \cdot h_3(n) + O((s+1)\log\log n) && \text{(by (18))} \end{aligned}$$

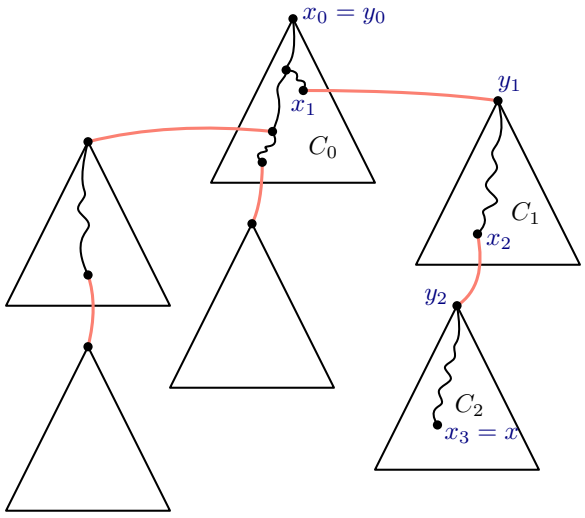


Figure 1: The tree T with edges in Y coloured and an example of the sequences $y_0, x_1, y_1, x_2, y_2, x_3$ and C_0, C_1, C_2 used to define $\mathrm{id}(x) = \langle \tau(C_0, C_1), \tau(C_1, C_2), \rho_{C_2}(x) \rangle$.

$$\begin{aligned}
&= \log \delta(C_0) - \log \delta(C_s) + s \cdot h_3(n) + O((s+1)\log\log n) \\
&= \log \delta(C_0) - \log \delta(C_s) + O((h_3(n) + \log\log n) \cdot \log_b n) && \text{(by (P6))} \\
&\leqslant \log \omega(G) - \log \delta(C_s) + O((h_3(n) + \log\log n) \cdot \log_b n) && \text{(by (9))} \qquad (19)
\end{aligned}$$

and

$$\begin{aligned}
|\mathrm{id}(x)| &= |\mathrm{id}(C_s)| + |\rho_{C_s}(x)| + O(\log\log n) \\
&\leqslant \log \omega(G) - \log \delta(x) + O((h_3(n) + \log\log n) \cdot \log_b n) \qquad \text{(by (19) and (11))} \ . \qquad (20)
\end{aligned}$$

It is straightforward to verify (by induction on s) that the function $\mathrm{id} : V(T) \to \{0,1\}^*$ is injective. (With a slight abuse of notation, for $i \leqslant s$ we say that $\mathrm{id}(C_i)$ is a *prefix* of $\mathrm{id}(C_s)$ and $\mathrm{id}(x)$, and write $\mathrm{id}(C_i) = \mathrm{id}(C_{i-1}) \circ \tau(C_{i-1}, C_i)$ and $\mathrm{id}(x) = \mathrm{id}(C_s) \circ \rho_{C_s}(x)$ to denote the multipart encodings obtained by appending one part.)

The vertex label $\ell(v)$: For each $x \in V(T)$ and each $v \in B_x \setminus A_x$, define

$$\ell(v) := \langle \mathrm{id}(x), \mu_x(v), \alpha(v) \rangle \ .$$

The parts $\mathrm{id}(x)$ and $\mu_x(v)$ are defined above. We now describe $\alpha(v)$, which contains an entry $\alpha(v,u)$ for each $u \in N_G(v) \cap A_x$.

Let $u \in N_G(v) \cap A_x$. Then $u \in B_{x'} \setminus A_{x'}$ for some T-ancestor $x' \neq x$ of x. Let y' be the child of x' on the path from x' to x. The idea is that $\alpha(v,u)$ (in combination with $\mathrm{id}(x)$) should store enough information to reconstruct $\mathrm{id}(x')$, $\mu_{x'}(A_{y'} \setminus A_{x'})$, and $\kappa_{x'}(A_{y'} \setminus A_{x'}, u)$. Let C' be the component of F that contains x' and let D' be the component of $\bar{F}$ that contains y'. Observe that some prefix of $\mathrm{id}(x)$ contains $\mathrm{id}(C')$. Furthermore, we can reconstruct $\mathrm{id}(x')$ by appending $\rho_{C'}(x')$ to $\mathrm{id}(C')$. Let $\mathrm{idx}(C')$ be the number of edges in Y on the path from the root of T to $r(C')$. There are two cases to consider for the edge $x'y'$.

- If $x'y' \in Y$, then $C' \neq D'$. Then $\mathrm{id}(x)$ contains $\mathrm{id}(D')$ as a prefix, and $\mathrm{id}(D')$ contains $\tau(C', D') = \langle \rho_{C'}(x'), \mu_{x'}(A_{y'} \setminus A_{x'}), \rho_{A_{y'} \setminus A_{x'}}(y') \rangle$. In this case, the only extra information needed in $\alpha(v, u)$ is $\mathrm{idx}(C')$ and $\kappa_{x'}(A_{y'} \setminus A_{x'}, u)$.
- If $x'y' \notin Y$, then $C' = D'$. In this case, $\mathrm{id}(x)$ contains $\mathrm{id}(C')$ but does not contain $\mathrm{id}(x')$, since it does not contain $\rho_{C'}(x')$. In this case, $\alpha(v, u)$ will store $\rho_{C'}(x')$ and $\mu_{x'}(A_{y'} \setminus A_{x'})$ in addition to $\mathrm{idx}(C')$ and $\kappa_{x'}(A_{y'} \setminus A_{x'}, u)$. We now explain why these strings are sufficiently short.

  By (12), $|\rho_{C'}(x')| \in O(\log b + \log\log n)$. More importantly, since $C' = D'$, $x'y' \notin Y$, so $A_{y'} \setminus A_{x'}$ is a non-light clique in $G\langle \mathcal{T}, B_{x'} \rangle - A_{x'}$, by (P5). Therefore, $|\mu_{x'}(A_{y'} \setminus A_{x'})| \leqslant h_3(n) + O(\log b + \log\log n)$, by (16). This allows us to store both $\rho_{C'}(x')$ and $\mu_{x'}(A_{y'} \setminus A_{x'})$ in $\alpha(v, u)$.

Summarizing,

$$\alpha(v,u) := \begin{cases} \langle 0, \mathrm{bin}(\mathrm{idx}(C')), \kappa_{x'}(A_{y'} \setminus A_{x'}, u) \rangle & \text{if } x'y' \in Y, \\ \langle 1, \mathrm{bin}(\mathrm{idx}(C')), \rho_{C'}(x'), \mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, u) \rangle & \text{if } x'y' \notin Y. \end{cases}$$

Then, in the first case,

$$|\alpha(v,u)| \leqslant h_2(n) + O(\log\log n) \ .$$

In the second case,

$$|\alpha(v,u)| \leqslant h_3(n) + h_2(n) + O(\log b + \log\log n) \ .$$

In either case, $|\alpha(v,u)| \leqslant h_3(n) + h_2(n) + O(\log b + \log\log n)$. Define

$$\alpha(v) := \langle \alpha(v,u) : u \in N_G(v) \cap A_x \rangle \ .$$

Then,

$$\begin{aligned} |\alpha(v)| &\leqslant |N_G(v) \cap A_x| \cdot (h_3(n) + h_2(n) + O(\log b + \log\log n)) \\ &\leqslant 3k \cdot (h_3(n) + h_2(n) + O(\log b + \log\log n)) \ , && \text{(by property (P1) of } \mathcal{T}) \end{aligned}$$

and, by Equations (13) and (20) and since $\delta(v) \geqslant \omega(v)$,

$$\begin{aligned} |\ell(v)| &\leqslant |\mathrm{id}(x)| + |\mu_x(v)| + |\alpha(v)| + O(\log\log n) \\ &\leqslant \log\omega(G) - \log\omega(v) + h_1(n) + O((h_3(n) + \log\log n)\log_b n) + |\alpha(v)| \\ &\leqslant \log\omega(G) - \log\omega(v) + h_1(n) \\ &\quad + O((h_3(n) + \log\log n)\log_b n) \\ &\quad + O(k \cdot (h_3(n) + h_2(n) + \log b + \log\log n)) \ , \end{aligned}$$

which satisfies the bound on $f(n)$.

We first verify that ℓ_{G_0} is injective. For distinct $v, w \in V(G_0)$, let x and y be their respective home nodes in $\mathcal{T}$. If $x \neq y$, then $\mathrm{id}(x) \neq \mathrm{id}(y)$ because $\mathrm{id} : V(T) \to \{0,1\}^*$ is injective, so $\ell_{G_0}(v) = \langle \mathrm{id}(x), \dots \rangle \neq \langle \mathrm{id}(y), \dots \rangle = \ell_{G_0}(w)$. If $x = y$, then $v, w \in B_x \setminus A_x$ are distinct vertices in the torso at x. By Lemma 13, the vertex labelling μ_x is injective, so $\mu_x(v) \neq \mu_x(w)$, and therefore $\ell_{G_0}(v) \neq \ell_{G_0}(w)$.

The adjacency tester A': We now describe the adjacency tester $A' : \{0,1\}^* \times \{0,1\}^* \to \{0,1\}$ which takes two labels $\ell(v)$ and $\ell(w)$ and returns 1 if and only if $vw \in E(G)$. Suppose $v \in B_x \setminus A_x$ and $w \in B_y \setminus A_y$ for some $x, y \in V(T)$. Then $\ell(v) = \langle \mathrm{id}(x), \mu_x(v), \alpha(v) \rangle$ and $\ell(w) = \langle \mathrm{id}(y), \mu_y(w), \alpha(w) \rangle$. The adjacency tester begins by comparing $\mathrm{id}(x)$ and $\mathrm{id}(y)$. If $\mathrm{id}(x) = \mathrm{id}(y)$ then $x = y$, since $\mathrm{id} : V(T) \to \{0,1\}^*$ is injective. In this case $v, w \in B_x \setminus A_x$, and $vw \in E(G)$ if and only if $vw \in E(G[B_x \setminus A_x])$. In this case, the tester returns $A'(\ell(v), \ell(w)) := A(\mu_x(v), \mu_y(w))$.

Now suppose that $\mathrm{id}(x) \neq \mathrm{id}(y)$, so $x \neq y$. Since $x \neq y$, the only way in which v and w can be adjacent in G is if $v \in N_G(w) \cap A_y$ or $w \in N_G(v) \cap A_x$. The former case requires that x be a T-ancestor of y, and the latter case requires that y be a T-ancestor of x. Let C be the component of F that contains x and let D be the component of F that contains y. Removing the last part of $\mathrm{id}(x)$ gives $\mathrm{id}(C)$ and removing the last part of $\mathrm{id}(y)$ gives $\mathrm{id}(D)$, so the adjacency tester has access to $\mathrm{id}(C)$ and $\mathrm{id}(D)$. There are several cases to consider:

1. If $\mathrm{id}(C) \neq \mathrm{id}(D)$ but $\mathrm{id}(C)$ is a prefix of $\mathrm{id}(D)$, then the only way in which v and w can be adjacent is if x is a T-ancestor of y and $v \in N_G(w) \cap A_y$. To check if this is the case, the adjacency tester examines each entry $\alpha(w, u)$ in $\alpha(w)$ and tests if $u = v$. If it finds such an entry, then $vw = uw \in E(G)$, so $A'(\ell(v), \ell(w)) := 1$. If no such entry is found, $A'(\ell(v), \ell(w)) := 0$. All that remains is to explain how A' tests if $v = u$ for some entry $\alpha(w, u)$ in $\alpha(w)$. An entry $\alpha(w, u)$ can have one of two forms, which are distinguished by their first part.

   - If $\alpha(w, u) = \langle 0, \mathrm{bin}(\mathrm{idx}(C')), \kappa_{x'}(A_{y'} \setminus A_{x'}, u) \rangle$, then $u \in B_{x'} \setminus A_{x'}$ and C' is the component of F that contains x'. Furthermore $x'y' \in Y$, so y' is in a different component $D' \neq C'$ of F than x'. Then, the first $\mathrm{idx}(C') + 1$ parts of $\mathrm{id}(D)$ are $\mathrm{id}(D') = \mathrm{id}(C') \circ \tau(C', D')$. The last part of $\mathrm{id}(D')$ is $\tau(C', D') = \langle \rho_{C'}(x'), \mu_{x'}(A_{y'} \setminus A_{x'}), \rho_{A_{y'} \setminus A_{x'}}(y') \rangle$. Therefore, the tester can extract $\mathrm{id}(C')$ and $\mathrm{id}(x') = \mathrm{id}(C') \circ \rho_{C'}(x')$ using $\mathrm{id}(D)$ and $\mathrm{idx}(C')$. After doing this, the tester checks if $\mathrm{id}(x') = \mathrm{id}(x)$. If not, then $x' \neq x$ and $u \neq v$ since $u \in B_{x'} \setminus A_{x'}$ and $v \in B_x \setminus A_x$. If $\mathrm{id}(x') = \mathrm{id}(x)$ then $x' = x$, since $\mathrm{id} : V(T) \to \{0,1\}^*$ is injective. In this case, the tester can check if $u = v$ by checking if $I(\mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, u), \mu_x(v)) = 1$ (the first argument to $I(\cdot,\cdot,\cdot)$ comes from $\tau(C', D')$, the second from $\alpha(w, u)$ and the third from $\ell(v) = \langle \mathrm{id}(x), \mu_x(v), \alpha(v) \rangle$). If so, then $v = u \in N_G(w)$ so the tester outputs $A'(\ell(v), \ell(w)) := 1$. If not, the tester continues with the next entry (if any) in $\alpha(w)$.
   - If $\alpha(w, u) = \langle 1, \mathrm{bin}(\mathrm{idx}(C')), \rho_{C'}(x'), \mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, u) \rangle$, then $u \in B_{x'} \setminus A_{x'}$ and C' is the component of F that contains x'. In this case, the first $\mathrm{idx}(C')$ parts of $\mathrm{id}(y)$ are $\mathrm{id}(C')$. The tester computes $\mathrm{id}(x') = \mathrm{id}(C') \circ \rho_{C'}(x')$. After doing this, the tester checks if $\mathrm{id}(x') = \mathrm{id}(x)$. If not, then $x' \neq x$ and $u \neq v$ since $u \in B_{x'} \setminus A_{x'}$ and $v \in B_x \setminus A_x$. If $\mathrm{id}(x') = \mathrm{id}(x)$ then $x' = x$ since id is injective. In this case, the tester can check if $u = v$ by checking if $I(\mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, u), \mu_x(v)) = 1$ (the first and second arguments to $I(\cdot,\cdot,\cdot)$ come from $\alpha(w, u)$ and the third comes from $\ell(v) = \langle \mathrm{id}(x), \mu_x(v), \alpha(v) \rangle$). If so, then $v = u \in N_G(w)$ so the tester outputs $A'(\ell(v), \ell(w)) := 1$. If not, the tester continues with the next entry (if any) in $\alpha(w)$.

2. If $\mathrm{id}(C) \neq \mathrm{id}(D)$ but $\mathrm{id}(D)$ is a prefix of $\mathrm{id}(C)$, then the tester proceeds exactly as in the previous case, but swapping the roles of v and w.

3. If $\mathrm{id}(C) = \mathrm{id}(D)$ then $C = D$ and x and y are in the same component of F. In this case, there are two ways in which v and w can be adjacent: Either x is a T-ancestor of y and $v \in N_G(w) \cap A_y$ or y is a T-ancestor of x and $w \in N_G(v) \cap A_x$. Case 1 above explains how to test the former and Case 2 explains how to test the latter. The fact that $\mathrm{id}(C) = \mathrm{id}(D)$ simplifies some parts of the test, but also introduces a new complication. Specifically, the tester does not immediately know if x is an ancestor of y, y is an ancestor of x, or if neither of x nor y is an ancestor of the other. Therefore, the tester must check if $u = v$ for some $u \in N_G(w) \cap A_y$ and must also check if $u = w$ for some $u \in N_G(v) \cap A_x$. We only explain how to test if $u = v$ for some $u \in N_G(w) \cap A_y$, since testing if $u = w$ for some $u \in N_G(v) \cap A_x$ can be done in a symmetric manner.

   The tester tests each entry $\alpha(w,u)$ in $\alpha(w)$. There are two possibilities, that are distinguished by the first part of $\alpha(w,u)$.

   - If $\alpha(w,u) = \langle 0, \mathrm{bin}(\mathrm{idx}(C')), \kappa_{x'}(A_{y'} \setminus A_{x'}, u)\rangle$, then $u \in B_{x'} \setminus A_{x'}$ and C' is the component of F that contains x'. In this case, the edge $x'y' \in Y$, so x' and y' are in different components of F. In particular, $C' \neq C$, so $x' \neq x$. Thus $u \neq v$ since $u \in B_{x'} \setminus A_{x'}$ and $v \in B_x \setminus A_x$.
   - If $\alpha(w,u) = \langle 1, \mathrm{bin}(\mathrm{idx}(C')), \rho_{C'}(x'), \mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, u)\rangle$, then $u \in B_{x'} \setminus A_{x'}$ and C' is the component of F that contains x'. In this case, the first $\mathrm{idx}(C')$ parts of $\mathrm{id}(y)$ give $\mathrm{id}(C')$. Then $\mathrm{id}(x') = \mathrm{id}(C') \circ \rho_{C'}(x')$. The tester first checks that $\mathrm{id}(x') = \mathrm{id}(x)$. If not, then $x' \neq x$ and $u \neq v$ since $u \in B_{x'} \setminus A_{x'}$ and $v \in B_x \setminus A_x$. If $\mathrm{id}(x') = \mathrm{id}(x)$ then $x' = x$ since id is injective. In this case, the tester can check if $u = v$ by checking if $I(\mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, u), \mu_x(v)) = 1$. If so, then $v = u \in N_G(w)$ so the tester outputs $A'(\ell(v), \ell(w)) := 1$. If not, the tester continues with the next entry (if any) in $\alpha(w)$.

4. If neither $\mathrm{id}(C)$ nor $\mathrm{id}(D)$ is a prefix of the other then no node in C is a T-ancestor of a node in D and no node in D is a T-ancestor of a node in C. Thus x is not a T-ancestor of y and y is not a T-ancestor of x. Thus $vw \notin E(G)$ so $A'(\ell(v), \ell(w)) := 0$.

g_0'-orientability: All that remains is to argue that the adjacency labelling scheme we have just described is g_0'-orientable. For each $G_0 \in \mathcal{G}'$, let $\ell_{G_0} : V(G_0) \to \{0,1\}^*$ be the adjacency labelling of G_0 for A' defined above (by restricting ℓ to $V(G_0) = V(G) \setminus \{v_0\}$). Let $\mathcal{A}' := (\ell_{G_0} : G_0 \in \mathcal{G}')$. We must show that, for each $n \in \mathbb{N}^+$, the n-universal graph U_n' defined by $(\mathcal{A}', A')$ has an orientation in which each vertex has outdegree at most $g_0'(n)$.

For each $n \in \mathbb{N}^+$ and each edge $\beta\beta'$ in U_n' there exists a graph $G_0 \in \mathcal{G}'$ with at most n vertices and an edge $vv' \in E(G_0)$ such that $\beta = \ell_{G_0}(v)$ and $\beta' = \ell_{G_0}(v')$. Let $\ell := \ell_G$ where G is the graph obtained from G_0 as above. We now explain how to choose the orientation of the edge $\beta\beta'$ by examining the operation of the adjacency tester on the input pair $(\ell_{G_0}(v), \ell_{G_0}(v'))$.[4] Let $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ be the tree-decomposition of G used to define

[4] Although this doesn't happen, it would not matter if the orientation of $\beta\beta'$ depended on the choice of G_0,

ℓ_G and let $((\mu_x, \kappa_x) \mid x \in V(T))$, $Y \subseteq E(T)$ and $F = T - Y$ be defined as above. Let x be the home node of v in $\mathcal{T}$, let x' be the home node of v' in $\mathcal{T}$, let C be the component of F that contains x and let C' be the component of F that contains x'. By the definition of the adjacency tester A', exactly one of the following cases applies:

0. $\mathrm{id}(x) = \mathrm{id}(x')$. In this case $x = x'$ since $\mathrm{id} : V(T) \to \{0,1\}^*$ is injective. Thus, $v, v' \in B_x \backslash A_x$. Then $1 = A'(\ell(v), \ell(v')) = A(\mu_x(v), \mu_x(v'))$. The triple $\tau_x = (G\langle \mathcal{T}, B_x\rangle - A_x, G[B_x] - A_x, \widehat{\delta_x})$ belongs to $\mathcal{T}'_{\mathcal{G}^{+3k}, \leqslant n}$: its graph has $m_x \leqslant n$ vertices and its positive integer weights sum to less than $2m_x \leqslant 2n$. Since $vv' \in E(G[B_x] - A_x)$, this triple witnesses that $\mu_x(v)\mu_x(v')$ is an edge of the universal graph U_n for $(\mathcal{S}, A)$. Since the mixed labelling scheme for (A, I) is h_0-orientable, there is an orientation of U_n of outdegree at most $h_0(n)$. Without loss of generality, suppose that this orientation orients the edge $\mu_x(v)\mu_x(v')$ towards $\mu_x(v')$. In this case, we orient the edge $\beta\beta'$ of U'_n towards β'.

   We now bound the maximum contribution to the outdegree of β from this case. When $\mathrm{id}(x) = \mathrm{id}(x')$ and U_n orients $\mu_x(v)\mu_x(v')$ towards $\mu_x(v')$, the number of possible values for $\mu_{x'}(v')$ is at most $h_0(n)$. Since $|\alpha(v')| \in O(k(h_3(n)+h_2(n)+\log b+\log\log n))$, the number of possible values for $\alpha(v')$ is at most $\exp(O(k(h_3(n)+h_2(n)+\log b+\log\log n)))$. Thus, the number of edges of U'_n incident to β that are oriented away from β by this branch of the adjacency testing procedure is at most $h_0(n) \cdot \exp(O(k(h_3(n) + h_2(n) + \log b + \log\log n)))$.

1. $\mathrm{id}(x) \neq \mathrm{id}(x')$. In this case, $x \neq x'$. Since $vv' \in E(G)$, one of x or x' is a T-ancestor of the other. We orient the edge $\beta\beta'$ towards the endpoint whose home node is an ancestor of the other's home node; that is, if x' is a T-ancestor of x, we orient the edge $\beta\beta'$ towards β'. Consequently, an edge is oriented away from β towards β' only when the home node x' of β' is an ancestor of x. Since x' is a T-ancestor of x, $\mathrm{id}(C')$ is a prefix of $\mathrm{id}(C)$ and is completely determined by $\mathrm{id}(C)$ and $\mathrm{idx}(C')$. Furthermore, $\mathrm{idx}(C')$ is contained in $\alpha(v, v')$, which is one of the at most $3k$ entries in $\alpha(v)$. Thus, there are at most $3k$ choices for $\mathrm{id}(C')$. Then $\mathrm{id}(x')$ is obtained by appending $\rho_{C'}(x')$ to $\mathrm{id}(C')$. Since $|\rho_{C'}(x')| \leqslant 2\log b + O(\log\log n)$, there are at most $O(b^2 \log n)$ choices for $\rho_{C'}(x')$. Thus, there are at most $O(kb^2 \log n)$ choices for $\mathrm{id}(x')$.

   Let y' be the T-child of x' that is on the path in T from x' to x. There are now two cases to consider.

   - If $\alpha(v, v') = \langle 0, \mathrm{bin}(\mathrm{idx}(C')), \kappa_{x'}(A_{y'} \setminus A_{x'}, v')\rangle$ then the edge $x'y'$ is in Y. Let D' be the component of $T - Y$ that contains y'. Then $\mathrm{id}(D')$ is a prefix of $\mathrm{id}(C)$ and the last part of $\mathrm{id}(D')$ is $\tau(C', D') = \langle \rho_{C'}(x'), \mu_{x'}(A_{y'} \setminus A_{x'}), \rho_{A_{y'} \setminus A_{x'}}(y')\rangle$. Since $vv' \in E(G)$, and x' is the home node of v' in $\mathcal{T}$, $v' \in A_{y'} \setminus A_{x'}$. By definition, $I(\mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, v'), \mu_{x'}(v')) = 1$. The triple $\tau_{x'} \in \mathcal{T}'_{\mathcal{G}^{+3k}, \leqslant n}$, together with the clique $A_{y'} \setminus A_{x'}$ and vertex v', witnesses the occurrence of this clique label, local identifier, and vertex label. Since $(\mathcal{S}, A, I)$ is strongly h_0-orientable, there are at most $h_0(n)$ choices for $\mu_{x'}(v')$ for each fixed clique label and local identifier.

since we will argue that the number of edges oriented away from a fixed β by each branch of the adjacency testing algorithm is at most $g'_0(n)$.

- If $\alpha(v,v') = \langle 1, \mathrm{bin}(\mathrm{idx}(C')), \rho_{C'}(x'), \mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, v') \rangle$, then $x'y' \notin Y$. Since $v' \in A_{y'} \setminus A_{x'}$, we have $I(\mu_{x'}(A_{y'} \setminus A_{x'}), \kappa_{x'}(A_{y'} \setminus A_{x'}, v'), \mu_{x'}(v')) = 1$ by definition. The triple $\tau_{x'} \in \mathcal{T}'_{\mathcal{G}^{+3k}, \leqslant n}$, together with the clique $A_{y'} \setminus A_{x'}$ and vertex v', witnesses the occurrence of this clique label, local identifier, and vertex label. Since $(\mathcal{S}, A, I)$ is strongly h_0-orientable, there are at most $h_0(n)$ choices for $\mu_{x'}(v')$ for each fixed clique label and local identifier.

In either case, there are at most $h_0(n)$ choices for $\mu_{x'}(v')$.

Summarizing, there are at most $O(kb^2 \log n) \leqslant \exp(O(\log b + \log\log n))$ choices for $\mathrm{id}(x')$. There are at most $h_0(n) = (2g_0(n) + 3k)2^{3k} \leqslant g_0(n)\exp(O(k))$ choices for $\mu_{x'}(v')$. There are at most $\exp(O(k(h_3(n) + h_2(n) + \log b + \log\log n)))$ choices for $\alpha(v')$. Thus, the number of edges of U'_n incident to β that are oriented away from β by this branch of the adjacency testing procedure is at most $h_0(n) \cdot \exp(O(k(h_3(n) + h_2(n) + \log b + \log\log n)))$.

Thus, U'_n has an orientation of outdegree at most $g'_0(n) \leqslant g_0(n)\exp(O(k(h_3(n)+h_2(n)+\log b+\log\log n)))$. □

4 Proof of Theorems 1, 3 and 4 and Corollary 2 with Explicit Bounds

We now prove our main results, Theorems 1, 3 and 4 and Corollary 2, with explicit lower order terms. We rely on the following lemma, which we prove in Appendix A.

Lemma 19. *For any fixed integer $k \geqslant 0$, the graph class $\mathcal{R}_k$ admits a strongly g_0-orientable weighted (g_1, g_2, g_3) mixed labelling scheme, where*

$$\begin{aligned} g_0(n) &\leqslant \exp(O(\sqrt{\log n})) \ , \\ g_1(n) &\in O(\sqrt{\log n}) \ , \\ g_2(n) &\in O(1) \ , \\ g_3(n) &\in O(\sqrt{\log n}) \ . \end{aligned}$$

(More generally, for any non-decreasing function $k : \mathbb{N} \to \mathbb{N}$ with $k(n) \in o(\log n)$, the same scheme achieves $g_0(n) \leqslant \exp(O(k(n)\sqrt{\log n}))$, $g_1(n) \in O(k(n)\sqrt{\log n})$, $g_3(n) \in O(k(n)\sqrt{\log n})$, and $g_2(n) \leqslant \lceil \log(2k(n)+2) \rceil \in O(\log(k(n)+2))$.)

Lemma 20. *Every proper minor-closed graph class $\mathcal{C}$ admits an injective $\exp(O((\log n)^{3/4}))$-orientable $(\log n + O\big((\log n)^{3/4}\big))$-bit adjacency labelling scheme.*

Proof. Let $k, a \geqslant 0$ be integers given by Theorem 5 for the class $\mathcal{C}$. Since $\mathcal{R}_k$ is monotone, $\mathcal{R}_k^{+a}$ is also monotone. By Lemma 19 (if $a = 0$) or by Lemmas 17 and 19 (if $a \geqslant 1$), $\mathcal{R}_k^{+a}$ admits a strongly $\exp(O(\sqrt{\log n}))$-orientable weighted (g'_1, g'_2, g'_3) mixed labelling scheme, where $g'_1(n), g'_3(n) \in O(\sqrt{\log n})$ and $g'_2(n) \in O(1)$.

Let $\mathcal{Q}$ be the class of graphs that have a tree-decomposition whose torsos are in $\mathcal{R}_k^{+a}$. The graphs in $\mathcal{R}_k^{+a}$ have maximum clique size at most $2k + a + 2$. Since each adhesion

of a tree-decomposition is a clique in some torso, each graph in $\mathcal{Q}$ has a rooted tree-decomposition of adhesion-width at most $k' := 2k + a + 2$ whose torsos are in $\mathcal{R}_k^{+a}$.

By Lemma 18 with $b(n) = \lceil 2^{(\log n)^{3/4}} \rceil$, $\mathcal{Q}$ admits an injective $\exp(O((\log n)^{3/4}))$-orientable $f(n)$-bit adjacency labelling scheme with

$$\begin{aligned} f(n) &\leqslant \log n + g_1'(n) + O\Big((g_3'(n) + k' + \log\log n)\log_{b(n)} n \\ &\qquad + k'(g_3'(n) + g_2'(n) + k' + \log b(n) + \log\log n)\Big) \\ &= \log n + O((\log n)^{3/4}) \ . \end{aligned}$$

By the choice of k and a, $\mathcal{C} \subseteq \mathcal{Q}$. Therefore, $\mathcal{C}$ admits an injective $\exp(O((\log n)^{3/4}))$-orientable $f(n)$-bit adjacency labelling scheme with $f(n) = \log n + O((\log n)^{3/4})$. □

Together with Subsection 3.2, Lemma 20 immediately yields our main results:

Proof of Theorems 1 and 3 and Corollary 2. Let $\mathcal{C}$ be a proper minor-closed graph class. By Lemma 20, $\mathcal{C}$ admits an $f(n)$-bit adjacency labelling scheme with $f(n) = \log n + O((\log n)^{3/4}) = (1 + o(1))\log n$. This proves Theorem 1.

Furthermore, this scheme is $g_0'(n)$-orientable with $g_0'(n) \leqslant \exp(O((\log n)^{3/4})) = n^{o(1)}$. For each $n \in \mathbb{N}^+$, the n-universal graph U_n defined by this scheme has $|V(U_n)| \leqslant 2^{f(n)+1} = n \cdot 2^{O((\log n)^{3/4})} = n^{1+o(1)}$ vertices. By orientability, U_n has an orientation with maximum outdegree at most $g_0'(n)$, so $|E(U_n)| \leqslant |V(U_n)| \cdot g_0'(n) \leqslant n^{1+o(1)} \cdot n^{o(1)} = n^{1+o(1)}$ edges. Since every n-vertex graph in $\mathcal{C}$ is isomorphic to an induced subgraph of U_n, this completes the proof of Corollary 2 and Theorem 3. □

Proof of Theorem 4. Let $k \geqslant 1$ be an integer. By Lemma 19, the class $\mathcal{R}_k$ admits a strongly g_0-orientable weighted (g_1, g_2, g_3) mixed labelling scheme with $g_1(n) \in O(k\sqrt{\log n})$ and $g_0(n) \leqslant \exp(O(k\sqrt{\log n}))$. By Observation 15, $\mathcal{R}_k$ admits an injective g_0-orientable $f(n)$-bit adjacency labelling scheme with $f(n) = \log n + g_1(n) = \log n + O(k\sqrt{\log n})$. For each $n \in \mathbb{N}^+$, the n-universal graph U_n^k defined by this scheme has $|V(U_n^k)| \leqslant 2^{f(n)+1} = n \cdot \exp(O(k\sqrt{\log n}))$ vertices and, by orientability, at most $|V(U_n^k)| \cdot g_0(n) \leqslant n \cdot \exp(O(k\sqrt{\log n}))$ edges. By injectivity of the vertex labels, every n-vertex graph in $\mathcal{R}_k$ is isomorphic to an induced subgraph of U_n^k. □

5 Discussion

We conclude with a brief discussion about constants, computational complexity, and extensions beyond minor-closed classes. The constant a in Theorem 5 is surprisingly small. Suppose that some graph X is not in $\mathcal{G}$, where $X - A$ is planar for some non-empty set $A \subseteq V(X)$. Then Theorem 42 in [13] (which employs the structure theorem of Dvořák and Thomas [15]) shows that Theorem 5 holds with $a = |A| - 1$. For example, if K_t is not in $\mathcal{G}$, then Theorem 5 holds with $a = \max\{t - 5, 0\}$. Moreover, using the recent polynomial bounds in the Graph Minor Structure Theorem by Gorsky, Seweryn, and Wiederrecht [22], in Theorem 5 one can obtain polynomial bounds on k and a (as a function of the excluded

minor). However, it is open whether one can simultaneously get the above optimal bound on a and a polynomial bound on k. Gorsky et al. [22, Conjecture 18.9] conjecture this is possible (also see the discussion in [23]).

We now explain why the proof of Theorem 1 is constructive and yields a polynomial-time labelling algorithm. The only difficult part of this process is finding the decomposition described by the Graph Minor Product Structure Theorem (Theorem 5). Dujmović et al. [13] show how to obtain this decomposition in polynomial time, given the structural decomposition in the original Robertson–Seymour Graph Minor Structure Theorem. Finding the Robertson–Seymour decomposition is complicated, but polynomial-time algorithms exist [9, 22, 28, 29]. Together, these give the parameters k and a, the tree-decomposition $\mathcal{T}_0$ of G_0, and the embedding of each torso of $\mathcal{T}_0$ into a graph in $\mathcal{R}_k^{+a}$. The adjacency labelling scheme in [11] and its modification to weighted mixed labellings in Appendix A and Lemma 17 are easily implemented in polynomial time directly from their definitions. The transformation of $\mathcal{T}_0$ into the tidy rooted tree-decomposition $\mathcal{T} = (T, (B_x \mid x \in V(T)))$ and edge set $Y \subseteq E(T)$ in Lemma 12 (via Lemmas 9–11) also runs in polynomial time. Finally, the construction in Lemma 18 requires only computing the weight function δ, alphabetic prefix-free codes (Lemma 7 and Corollary 8), and weighted mixed labellings on the modified torsos $G\langle \mathcal{T}, B_x\rangle - A_x \in \mathcal{R}_k^{+a+3k'}$ (where $k' := 2k+a+2$), and then assembling each vertex label $\ell(v) = \langle \mathrm{id}(x), \mu_x(v), \alpha(v)\rangle$ from the root-to-x path in T.

Finally, we note that Lemma 18 applies beyond proper minor-closed classes. Recall that, for fixed k, $\mathcal{R}_k$ consists of the graphs isomorphic to subgraphs of $H \boxtimes P$ for some graph H of treewidth at most k and some path P. This class is monotone and has an efficient strongly orientable weighted mixed labelling scheme (Lemma 19), so Lemma 18 applies to it. Moreover, for suitable $k = k(p)$, $\mathcal{R}_k$ contains every p-planar graph (graphs that can be drawn in the plane with at most p crossings per edge) [14], and more generally every p-matching-planar graph [24] (graphs that have a drawing in the plane such that, for every edge e, every matching among the edges crossing e has size at most p). Note that these classes are not minor-closed. Consequently, for all fixed p and q, the class of graphs having a tree-decomposition of adhesion-width at most q in which every torso is p-matching-planar (again a class that is not minor-closed) has a $(1+o(1))\log n$-bit adjacency labelling scheme and n-induced-universal graphs with $n^{1+o(1)}$ vertices and edges.

Acknowledgement

This work was initiated during the Ninth Annual Workshop on Geometry and Graphs (v2), held January 21–28, 2022, at the Bellairs Research Institute of McGill University. The authors are thankful to the other organizers and participants for providing a stimulating research environment.

Research History and Statement on AI Use

The results in this paper are the product of years of work by the authors, all of whom have been thinking about adjacency-labelling and sparse universal graphs since at least February 2020, when the work on planar graphs resulting in publication [11] began at the Eighth Workshop on Geometry and Graphs. The work on adjacency labelling for proper

minor-closed graph families began at the 2022 workshop mentioned above and several of the ideas in the paper (using weights to deal with an unbalanced tree-decomposition and using clique labels as a means of navigating a tree-decomposition) come from that week. The final ideas (decomposition into skinny subtrees and a mixed labelling scheme for skinny subtrees) needed to complete the adjacency labelling scheme for proper minor-closed graph classes that resulted in publication [12] were obtained during a visit to Ottawa in February 2026. The strongly g_0-orientable version of this scheme that yields sparse universal graphs was developed during a visit to Kraków in June 2026. The revision history is visible at https://github.com/patmorin/mflabelling.

Piotr Micek gave a talk about adjacency labelling for proper minor-closed graph classes at the MATRIX Workshop on Global Structure and Geometry of Graphs in Creswick, Australia in April 2026. Pat Morin gave a similar talk at the Jagiellonian TCS Seminar in June 2026. Vida Dujmović gave a talk about sparse induced-universal graphs for proper minor-closed graph classes at the 11th Polish Combinatorial Conference in September 2026.

Every line of LaTeX in this paper was written by the authors. One of the authors, Pat Morin, used an AI-enabled text editor (Antigravity IDE with Gemini) to proofread the manuscript and suggest corrections for minor errors. The final version was checked and approved by all authors.

References

[1] Stephen Alstrup, Søren Dahlgaard, and Mathias Bæk Tejs Knudsen. Optimal induced universal graphs and adjacency labeling for trees. *J. ACM*, 64(4):27:1–27:22, 2017. doi:10.1145/3088513. Preliminary version appeared in FOCS 2015, pages 1311–1326.

[2] László Babai, Fan R. K. Chung, Paul Erdős, Ron L. Graham, and Joel H. Spencer. On graphs which contain all sparse graphs. In *Theory and practice of combinatorics*, volume 60 of *North-Holland Math. Stud.*, pages 21–26. North-Holland, 1982.

[3] Sandeep N. Bhatt, Fan R. K. Chung, F. Thomson Leighton, and Arnold L. Rosenberg. Universal graphs for bounded-degree trees and planar graphs. *SIAM Journal on Computing*, 18(6):1144–1159, 1989.

[4] Hans L. Bodlaender and Torben Hagerup. Parallel algorithms with optimal speedup for bounded treewidth. *SIAM J. Comput.*, 27(6):1725–1746, 1998. doi:10.1137/S0097539795289859.

[5] Marthe Bonamy, Cyril Gavoille, and Michał Pilipczuk. Shorter labeling schemes for planar graphs. *SIAM J. Discrete Math.*, 36(3):2082–2099, 2022. doi:10.1137/20M1330464. Preliminary version appeared in SODA 2020, pages 446–462.

[6] Krishnendu Chatterjee, Rasmus Ibsen-Jensen, and Andreas Pavlogiannis. Optimal reachability and a space-time tradeoff for distance queries in constant-treewidth graphs. In *24th Annual European Symposium on Algorithms, ESA 2016*, volume 57 of *LIPIcs*, pages 28:1–28:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPIcs.ESA.2016.28.

[7] Fan R. K. Chung. Universal graphs and induced-universal graphs. *J. Graph Theory*, 14(4):443–454, 1990. doi:10.1002/jgt.3190140408.

[8] Fan R. K. Chung, Ronald L. Graham, and Nicholas Pippenger. On graphs which contain all small trees II. In *Combinatorics (Proc. Fifth Hungarian Colloq., Keszthely, 1976)*, *Vol. I*, *Colloq. Math. Soc. János Bolyai*, *18*, pages 213–223. North-Holland, 1978.

[9] Erik D. Demaine, MohammadTaghi Hajiaghayi, and Ken ichi Kawarabayashi. Algorithmic graph minor theory: Decomposition, approximation, and coloring. In *Proc. 46th Annual IEEE Symposium on Foundations of Computer Science* (FOCS'05), pages 637–646. IEEE Computer Society, 2005. doi:10.1109/SFCS.2005.14.

[10] Matt DeVos, Guoli Ding, Bogdan Oporowski, Daniel P. Sanders, Bruce A. Reed, Paul D. Seymour, and Dirk Vertigan. Excluding any graph as a minor allows a low tree-width 2-coloring. *J. Comb. Theory, Ser. B*, 91(1):25–41, 2004. doi:10.1016/j.jctb.2003.09.001.

[11] Vida Dujmović, Louis Esperet, Cyril Gavoille, Gwenaël Joret, Piotr Micek, and Pat Morin. Adjacency labelling for planar graphs (and beyond). *J. ACM*, 68(6):42:1–42:33, 2021. doi:10.1145/3477542. Preliminary version appeared in FOCS 2020, pages 577–588.

[12] Vida Dujmović, Cyril Gavoille, Gwenaël Joret, Piotr Micek, Pat Morin, and David R. Wood. Adjacency labeling schemes for proper minor-closed classes. In *SODA 2027*. 2027. To appear.

[13] Vida Dujmović, Gwenaël Joret, Piotr Micek, Pat Morin, Torsten Ueckerdt, and David R. Wood. Planar graphs have bounded queue-number. *J. ACM*, 67(4):22:1–22:38, 2020. doi:10.1145/3385731. Preliminary version appeared in FOCS 2019, pages 862–875.

[14] Vida Dujmović, Pat Morin, and David R. Wood. Graph product structure for non-minor-closed classes. *J. Combin. Theory, Ser B.*, 162:34–67, 2023.

[15] Zdeněk Dvořák and Robin Thomas. List-coloring apex-minor-free graphs. *arXiv:1401.1399*, 2014. doi:10.48550/1401.1399.

[16] Peter Elias. Universal codeword sets and representations of the integers. *IEEE Trans. Inf. Theory*, 21(2):194–203, 1975. doi:10.1109/TIT.1975.1055349.

[17] Louis Esperet, Gwenaël Joret, and Pat Morin. Sparse universal graphs for planarity. *J. London Math. Soc.*, 108(4):1333–1357, 2023. doi:10.1112/jlms.12781.

[18] Cyril Gavoille and Arnaud Labourel. Shorter implicit representation for planar graphs and bounded treewidth graphs. In Lars Arge, Michael Hoffmann, and Emo Welzl, editors, *Proc. 15th Annual European Symposium on Algorithms* (ESA 2007), volume 4698 of *Lecture Notes in Comput. Sci.*, pages 582–593. Springer, 2007. doi:10.1007/978-3-540-75520-3_52.

[19] Pawel Gawrychowski and Wojciech Janczewski. Simpler adjacency labeling for planar graphs with B-trees. In Karl Bringmann and Timothy M. Chan, editors, *Proc. 5th Symposium on Simplicity in Algorithms* (SOSA@SODA 2022), pages 24–36. SIAM, 2022. doi:10.1137/1.9781611977066.3.

[20] Edgar N. Gilbert and Edward F. Moore. Variable-length binary encodings. *Bell System Technical Journal*, 38(4):933–967, 1959. doi:10.1002/j.1538-7305.1959.tb01583.x.

[21] Paul C. Gilmore and Alan J. Hoffman. A characterization of comparability graphs

and of interval graphs. *Canadian J. Math.*, 16:539–548, 1964. doi:10.4153/CJM-1964-055-5.

[22] Maximilian Gorsky, Michal T. Seweryn, and Sebastian Wiederrecht. Polynomial bounds for the graph minor structure theorem. In *Proc. 66th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2025*, pages 1961–1978. IEEE, 2025. doi:10.1109/FOCS63196.2025.00104.

[23] Maximilian Gorsky, Michał T. Seweryn, and Sebastian Wiederrecht. The price of homogeneity is polynomial. *arXiv:2602.01882*, 2026. doi:10.48550/arXiv.2602.01882.

[24] Kevin Hendrey, Nikolai Karol, and David R. Wood. Structure of k-matching-planar graphs. *CoRR*, abs/2507.22395, 2025. doi:10.48550/ARXIV.2507.22395. 2507.22395.

[25] Rahul Jain and Raghunath Tewari. Space efficient algorithm for solving reachability using tree decomposition and separators. *Theoretical Computer Science*, 982:114251, 2024. doi:10.1016/j.tcs.2023.114251.

[26] Sampath Kannan, Moni Naor, and Steven Rudich. Implicit representation of graphs. *SIAM J. Discrete Math.*, 5(4):596–603, 1992. doi:10.1137/0405049. Preliminary version appeared in STOC 1988, pages 334–343.

[27] Neel Kaul, Jaehoon Kim, Minseo Kim, and David R. Wood. On universal graphs for trees and tree-like graphs. *arXiv:2511.22358*, 2025. doi:10.48550/arXiv.2511.22358. 2511.22358.

[28] Ken ichi Kawarabayashi, Robin Thomas, and Paul Wollan. A new proof of the flat wall theorem. *J. Combinatorial Theory, Series B*, 129:158–203, 2018. doi:10.1016/j.jctb.2017.09.002.

[29] Ken-ichi Kawarabayashi, Robin Thomas, and Paul Wollan. Quickly excluding a non-planar graph. *arXiv:2010.12397*, 2020. doi:10.48550/arXiv.2010.12397.

[30] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.

[31] Leon G. Kraft. *A Device for Quantizing, Grouping, and Coding Amplitude Modulated Pulses*. M.S. thesis, Massachusetts Institute of Technology, Cambridge, MA, 1949. Hdl:1721.1/12390.

[32] Kurt Mehlhorn. Nearly optimal binary search trees. *Acta Informatica*, 5(4):287–295, 1975. doi:10.1007/BF00264563.

[33] Jeremy P. Spinrad. *Efficient graph representations*, volume 19 of *Fields Institute Monographs*. American Math. Society, 2003.

A A Mixed Labelling Scheme for $\mathcal{R}_k$

In this appendix, we modify the labelling scheme of Dujmović et al. [11] and its improvements by Gawrychowski and Janczewski [19] in order to show the existence of a strongly g_0-orientable efficient weighted mixed labelling scheme for $\mathcal{R}_k$ mentioned in Section 3. In addition to mixing results from [11, 19], we obtain a further simplification using logarithmic height tree-decompositions (see Lemma 21, below). This not only simplifies the labelling schemes from [11, 19], but also produces a sparse induced-universal graph without requiring any of the additional machinery used by Esperet et al. [17].

A rooted tree-decomposition $(T, (B_x \mid x \in V(T)))$ is a *binary tree-decomposition* if T is a binary tree. Variants of the following result appear in several contexts, with different bounds for the width of the tree-decomposition and the height of the tree T. For example, Bodlaender and Hagerup [4] use a similar result in the context of parallel algorithms and Jain and Tewari [25] use a similar result in the context of space-efficient algorithms. The result cited here is due to Chatterjee et al. [6].

Lemma 21 ([6, Lemma 7])**.** *Let H be an n-vertex graph of treewidth at most k. Then H has a rooted binary tree-decomposition $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ of width at most $6k+5$ in which* $\text{height}(T) \in O(\log n)$.

Proof of Lemma 19. Lemma 19 is establishing several results at once, using combinations of techniques from [11, 17, 19] and Lemma 21. Thus, we present the proof in stages. We begin by describing a $(\log n + O(\sqrt{\log n}))$-bit adjacency labelling scheme for $\mathcal{R}_k$ that uses an adjacency tester A. Next we describe clique labels, local identifiers, and an identity tester I that, along with the adjacency labelling for A, give an unweighted strongly g_0-orientable (g_1, g_2, g_3) mixed labelling scheme. Finally, we show how to make this into a weighted strongly g_0-orientable (g_1, g_2, g_3) mixed labelling scheme using the trick of *simulating weight by multiplicity*.

Since $\mathcal{R}_0 \subseteq \mathcal{R}_1$, we may assume without loss of generality that $k \geqslant 1$; we may also assume that $n \geqslant 2$, since for $n = 1$ the bounds hold trivially. As usual, we first describe the format of the labels before describing the operation of the adjacency tester A. Let $\tau = (G^+, G, \omega) \in \mathcal{T}_{\mathcal{R}_k}$ and let $n = |V(G)|$. (For now we assume $\omega(v) = 1$ for all $v \in V(G)$.) Thus, G^+ and G are subgraphs of $H \boxtimes P$, where H is a graph of treewidth at most k and P is a path. Without loss of generality, assume that $P = (1, 2, \ldots, h)$, with $h \leqslant n$. Each vertex of G is a pair (v, i) with an *H-coordinate* $v \in V(H)$ and a *P-coordinate* $i \in V(P)$. We may assume, without loss of generality, that G contains at least one vertex with P-coordinate i for each $i \in [h]$ (and therefore $h \leqslant n$) and that each vertex of H appears as the H-coordinate of at least one vertex of G (and therefore $|V(H)| \leqslant n$). The label of each vertex (v, i) of G has two parts. The first part, $\lambda_1(v, i)$, is determined by the P-coordinate, i. The second part, $\lambda_2(v, i)$, is obtained from a labelling of $H[V_i]$ (where V_i is defined below), so it represents the H-coordinate, v.

By Lemma 21, H has a rooted binary tree-decomposition $\mathcal{T} := (T, (B_x \mid x \in V(T)))$ of width at most $6k+5$ in which $\text{height}(T) \in O(\log n)$. For each $x \in V(T)$, add edges to H so that B_x becomes a clique. (Note that for any clique K in G^+, its projection onto $V(H)$ is a clique in the original treewidth-k graph H before adding these edges, and therefore has

size at most $k+1$.) Define a strict partial order $<$ on $V(H)$ such that $v < w$ if the home of v in $\mathcal{T}$ is a strict T-ancestor of the home of w in $\mathcal{T}$ (resolving ties arbitrarily for vertices with the same home node). For each $w \in V(H)$, let $N_H^+(w) = \{v \in N_H(w) \mid v < w\}$. Then $|N_H^+(w)| \leqslant 6k+5$ since $v \in N_H^+(w)$ implies that $v \in B_x \setminus \{w\}$ where x is the home of w in $\mathcal{T}$. Note that, for each clique K in H, $K \subseteq N_H^+(w) \cup \{w\}$ for some $w \in K$.

Mapping $V(H)$ onto intervals: The first step in [11] is an idea used (at least implicitly) by Gavoille and Labourel [18] in their labelling scheme for bounded-treewidth graphs. Here we refine this idea using Lemma 21. We map each vertex v of H to a closed interval $f(v)$ with integer endpoints in $\{1,\dots,2|V(H)|\}$ and such that each integer is an endpoint of at most 1 interval. This mapping has the following properties (for some fixed $c > 0$):

(I1) For any two vertices $v, w \in V(H)$, $f(v) \subset f(w)$, $f(w) \subset f(v)$ or $f(v) \cap f(w) = \emptyset$.
(I2) For each $v \in V(H)$ and each $w \in N_H^+(v)$, $f(v) \subset f(w)$.
(I3) For each $p \in \mathbb{R}$, $|\{v \in V(H) : p \in f(v)\}| \leqslant ck \log n$.

The existence of such a mapping f is easily established by a recursive procedure that takes a node x of T and an interval $[a,b]$ where a and b are integers with $b-a+1 = 2|\bigcup_{y \in V(T_x)}(B_y \setminus A_y)|$, where A_y denotes the parent adhesion of y in $\mathcal{T}$. Let $v_1,\dots,v_r$ be the vertices in $B_x \setminus A_x$ ordered so that $v_1 < \cdots < v_r$. Then $f(v_i) = [a+i-1, b-i+1]$ for each $i \in \{1,\dots,r\}$. If x has a left child x_1 then let $n_1 := |\bigcup_{y \in V(T_{x_1})}(B_y \setminus A_y)|$. If x has no left child, then $n_1 := 0$. The procedure is then applied recursively on the left child x_1 of x (if any) with the interval $[a+r, m]$ and the procedure is then applied recursively to the right child x_2 of x (if any) with the interval $[m+1, b-r]$, where $m := a+r+2n_1-1$. Initially, the procedure is called with $[a,b] = [1, 2|V(H)|]$ and x as the root of T.

Let G_f be the interval intersection graph with vertex-set $V(G_f) := V(H)$ and edge-set $E(G_f) := \{vw \in \binom{V(H)}{2} : f(v) \cap f(w) \neq \emptyset\}$. Property (I2) implies that H is a subgraph of G_f. Property (I3) implies that G_f has no clique larger than $ck \log n$, for some fixed constant c. The graph G_f has a proper colouring $\varphi : V(H) \to [\lfloor ck \log n \rfloor]$ [21]. The label for each vertex (v,i) of G will include the $O(\log(k \log n))$-bit integer $\varphi(v)$.

Since every clique K in H satisfies $K \subseteq N_H^+(w) \cup \{w\}$ for some $w \in K$, Property (I2) implies:

(I4) Let K be a clique in H. Then there exists $w \in K$ such that $K \setminus \{w\} \subseteq N_H^+(w)$ and $f(w) \subset f(v)$ for each $v \in K \setminus \{w\}$.

Property (I4) is a crucial difference between the scheme we describe here and the labelling schemes of Dujmović et al. [11] and that of Gavoille and Labourel [18] on which it is based. In essence, it ensures that the ancestor-descendant relationships in T are consistent with containment relationships in the interval representation. This simultaneously simplifies the labelling scheme and makes it g_0-orientable without requiring additional tricks used by Esperet et al. [17].[5]

[5]Specifically, Property (I4) completely eliminates the need for the extended signature $\sigma^+(v,i)$ of a vertex (v,i) of G. This allows for a number of simplifications. Furthermore, the difference in length between $\sigma^+(v,i)$ and the signature $\sigma(v,i)$ is the main reason that the adjacency labelling scheme in [11] is not $n^{o(1)}$-orientable, so this modification almost immediately yields a universal graph with $n^{1+o(1)}$ edges.

Treating G as a sequence of graphs and mapping $V(H)$ onto points: The next idea in [11] is to treat $V(G)$ as a sequence of vertex subsets of H. For each $i \in [h]$, let $V_i := \{v \in V(H) : \{(v,i-1),(v,i),(v,i+1)\} \cap V(G) \neq \emptyset\}$. Each vertex (v,i) of G contributes a vertex to at most three sets V_{i-1}, V_i and V_{i+1}, so $\sum_{i=1}^{h} |V_i| \leqslant 3n$. For each vertex (v,i) of G, define the set $X_{v,i} := \{v\} \cup (V_i \cap N_H^+(v))$. In our labelling scheme, the label of (v,i) will include enough information to identify the vertices in $N_G((v,i)) \cap (X_{v,i} \times \{i-1,i,i+1\})$. The following property shows that this is sufficient to cover every edge of G:

(Q1) For each edge $(v,i)(w,j)$ of G with $w \in N_H^+(v)$, we have that $w \in X_{v,i}$.

For each vertex v of H, let $x_f(v)$ be the right endpoint of the interval $f(v)$. For each $i \in [h]$, let $S_i := \{x_f(v) : v \in V_i\}$. Note that S_i is a set of integers, for each $i \in [h]$. It is helpful for consecutive sets S_i and S_{i+1} to satisfy certain properties, for each $i \in [h-1]$. Therefore, Dujmović et al. [11] and Gawrychowski and Janczewski [19] define a sequence of supersets $S_1^+, \ldots, S_h^+$. The exact definition of $S_1^+, \ldots, S_h^+$ is not necessary for the current discussion. For our purposes it is sufficient to know that for any fixed integer $q \geqslant 1$ the sets $S_1^+, \ldots, S_h^+$ can be chosen so that

(a) $S_i^+ \supseteq S_i$ for each $i \in [h]$;
(b) there is a constant $c > 0$ such that $\sum_{i=1}^{h} |S_i^+| \leqslant cq^2 n$;
(c) for each $i \in \{1, \ldots, h-1\}$, for each $t \in [q]$, and for each set of t consecutive elements of S_i^+, at least $t-1$ of these elements are in S_{i+1}^+; and
(d) for each $i \in \{2, \ldots, h\}$, for each $t \in [q]$, and for each set of t consecutive elements of S_i^+, at least $t-1$ of these elements are in S_{i-1}^+.

We remark that the contents of the sets $S_1^+, \ldots, S_h^+$ are entirely determined by the contents of the sets $S_1, \ldots, S_h$. (Formally, the sequence $S_1^+, \ldots, S_h^+$ is the result of applying a function $Q : (2^{\mathbb{R}})^* \to (2^{\mathbb{R}})^*$ whose input is a sequence of finite sets of real numbers and whose output is a sequence of finite sets of real numbers. The function Q is universal; it does not depend on H, P, or G.)

The row label λ_1: With the sets $S_1^+, \ldots, S_h^+$ described, we can now discuss λ_1. The first part of $\lambda_1(v,i)$ is obtained by applying Corollary 8 to the set $[h]$ using the weight function defined by $i \mapsto |S_i^+|$ to obtain a prefix-free code $\rho : [h] \to \{0,1\}^*$ and functions $\overleftarrow{\pi} : [h] \to \{0,1\}^*$ and $\overrightarrow{\pi} : [h] \to \{0,1\}^*$. By Corollary 8 and Property (b) of $S_1^+, \ldots, S_h^+$, $|\rho(i)| \leqslant \log(\sum_{i=1}^{h} |S_i^+|) - \log|S_i^+| + O(1) \leqslant \log n - \log|S_i^+| + O(1)$ for each $i \in [h]$.

Then

$$\lambda_1(v,i) := \langle \rho(i), \overleftarrow{\pi}(i), \overrightarrow{\pi}(i) \rangle$$

and

$$|\lambda_1(v,i)| \leqslant \log n - \log|S_i^+| + O(\log\log n) \ , \tag{21}$$

for each vertex (v,i) of G.

B-trees: Until this point, the labelling scheme we have been describing comes from [11], except for the modifications to the interval mapping function f using Lemma 21 that guarantee (I4). At this point, we continue with the B-tree based labelling scheme from [19] rather than the binary-tree based scheme in [11].

A *B-tree* stores a set of real numbers in the leaves of a tree B in which all leaves have the same depth. A *weight-balanced B-tree* is parameterized by a parameter $a \geqslant 6$. A weight-balanced B-tree B is *semi-balanced* if, for each node x of B, the number of leaves in the subtree B_x is at most $6a^{\text{height}(B_x)}$ and for each *non-root* node x of B, the number of leaves in B_x is at least $\frac{1}{2}a^{\text{height}(B_x)}$. Let B be a weight-balanced B-tree with $r \in O(n)$ leaves. The weight-balance conditions guarantee that $\text{height}(B) \leqslant \log_a r + 1$. The weight-balance conditions also ensure that the number of children of any node x of B is at most $12a$. This implies that any root-to-leaf path in a B-tree can be encoded by a sequence of $\log_a r + 1$ integers, each requiring $\lceil \log(12a) \rceil \leqslant 5 + \log a$ bits to represent, for a total of at most $\lceil \log(12a) \rceil \cdot (\log_a r + 1) = \log r + O(\log_a r + \log a)$ bits. The value of a used in [19] and here is approximately $2^{\sqrt{\log n}}$, so that $\log a \in O(\sqrt{\log n})$ and $\log_a n \in O(\sqrt{\log n})$.

We use a sequence of weight-balanced B-trees, $B_1, \ldots, B_h$, where the leaves of B_i store the values in S_i^+. The implementation details of these B-trees are not important here. What is important is that the transformation from B_i to B_{i+1} is performed by a rebalancing operation that is applied only to nodes of depth d_i. Rebalancing a node x of depth d_i involves replacing the children $y_1, \ldots, y_p$ of x with children $y'_1, \ldots, y'_m$ and redistributing the set of children of $y_1, \ldots, y_p$ so that they become children of $y'_1, \ldots, y'_m$. Therefore, the set of grandchildren of x in B_i is the same as the set of grandchildren of x in B_{i+1} (but the path in B_i from x to a grandchild z is different from the path in B_{i+1} from x to z). This has the following important consequence:

(Q2) For each $i \in [h-1]$ and each $\ell \in S_i^+ \cap S_{i+1}^+$, the root-to-$\ell$ path $P_{B_i}(\ell)$ in B_i and the root-to-ℓ path $P_{B_{i+1}}(\ell)$ in B_{i+1} differ by at most a single vertex. More precisely, a subpath xyz in $P_{B_i}(\ell)$ can be replaced by a subpath $xy'z$ to give $P_{B_{i+1}}(\ell)$.[6]

The vertex signature $\sigma(v,i)$: For each $i \in [h]$ and each $x \in V(B_i)$, define the *signature* $\sigma_{B_i}(x)$ as the encoding of $P_{B_i}(x)$ as a sequence of $\text{depth}_{B_i}(x)$ integers in $\{0, \ldots, 12a-1\}$. Then

$$|\sigma_{B_i}(x)| \leqslant \lceil \log(12a) \rceil \cdot \text{depth}_{B_i}(x) \leqslant (\log a) \cdot \text{depth}_{B_i}(x) + O(\sqrt{\log n}) \ .$$

For each $i \in [h]$ and each $S \subseteq S_i^+$, define $\text{lca}'_{B_i}(S)$ as the maximum depth non-leaf node in B_i that contains all elements of S in its subtree. (Equivalently, $\text{lca}'_{B_i}(S) = \text{lca}_{B_i}(S)$ if $\text{lca}_{B_i}(S)$ is not a leaf of B_i, otherwise $\text{lca}'_{B_i}(S) = \text{p}_{B_i}(\text{lca}_{B_i}(S))$.) For each $i \in [h]$ and each $v \in V_i$, define $x'_{B_i}(v) := \text{lca}'_{B_i}(f(v) \cap S_i^+)$ and define $x_{B_i}(v)$ as the child of $x'_{B_i}(v)$ that has $x_f(v)$ as a leaf-descendant. (The node $x'_{B_i}(v)$ exists because $x_f(v) \in S_i \subseteq S_i^+$ and $x_f(v) \in f(v)$ and $x_{B_i}(v)$ exists because $x'_{B_i}(v)$ is a non-leaf node.) We make use of the following property of this definition:

(Q3) For each $i \in [h]$ and each $v \in V_i$, the smallest leaf-descendant of $x_{B_i}(v)$ is contained in $f(v)$.

For each $i \in [h]$ and each $v \in V_i$, define the *signature* $\sigma(v,i) := \sigma_{B_i}(x_{B_i}(v))$. Then

$$|\sigma(v,i)| = |\sigma_{B_i}(x_{B_i}(v))|$$

[6] There are two exceptions. Occasionally, $P_{B_{i+1}}(\ell)$ is obtained by removing the first vertex of $P_{B_i}(\ell)$ or by adding a vertex at the beginning of $P_{B_i}(\ell)$. Since these exceptions are distracting and easily handled, we ignore them here.

$$
\begin{aligned}
&\leqslant (\log a)\cdot \mathrm{depth}_{B_i}(\mathrm{lca}_{B_i}(f(v)\cap S_i^+)) + O(\log_a n)\\
&\leqslant (\log a)\cdot \mathrm{height}(B_i) + O(\log_a n)\\
&\leqslant \log|S_i^+| + O(\log_a n + \log a)\\
&= \log|S_i^+| + O(\sqrt{\log n})\ .
\end{aligned}
$$

The following is the critical property of the vertex signatures:

(Q4) For any two vertices (v,i) and (w,j) of G, $(\rho(i),\sigma(v,i),\varphi(v)) = (\rho(j),\sigma(w,j),\varphi(w))$ if and only if $(v,i)=(w,j)$.

Indeed, since ρ is an injective function $\rho(i)=\rho(j)$ if and only if $i=j$. If $\sigma(v,i)=\sigma(w,i)$ then $y := x_{B_i}(v) = x_{B_i}(w)$. By (Q3), $f(v)$ and $f(w)$ each contain the smallest leaf-descendant of y, so $f(v)\cap f(w)\neq\emptyset$. Since φ is a proper colouring of the intersection graph G_f, $\varphi(v)=\varphi(w)$ and $f(v)\cap f(w)\neq\emptyset$ if and only if $v=w$.

We make use of the fact that $\sigma(v,i)$ is defined for many pairs (v,i) that are not vertices of G (but not necessarily all vertices of $H\boxtimes P$). We make use of the fact that, for each $i\in[h]$ and each $v\in V_i$, $\sigma(v,i)$ is defined, which implies that $\sigma(v,i)$ is defined if any of $(v,i-1)$, (v,i), or $(v,i+1)$ is a vertex of G.

Let (v,i) and (w,j) be two vertices of G such that $w\in\{v\}\cup N_H^+(v)$ and $|j-i|\leqslant 1$. Since $|j-i|\leqslant 1$, $x'_{B_j}(v)$, $x_{B_j}(v)$, and $\sigma(v,j)$ are all defined. By Property (I2), $f(v)\subseteq f(w)$. Therefore $x'_{B_j}(w)$ is a B_j-ancestor of $x'_{B_j}(v)$ and of its child $x_{B_j}(v)$. Therefore, $\sigma_{B_j}(x'_{B_j}(w))$ is a prefix of $\sigma(v,j)=\sigma_{B_j}(x_{B_j}(v))$. Since $x_{B_j}(w)$ is a child of $x'_{B_j}(w)$, this implies that $\sigma(w,j)$ consists of a prefix of $\sigma(v,j)$ followed by the $\lceil\log(12a)\rceil$-bit representation of $c_{w,j}$, where $c_{w,j}\in\{0,\dots,12a-1\}$ is the index of $x_{B_j}(w)$ in the list of children of $x'_{B_j}(w)$. To summarize, it is possible to construct $\sigma(w,j)$ given $\sigma(v,j)$, $|\sigma(w,j)|$, and $c_{w,j}$.

The adjacency list $\alpha(v,i)$: For each $(v,i)\in V(G)$, define

$$
\begin{aligned}
\alpha^-(v,i) &:= \langle\, \langle|\sigma(w,i-1)|, c_{w,i-1}, \varphi(w)\rangle \mid w\in\{v\}\cup N_H^+(v),\ (w,i-1)\in N_G((v,i))\,\rangle\\
\alpha^0(v,i) &:= \langle\, \langle|\sigma(w,i)|, c_{w,i}, \varphi(w)\rangle \mid w\in N_H^+(v),\ (w,i)\in N_G((v,i))\,\rangle\\
\alpha^+(v,i) &:= \langle\, \langle|\sigma(w,i+1)|, c_{w,i+1}, \varphi(w)\rangle \mid w\in\{v\}\cup N_H^+(v),\ (w,i+1)\in N_G((v,i))\,\rangle\\
\alpha(v,i) &:= \langle\, \alpha^-(v,i), \alpha^0(v,i), \alpha^+(v,i)\,\rangle\ .
\end{aligned}
$$

Since $|N_H^+(v)|\leqslant 6k+5$ for each $v\in V(H)$, $|\alpha(v,i)|\in O(k\sqrt{\log n})$.

The transition codes $\overleftarrow{\tau}(v,i)$ and $\overrightarrow{\tau}(v,i)$: The last (and critical) piece of the labelling schemes in [11, 19] is the notion of a *transition code* that makes it possible to derive the signature $\sigma(v,i+1)$ (or $\sigma(v,i-1)$) from $\sigma(v,i)$ and a bitstring of length $O(\sqrt{\log n})$. Formally, there is a universal function $J:(\{0,1\}^*)^2\to\{0,1\}^*$ such that, for each $i\in[h-1]$ and each $v\in V_i\cap V_{i+1}$ (which includes every $(v,i)\in V(G)$ with $i<h$), there exists a bitstring $\overrightarrow{\tau}(v,i)$ of length $O(\sqrt{\log n})$ such that $J(\sigma(v,i),\overrightarrow{\tau}(v,i))=\sigma(v,i+1)$ (for $i=h$, let $\overrightarrow{\tau}(v,h):=\varepsilon$). Symmetrically, for each $i\in\{2,\dots,h\}$ and each $v\in V_i\cap V_{i-1}$ (which includes every $(v,i)\in V(G)$ with $i\geqslant 2$), there exists a bitstring $\overleftarrow{\tau}(v,i)$ of length $O(\sqrt{\log n})$ such that $J(\sigma(v,i),\overleftarrow{\tau}(v,i))=\sigma(v,i-1)$ (for $i=1$, let $\overleftarrow{\tau}(v,1):=\varepsilon$).

We now describe the format of the bitstring $\overrightarrow{\tau}(v,i)$ and how it is used by the transition function J. The bitstring $\overleftarrow{\tau}(v,i)$ is defined in a completely symmetric manner. The transition from $\sigma(v,i)$ to $\sigma(v,i+1)$ proceeds in three steps.

In the first step, we take $\sigma(v,i)$ onto $\sigma_{B_i}(\ell)$ for some leaf $\ell \in f(v) \cap S_i^+ \cap S_{i+1}^+$. At least one such leaf ℓ exists because $v \in V_i \cap V_{i+1}$, and therefore $x_f(v) \in S_i \cap S_{i+1} \subseteq S_i^+ \cap S_{i+1}^+$ is a leaf in B_i and B_{i+1}. Let $y = x_{B_i}(v)$. If y is a leaf of B_i then $y = x_f(v)$ because $x_f(v) \in S_i \subseteq S_i^+$ and $x_f(v) \in f(v)$. In this case we can simply take $\ell := y$ and we already have the signature $\sigma_{B_i}(\ell) = \sigma(v,i)$.

Now suppose that $y := x_{B_i}(v)$ is not a leaf of B_i. By (Q3), the smallest leaf-descendant ℓ' of y is in $f(v)$. The signature $\sigma_{B_i}(\ell')$ can be obtained from $\sigma_{B_i}(y) = \sigma(v,i)$ by appending a sequence of $\lceil \log(12a) \rceil (\text{height}(B_i) - \text{depth}_{B_i}(y))$ zeros. If $\ell' \in S_{i+1}^+$ then we take $\ell := \ell'$ and we are done. If $\ell' \notin S_{i+1}^+$ then $\ell' < x_f(v)$ and, by property (c) of $S_1^+, \ldots, S_h^+$, the successor ℓ'' of ℓ' in S_i^+ is in S_{i+1}^+. In this case, we take $\ell := \ell''$ and the signature $\sigma_{B_i}(\ell'')$ can be obtained from $\sigma_{B_i}(y)$ by appending a sequence of $\lceil \log(12a) \rceil (\text{height}(B_i) - \text{depth}_{B_i}(y)) - 1$ zeros followed by a 1. In each case, the information needed to obtain $\sigma_{B_i}(\ell)$ can be recorded in $O(\log\log n)$ bits. Summarizing, there exists a universal function J_1 such that, for each $i < h$ and each $v \in V_i \cap V_{i+1}$, there exists $\ell \in f(v) \cap S_i^+ \cap S_{i+1}^+$ and a bitstring $\overrightarrow{\tau}_1(v,\ell,i)$ of length $O(\log\log n)$ such that $J_1(\sigma(v,i), \overrightarrow{\tau}_1(v,\ell,i)) = \sigma_{B_i}(\ell)$.

In the second step we take $\sigma_{B_i}(\ell)$ onto $\sigma_{B_{i+1}}(\ell)$. By property (Q2) of $B_1, \ldots, B_h$, the path $P_{B_{i+1}}(\ell)$ can be obtained by replacing a subpath xyz in $P_{B_i}(\ell)$ with a subpath $xy'z$. In terms of the signature function σ, this difference between $P_{B_i}(\ell)$ and $P_{B_{i+1}}(\ell)$ can be described by an $O(\log\log n)$-bit encoding of $\text{depth}_{B_i}(x)$ and two $\lceil \log(12a) \rceil$-bit integers that give the index of y' in the list of children of x and the index of z in the list of children of y'. Although this is the only change to the root-to-ℓ path $P_{B_i}(\ell)$, there is an additional change to $\sigma_{B_i}(\ell)$ that needs to be recorded. Let $p := \text{p}_{B_i}(\ell)$ be the parent of ℓ. Then the set of children of p in B_i and the set of children of p in B_{i+1} are different. There are B_i-children of p in $S_i^+ \setminus S_{i+1}^+$ that are not in B_{i+1} and there are B_{i+1}-children of p in $S_{i+1}^+ \setminus S_i^+$ that are not in B_i. Therefore, the last integer in $\sigma_{B_i}(\ell)$ may be different from the last integer in $\sigma_{B_{i+1}}(\ell)$ even though each of these describes the edge $p\ell$. When this occurs, it can also be described by a $\lceil \log(12a) \rceil$-bit integer, namely the last integer in $\sigma_{B_{i+1}}(\ell)$. To summarize, there exists a universal function $J_2 : (\{0,1\}^*)^2 \to \{0,1\}^*$ such that, for each $i \in [h-1]$ and each $\ell \in S_i^+ \cap S_{i+1}^+$, there exists a bitstring $\overrightarrow{\tau}_2(\ell,i)$ of length $O(\sqrt{\log n})$ such that $\sigma_{B_{i+1}}(\ell) = J_2(\sigma_{B_i}(\ell), \overrightarrow{\tau}_2(\ell,i))$.

In the third and final step, we take $\sigma_{B_{i+1}}(\ell)$ onto $\sigma(v,i+1)$. Since $\ell \in f(v) \cap S_{i+1}^+$, the node $x'_{B_{i+1}}(v)$ is a B_{i+1}-ancestor of ℓ. Therefore, the signature $\sigma(v,i+1)$ can be obtained from $\sigma_{B_{i+1}}(\ell)$, $\text{depth}_{B_{i+1}}(x_{B_{i+1}}(v))$, and the $\lceil \log(12a) \rceil$-bit integer $c_{v,i+1}$ that gives the index of $x_{B_{i+1}}(v)$ in the list of children of $x'_{B_{i+1}}(v)$. Formally, there exists a universal function J_3 such that, for each $i < h$ and each $v \in V_i \cap V_{i+1}$, there exists a bitstring $\overrightarrow{\tau}_3(v,i)$ of length $O(\sqrt{\log n})$ such that $J_3(\sigma_{B_{i+1}}(\ell), \overrightarrow{\tau}_3(v,i)) = \sigma(v,i+1)$.

Summarizing, the forward transition code

$$\overrightarrow{\tau}(v,i) := \langle \overrightarrow{\tau}_1(v,\ell,i),\ \overrightarrow{\tau}_2(\ell,i),\ \overrightarrow{\tau}_3(v,i) \rangle$$

has length $O(\sqrt{\log n})$, and the transition function J on $\overrightarrow{\tau}(v,i)$ is defined by

$$J(\sigma(v,i),\overrightarrow{\tau}(v,i)) := J_3(J_2(J_1(\sigma(v,i),\overrightarrow{\tau}_1(v,\ell,i)),\overrightarrow{\tau}_2(\ell,i)),\overrightarrow{\tau}_3(v,i)) = \sigma(v,i+1) \ .$$

As mentioned above, the backward transition code $\overleftarrow{\tau}(v,i)$, also of length $O(\sqrt{\log n})$, is defined symmetrically (using property (d) in place of (c)) so that $J(\sigma(v,i),\overleftarrow{\tau}(v,i)) = \sigma(v,i-1)$.

The second part $\lambda_2(v,i)$: This gives all the pieces needed for the second part of the label for a vertex (v,i) of G:

$$\lambda_2(v,i) := \langle \sigma(v,i),\ \varphi(v),\ \alpha(v,i),\ \overleftarrow{\tau}(v,i),\ \overrightarrow{\tau}(v,i)\rangle \ .$$

The complete vertex label for a vertex (v,i) of G is

$$\mu(v,i) := \langle \lambda_1(v,i),\ \lambda_2(v,i)\rangle$$

which has length $\log n + O(\sqrt{\log n})$, for fixed k. (For k that grows with n, $|\mu(v,i)| = \log n + O(k\sqrt{\log n})$.)

Adjacency testing: We now describe a procedure that defines the output of the adjacency tester A. The operation of this procedure makes use of the fact that $\lambda_2(v,i)$ contains all the information needed to evaluate $\sigma(v,i)$, $\sigma(v,i-1) = J(\sigma(v,i),\overleftarrow{\tau}(v,i))$, and $\sigma(v,i+1) = J(\sigma(v,i),\overrightarrow{\tau}(v,i))$.

Given the labels $\mu(v,i)$ and $\mu(w,j)$ for two vertices (v,i) and (w,j) of G, the procedure first uses $\lambda_1(v,i)$ and $\lambda_1(w,j)$ (along with the function F) to test if $i=j$, $i=j+1$ or $i=j-1$. If none of these is the case, then the procedure can immediately conclude that $(v,i)(w,j)$ is not an edge of G, since $i\neq j$ and ij is not an edge of P. Since G is a subgraph of $H\boxtimes P$, this implies that $A(\mu(v,i),\mu(w,j)) := 0$. Otherwise, the procedure deals with one of the following cases:

1. If $i=j$, then the procedure uses $\sigma(v,i)$ and $\alpha^0(v,i)$ to compute $\sigma(w',i)$ and $\varphi(w')$ for each $w'\in N_H^+(v)$ such that $(w',i)\in N_G((v,i))$. For each of these the procedure checks if $(\sigma(w',i),\varphi(w')) = (\sigma(w,i),\varphi(w))$. If so, then $(w,i) = (w',i)\in N_G((v,i))$ so $A(\mu(v,i),\mu(w,j)) := 1$.

   If this test fails for each of the entries in $\alpha^0(v,i)$, then the procedure concludes that $w\notin X_{v,i}$ or $(w,i)\notin N_G((v,i))$. This still leaves the possibility that $v\in X_{w,i}$ and $(v,i)\in N_G((w,j))$. The procedure checks this in exactly the same manner, but with the roles of $\mu(v,i)$ and $\mu(w,j)$ reversed. If this test also fails to return a result, then $A(\mu(v,i),\mu(w,j)) := 0$.

2. If $i=j+1$ (so $j=i-1$), then the procedure computes $\sigma(v,i-1) = J(\sigma(v,i),\overleftarrow{\tau}(v,i))$. Then using the entries from $\alpha^-(v,i)$, it searches for an entry such that $(\sigma(w',i-1),\varphi(w')) = (\sigma(w,j),\varphi(w))$. If such an entry is found, then $A(\mu(v,i),\mu(w,j)) := 1$.

   If this search fails, then the procedure computes $\sigma(w,j+1) = J(\sigma(w,j),\overrightarrow{\tau}(w,j))$. Then using the entries from $\alpha^+(w,j)$, it searches for an entry such that $(\sigma(v',j+1),\varphi(v')) = (\sigma(v,i),\varphi(v))$. If such an entry is found, then $A(\mu(v,i),\mu(w,j)) := 1$. If this search also fails, then $A(\mu(v,i),\mu(w,j)) := 0$.

3. If $i = j - 1$ (so $j = i + 1$), then the procedure computes $\sigma(v, i+1) = J(\sigma(v,i), \overrightarrow{\tau}(v,i))$. Then using the entries from $\alpha^+(v,i)$, it searches for an entry such that $(\sigma(w', i+1), \varphi(w')) = (\sigma(w,j), \varphi(w))$. If such an entry is found, then $A(\mu(v,i), \mu(w,j)) := 1$.

   If this search fails, then the procedure computes $\sigma(w, j-1) = J(\sigma(w,j), \overleftarrow{\tau}(w,j))$. Then using the entries in $\alpha^-(w,j)$, it searches for an entry such that $(\sigma(v', j-1), \varphi(v')) = (\sigma(v,i), \varphi(v))$. If such an entry is found, then $A(\mu(v,i), \mu(w,j)) := 1$. If this search also fails, then $A(\mu(v,i), \mu(w,j)) := 0$.

This concludes the description of the adjacency testing procedure A as well as the $(\log n + O(\sqrt{\log n}))$-bit adjacency labelling scheme for $\mathcal{R}_k$.

Clique labelling and identity testing: Thus far, we have described a g_0-orientable $(\log n + O(\sqrt{\log n}))$-bit adjacency labelling scheme for $\mathcal{R}_k$ (for any fixed k). Next we describe clique labels, the local identifiers, and the identity testing procedure that makes this into a strongly g_0-orientable unweighted (g_1, g_2, g_3) mixed labelling scheme.

Let K be a clique in G^+ and let $V_K := \{v \mid (v,j) \in K\}$. Then $K \subseteq V_K \times \{i_K - 1, i_K\}$ and $V_K \subseteq V_{i_K}$ for some $i_K \in [h]$ (with $i_K \geqslant 2$ unless $h = 1$). Define

$$\lambda_1(K) := \langle \rho(i_K), \overleftarrow{\pi}(i_K), \overrightarrow{\pi}(i_K) \rangle \ .$$

Since K is a clique in $G^+ \subseteq H \boxtimes P$, V_K is a clique in the original treewidth-k graph H (before adding edges within bags of $\mathcal{T}$). Therefore $|V_K| \leqslant k+1$ and there exists $v_K \in V_K$ such that $V_K \subseteq \{v_K\} \cup N_H^+(v_K)$. Since $K \subseteq V_K \times \{i_K - 1, i_K\}$, $|K| \leqslant 2(k+1) = 2k+2$. Furthermore, at least one of (v_K, i_K) or $(v_K, i_K - 1)$ is in K. Therefore $v_K \in V_{i_K}$ (and $v_K \in V_{i_K - 1}$ if $i_K \geqslant 2$), so $\sigma(v_K, i_K)$ and $\overleftarrow{\tau}(v_K, i_K)$ are defined. We can now mimic the adjacency labelling. Define

$$\begin{aligned}
\alpha^-(K) &:= \langle\, \langle |\sigma(w, i_K - 1)|, c_{w, i_K - 1}, \varphi(w) \rangle \mid (w, i_K - 1) \in K \,\rangle \\
\alpha^0(K) &:= \langle\, \langle |\sigma(w, i_K)|, c_{w, i_K}, \varphi(w) \rangle \mid (w, i_K) \in K \,\rangle \\
\alpha(K) &:= \langle\, \alpha^-(K),\ \alpha^0(K) \,\rangle \ .
\end{aligned}$$

Since $|K| \leqslant 2k+2$, $|\alpha(K)| \in O(k\sqrt{\log n})$. By Property (I4), $f(v_K) \subseteq f(w)$ for each $w \in V_K$. As discussed above, this implies that $\sigma(w, i_K)$ is completely determined by $\sigma(v_K, i_K)$, $|\sigma(w, i_K)|$, and the integer c_{w, i_K} (specifically, by the prefix of $\sigma(v_K, i_K)$ of length $|\sigma(w, i_K)| - \lceil \log(12a) \rceil$ together with c_{w, i_K}). Define

$$\mu(K) := \langle\, \lambda_1(K),\ \sigma(v_K, i_K),\ \overleftarrow{\tau}(v_K, i_K),\ \alpha(K) \,\rangle \ .$$

Then,

$$\begin{aligned}
|\mu(K)| &\leqslant |\lambda_1(K)| + |\sigma(v_K, i_K)| + |\overleftarrow{\tau}(v_K, i_K)| + O(k(\sqrt{\log n} + \log\log n)) \\
&\leqslant \log n - \log |S^+_{i_K}| + (\log a)\, \mathrm{depth}_{B_{i_K}}(x_{B_{i_K}}(v_K)) + O(\log_a n + k\sqrt{\log n}) \\
&\leqslant \log n - \log |S^+_{i_K}| + (\log a)\, \mathrm{height}(B_{i_K}) + O(k\sqrt{\log n}) \\
&\leqslant \log n + O(\log a + \log_a n + k\sqrt{\log n}) \\
&\leqslant \log n + O(k\sqrt{\log n}) \ .
\end{aligned}$$

To see that μ is injective when restricted to cliques in G^+, suppose $\mu(K) = \mu(L)$ for two cliques K and L in G^+. The bitstring $\lambda_1(K)$ completely determines $\rho(i_K)$ and $\rho(i_K - 1) = F(\rho(i_K), \overleftarrow{\pi}(i_K))$. From $\sigma(v_K, i_K)$ and $\overleftarrow{\tau}(v_K, i_K)$, we obtain the signature $\sigma(v_K, i_K - 1) = J(\sigma(v_K, i_K), \overleftarrow{\tau}(v_K, i_K))$. The bitstrings $\sigma(v_K, i_K - 1)$ and $\alpha^-(K)$ determine $\{(\sigma(w, i_K - 1), \varphi(w)) : (w, i_K - 1) \in K\}$ because each $\sigma(w, i_K - 1)$ is uniquely reconstructed from $\sigma(v_K, i_K - 1)$, $|\sigma(w, i_K - 1)|$, and c_{w,i_K-1}. The bitstrings $\sigma(v_K, i_K)$ and $\alpha^0(K)$ determine $\{(\sigma(w, i_K), \varphi(w)) : (w, i_K) \in K\}$ similarly. Thus, $\mu(K)$ completely determines $\{(\rho(j), \sigma(w, j), \varphi(w)) : (w, j) \in K\}$. Since $\mu(K) = \mu(L)$, we have

$$\{(\rho(j), \sigma(w, j), \varphi(w)) : (w, j) \in K\} = \{(\rho(j), \sigma(w, j), \varphi(w)) : (w, j) \in L\} \ .$$

Since the triple $(\rho(j), \sigma(w, j), \varphi(w))$ is unique for each $(w, j) \in V(G)$, this implies that $K = L$.

Local identifiers: For each vertex $(u, j) \in K$, the local identifier for (u, j) is a single integer $\kappa(K, (u, j)) \in [2k + 2]$ that indicates the position of the entry for (u, j) in $\alpha(K)$. This integer also implicitly encodes whether $j = i_K - 1$ or $j = i_K$. Thus $|\kappa(K, (u, j))| \leqslant \lceil \log(2k + 2) \rceil \in O(\log(k + 2))$. For fixed k, this satisfies the requirement $g_2(n) \in O(1)$.

Identity testing: We now describe the identity tester I. Let K be a clique in G^+, let $(u, j) \in K$, and let (v, i) be a vertex of G. Let (v_K, i_K) be as in the description of $\mu(K)$, so that $K \subseteq V_K \times \{i_K - 1, i_K\}$ and $f(v_K) \subseteq f(w)$ for each $(w, j) \in K$.

Given the labels $\mu(K) = \langle \lambda_1(K), \sigma(v_K, i_K), \overleftarrow{\tau}(v_K, i_K), \alpha(K) \rangle$ and $\mu(v, i) = \langle \lambda_1(v, i), \lambda_2(v, i) \rangle$, along with the identifier $\kappa(K, (u, j))$, the identity tester proceeds as follows. First, the tester examines $\lambda_1(v, i)$ and $\lambda_1(K)$ to determine if $i = i_K$ or $i = i_K - 1$. If neither of these is the case, then $j \neq i$ so $(u, j) \neq (v, i)$ and the tester outputs $I(\mu(K), \kappa(K, (u, j)), \mu(v, i)) := 0$.

If $i \in \{i_K - 1, i_K\}$, then the tester locates the entry $\langle |\sigma(u, j)|, c_{u,j}, \varphi(u) \rangle$ in $\alpha(K)$ using $\kappa(K, (u, j))$, which encodes the position of this entry in $\alpha(K)$. The location of this entry also determines whether $j = i_K - 1$ or $j = i_K$. Since the tester now knows the values of j and i, it can immediately determine that $I(\mu(K), \kappa(K, (u, j)), \mu(v, i)) := 0$ if $j \neq i$. Otherwise, $j = i$ and there are two cases to consider:

- If $i = i_K$ then the tester extracts $\sigma(u, i)$ from $\sigma(v_K, i_K)$ using $|\sigma(u, i)|$ and $c_{u,i}$. If $(\sigma(u, i), \varphi(u)) = (\sigma(v, i), \varphi(v))$, then $(u, j) = (v, i)$ and the tester outputs 1; otherwise it outputs 0.
- If $i = i_K - 1$ then the tester computes $\sigma(v_K, i) = \sigma(v_K, i_K - 1) = J(\sigma(v_K, i_K), \overleftarrow{\tau}(v_K, i_K))$. The tester extracts $\sigma(u, i)$ from $\sigma(v_K, i)$ using $|\sigma(u, i)|$ and $c_{u,i}$. If $(\sigma(u, i), \varphi(u)) = (\sigma(v, i), \varphi(v))$, then $(u, j) = (v, i)$ and the tester outputs 1; otherwise it outputs 0.

Simulating weights by multiplicity: Finally, we explain how to make the unweighted (g_1, g_2, g_3) mixed labelling scheme described so far into a strongly g_0-orientable weighted mixed labelling scheme. By Lemma 14 and Observation 16, it suffices to construct a strongly g_0-orientable weak weighted mixed labelling scheme; the additive constants in the vertex and clique label lengths are absorbed into g_1 and g_3. Let $\tau = (G^+, G, \omega) \in \mathcal{T}_{\mathcal{R}_k}$ be a nice triple and let $n := |V(G)|$. Thus, $\omega : V(G) \to \mathbb{N}^+$ and $n \leqslant \omega(G) \leqslant 2n$, so $\log \omega(G) \leqslant \log n + 1$.

The only modification needed is in the definition of the sets $S_1, \ldots, S_h$. Define $\omega(v,i) := 0$ for each $(v,i) \in (V(H) \times \{0,\ldots,h+1\}) \setminus V(G)$. For each $i \in [h]$ and each $v \in V_i$, define $\delta(v,i) := \omega(v,i-1) + \omega(v,i) + \omega(v,i+1)$. For each $i \in [h]$ and each $v \in V_i$, at least one of $(v,i-1)$, (v,i), or $(v,i+1)$ is in $V(G)$, so $\delta(v,i) \geqslant 1$. Furthermore, $\sum_{i\in[h]} \sum_{v\in V_i} \delta(v,i) \leqslant 3\omega(G)$ since each vertex (v,i) of G contributes $\omega(v,i)$ to $\delta(v,j)$ for at most three values of j. Fix $\epsilon \in (0, 1/(6\omega(G))]$. For each $i \in \{1,\ldots,h\}$, define

$$S_i := \{x_f(v) - \epsilon j : v \in V_i, j \in \{0,\ldots,\delta(v,i)-1\}\} \ .$$

With this new definition of $S_1,\ldots,S_h$, everything proceeds exactly as described above. We define $(S_1^+,\ldots,S_h^+) := Q(S_1,\ldots,S_h)$ so that $S_1^+,\ldots,S_h^+$ satisfy properties (a)–(d). Note that $\sum_{i=1}^h |S_i| = \sum_{i\in[h]} \sum_{v\in V_i} \delta(v,i) \leqslant 3\omega(G)$ so, by property (b), $\sum_{i=1}^h |S_i^+| \in O(\omega(G))$.

Before going further, we give an intuitive explanation of why this works. Replacing each leaf $x_f(v)$ in B_i with a cluster of $\delta(v,i)$ leaves ensures that the subtree of B_i rooted at $x'_{B_i}(v)$ contains at least $\delta(v,i) \geqslant \omega(v,i)$ leaves. This places a lower bound on the height of the subtree of B_i rooted at $x'_{B_i}(v)$. Since $x_{B_i}(v)$ is a child of $x'_{B_i}(v)$, this places a lower bound on the height of the subtree of B_i rooted at $x_{B_i}(v)$. Since the height of B_i is $\log_a |S_i^+| + O(1)$, the lower bound on the height of the subtree rooted at $x_{B_i}(v)$ upper bounds the depth of $x_{B_i}(v)$ in B_i. We now make this precise.

For each $i \in [h]$ and each $v \in V_i$, let $S_i(v) := \{x_f(v) - \epsilon j : j \in \{0,\ldots,\delta(v,i)-1\}\}$. Since $S_i(v) \subseteq f(v) \cap S_i \subseteq f(v) \cap S_i^+$, the node $x'_{B_i}(v) = \mathrm{lca}'_{B_i}(f(v) \cap S_i^+)$ is a B_i-ancestor of $\mathrm{lca}'_{B_i}(S_i(v))$. The node $\mathrm{lca}'_{B_i}(S_i(v))$ is an internal node of B_i whose subtree contains all $\delta(v,i)$ leaves of $S_i(v)$. Because a subtree of height s in a weight-balanced B-tree of parameter a has at most $6a^s$ leaves, we have $6a^s \geqslant \delta(v,i)$, so $s \geqslant \log_a \delta(v,i) - O(1)$. Therefore,

$$\begin{aligned}
\mathrm{depth}_{B_i}(x_{B_i}(v)) &= \mathrm{depth}_{B_i}(x'_{B_i}(v)) + 1 \\
&\leqslant \mathrm{depth}_{B_i}(\mathrm{lca}'_{B_i}(S_i(v))) + 1 \\
&\leqslant \mathrm{height}(B_i) - \log_a(\delta(v,i)/6) + 1 \\
&\leqslant \log_a |S_i^+| - \log_a \delta(v,i) + O(1) \ .
\end{aligned}$$

Since $\delta(v,i) \geqslant \omega(v,i)$, this immediately implies that

$$\mathrm{depth}_{B_i}(x_{B_i}(v)) \leqslant \log_a |S_i^+| - \log_a \omega(v,i) + O(1) \ .$$

It follows that the length of $\sigma(v,i)$ satisfies

$$\begin{aligned}
|\sigma(v,i)| &= |\sigma_{B_i}(x_{B_i}(v))| \\
&\leqslant \lceil \log(12a) \rceil \mathrm{depth}_{B_i}(x_{B_i}(v)) \\
&\leqslant (5 + \log a)(\log_a |S_i^+| - \log_a \omega(v,i) + O(1)) \\
&\leqslant \log |S_i^+| - \log \omega(v,i) + O(\log_a |S_i^+| + \log a) \\
&\leqslant \log |S_i^+| - \log \omega(v,i) + O(\sqrt{\log n}) \ .
\end{aligned}$$

Then, as in the unweighted case, the length of the label $\lambda_2(v,i)$ satisfies

$$|\lambda_2(v,i)| \leqslant |\sigma(v,i)| + O(k\sqrt{\log n})$$

$$\leqslant \log|S_i^+| - \log\omega(v,i) + O(k\sqrt{\log n}) \ .$$

The code $\rho : [h] \to \{0,1\}^*$ is constructed from the weight function $\psi(i) := |S_i^+|$ using Corollary 8. Therefore, $|\rho(i)| \leqslant \log\psi([h]) - \log|S_i^+| + O(1) \leqslant \log\omega(G) - \log|S_i^+| + O(1)$. Thus, $|\lambda_1(v,i)| \leqslant \log\omega(G) - \log|S_i^+| + O(\log\log n)$. Therefore,

$$\begin{aligned}|\mu(v,i)| &= |\lambda_1(v,i)| + |\lambda_2(v,i)| + O(\log\log n)\\ &\leqslant \log\omega(G) - \log|S_i^+| + \log|S_i^+| - \log\omega(v,i) + O(k\sqrt{\log n})\\ &= \log\omega(G) - \log\omega(v,i) + O(k\sqrt{\log n}) \ ,\end{aligned}$$

which satisfies the requirement $g_1(n) \in O(k\sqrt{\log n})$.

For the clique label $\mu(K)$, since at least one of $(v_K, i_K - 1)$ or (v_K, i_K) belongs to K, we have $\delta(v_K, i_K) \geqslant \min_{u\in K}\omega(u)$, and the same depth bound gives $|\sigma(v_K, i_K)| \leqslant \log|S_{i_K}^+| - \log\delta(v_K, i_K) + O(\sqrt{\log n})$. Therefore, we similarly obtain

$$\begin{aligned}|\mu(K)| &\leqslant |\lambda_1(K)| + |\sigma(v_K, i_K)| + O(k\sqrt{\log n})\\ &\leqslant \log\omega(G) - \log|S_{i_K}^+| + \log|S_{i_K}^+| - \log\delta(v_K, i_K) + O(k\sqrt{\log n})\\ &\leqslant \log\omega(G) - \log\min_{u\in K}\omega(u) + O(k\sqrt{\log n}) \ ,\end{aligned}$$

which satisfies the requirement $g_3(n) \in O(k\sqrt{\log n})$.

g_0-orientability: For each nice triple $\tau = (G^+, G, \omega) \in \mathcal{T}_{\mathcal{R}_k}$, let (μ_τ, κ_τ) be the mixed labelling of τ for (A, I) described above, and let $\mathcal{S}$ be the indexed collection of these labellings over all nice triples in $\mathcal{T}_{\mathcal{R}_k}$. Then $(\mathcal{S}, A, I)$ is a weak weighted mixed labelling scheme for $\mathcal{R}_k$. Fix $n \in \mathbb{N}^+$, and let U_n be the n-universal graph defined by $(\mathcal{S}, A)$, using only nice triples in $\mathcal{T}'_{\mathcal{R}_k,\leqslant n}$. For every such triple, $|V(G)| \leqslant n$ and $\omega(G) \leqslant 2n$ and $\sum_i |S_i^+| \in O(\omega(G)) \subseteq O(n)$. Consequently, the bounds on the lengths of the auxiliary label components established above are uniform over these triples, with n now denoting the universal-graph parameter rather than the order of a particular graph. We now describe an orientation of U_n in which each vertex has outdegree at most $g_0(n)$.

Let $\beta\beta'$ be an edge of U_n. Then there exists a nice triple $\tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{R}_k,\leqslant n}$ and an edge $e = (v,i)(w,j) \in E(G)$ such that $\beta = \mu_\tau(v,i)$ and $\beta' = \mu_\tau(w,j)$. Since $G^+ \in \mathcal{R}_k$, G^+ is a subgraph of $H \boxtimes P$ for some graph H of treewidth at most k and path $P = (1, \ldots, h)$. Since $(v,i)(w,j) \in E(G)$, we have $|i - j| \leqslant 1$. In H, the ancestor ordering $<$ of $\mathcal{T}$ is a strict partial order. Exactly one of the following holds for the pair of vertices (v,i) and (w,j):

- $v \neq w$: then either $w < v$ (so $w \in N_H^+(v)$) or $v < w$ (so $v \in N_H^+(w)$). We orient the edge $\beta\beta'$ towards the endpoint whose H-coordinate is smaller in $<$. That is, if $w \in N_H^+(v)$, we orient $\beta\beta'$ towards β'; if $v \in N_H^+(w)$, we orient $\beta\beta'$ towards β.
- $v = w$: then $i \neq j$ and $|i - j| = 1$. We orient the edge $\beta\beta'$ towards the endpoint with smaller P-coordinate. That is, if $j = i - 1$, we orient $\beta\beta'$ towards β'; if $i = j - 1$, we orient $\beta\beta'$ towards β.

We now bound the maximum contribution to the outdegree of β from this orientation rule. An edge incident to $\beta = \mu_\tau(v,i)$ is oriented away from β towards $\beta' = \mu_\tau(w,j)$ only

when $w \in N_H^+(v)$ (with $j \in \{i-1, i, i+1\}$) or when $w = v$ and $j = i-1$. In every such case, $w \in \{v\} \cup N_H^+(v)$, so (w, j) corresponds to an entry in $\alpha(v, i)$. We consider the three cases corresponding to the operation of the adjacency tester $A(\beta, \beta')$:

1. $i = j$: In this case, the edge is oriented away from β only if $w \in N_H^+(v)$. The tester examines the at most $6k+5$ entries in $\alpha^0(v, i)$ and finds the entry $\langle |\sigma(w, i)|, c_{w,i}, \varphi(w)\rangle$. Using $\sigma(v, i)$ and this entry, $\sigma(w, i)$ and $\varphi(w)$ are uniquely determined. Since β fixes $\lambda_1(v, i) = \lambda_1(w, j)$, the prefix $\langle \lambda_1(w, j), \sigma(w, j), \varphi(w)\rangle$ of β' is fixed by β and the entry. There are at most $6k+5$ such entries.

2. $j = i-1$: In this case, the edge is oriented away from β if $w \in \{v\} \cup N_H^+(v)$. The tester computes $\sigma(v, i-1) = J(\sigma(v, i), \overleftarrow{\tau}(v, i))$ and searches in $\alpha^-(v, i)$. The matching entry $\langle |\sigma(w, i-1)|, c_{w,i-1}, \varphi(w)\rangle$ uniquely determines $\sigma(w, i-1)$ and $\varphi(w)$. Since $\rho(j) = F(\rho(i), \overleftarrow{\pi}(i))$ is uniquely determined by $\lambda_1(v, i)$, the prefix $\langle \rho(j), \sigma(w, j), \varphi(w)\rangle$ of β' is fixed by β and the entry. There are at most $6k+6$ such entries.

3. $j = i+1$: In this case, the edge is oriented away from β only if $w \in N_H^+(v)$. The tester computes $\sigma(v, i+1) = J(\sigma(v, i), \overrightarrow{\tau}(v, i))$ and searches in $\alpha^+(v, i)$. The matching entry uniquely determines $\sigma(w, i+1)$ and $\varphi(w)$, and $\rho(j) = F(\rho(i), \overrightarrow{\pi}(i))$ is determined by $\lambda_1(v, i)$. There are at most $6k+5$ such entries.

Across all three cases, there are at most $(6k+5) + (6k+6) + (6k+5) = 18k+16$ choices for the prefix $\langle \rho(j), \sigma(w, j), \varphi(w)\rangle$ of β'. The remaining components of β'—namely $\overleftarrow{\pi}(j)$, $\overrightarrow{\pi}(j)$, $\alpha(w, j)$, $\overleftarrow{\tau}(w, j)$, and $\overrightarrow{\tau}(w, j)$—have total length $O(k\sqrt{\log n})$, so there are at most $\exp(O(k\sqrt{\log n}))$ choices for these components. Thus, the number of edges oriented away from β is at most $(18k+16) \cdot \exp(O(k\sqrt{\log n})) = \exp(O(k\sqrt{\log n})) = g_0(n)$.

Strong g_0-orientability: All that remains is to show that $(\mathcal{S}, A, I)$ is strongly g_0-orientable. Fix $n \in \mathbb{N}^+$. Let $\tau_0 = (G_0^+, G_0, \omega_0) \in \mathcal{T}'_{\mathcal{R}_k, \leqslant n}$ be a nice triple, let $(\mu_0, \kappa_0) := (\mu_{\tau_0}, \kappa_{\tau_0})$, let K_0 be a clique in G_0^+, and let $(u_0, j_0) \in K_0$. Define $\zeta := \mu_0(K_0)$ and $\gamma := \kappa_0(K_0, (u_0, j_0))$. We must upper bound the number of strings β for which there exists a nice triple $\tau = (G^+, G, \omega) \in \mathcal{T}'_{\mathcal{R}_k, \leqslant n}$, a clique K in G^+, and a vertex $(u, j) \in K$ such that $\mu_\tau(K) = \zeta$, $\kappa_\tau(K, (u, j)) = \gamma$, $\mu_\tau(u, j) = \beta$, and $I(\zeta, \gamma, \beta) = 1$. Let β be such a string. The local identifier γ indexes an entry in $\alpha(K) = \langle \alpha^-(K), \alpha^0(K)\rangle$, which gives $|\sigma(u, j)|$, $c_{u,j}$, $\varphi(u)$, and whether $j = i_K$ or $j = i_K - 1$:

- If $j = i_K$, then $\lambda_1(u, j) = \lambda_1(K)$ is completely determined by ζ. The string $\sigma(v_K, i_K)$ extracted from ζ together with $|\sigma(u, i_K)|$ and c_{u,i_K} uniquely determines $\sigma(u, i_K) = \sigma(u, j)$. Along with $\varphi(u)$, the prefix $\langle \lambda_1(u, j), \sigma(u, j), \varphi(u)\rangle$ of β is fixed. The remaining components $\alpha(u, j), \overleftarrow{\tau}(u, j), \overrightarrow{\tau}(u, j)$ of β have total length $O(k\sqrt{\log n})$. Thus there are at most $\exp(O(k\sqrt{\log n}))$ such strings β.
- If $j = i_K - 1$, then $\rho(j) = F(\rho(i_K), \overleftarrow{\pi}(i_K))$ is determined by $\lambda_1(K)$. The tester computes $\sigma(v_K, i_K - 1) = J(\sigma(v_K, i_K), \overleftarrow{\tau}(v_K, i_K))$ using the strings $\sigma(v_K, i_K)$ and $\overleftarrow{\tau}(v_K, i_K)$ extracted from ζ. Together with $|\sigma(u, j)|$ and $c_{u,j}$ from γ, this uniquely determines $\sigma(u, j)$. Along with $\varphi(u)$, the prefix $\langle \rho(j), \sigma(u, j), \varphi(u)\rangle$ of β is fixed. The remaining components of β, namely $\overleftarrow{\pi}(j)$, $\overrightarrow{\pi}(j)$, $\alpha(u, j)$, $\overleftarrow{\tau}(u, j)$, and $\overrightarrow{\tau}(u, j)$, have total length $O(k\sqrt{\log n})$. Thus there are at most $\exp(O(k\sqrt{\log n}))$ such strings β.

In either case, the number of choices for β is at most $\exp(O(k\sqrt{\log n})) = g_0(n)$. Therefore, $(\mathcal{S}, A, I)$ is strongly g_0-orientable.

To obtain the full weighted scheme, for each $\tau = (G^+, G, \omega) \in \mathcal{T}_{\mathcal{R}_k}$, assign the labelling in $\mathcal{S}$ of $(G^+, G, \widehat{\omega})$, where

$$\widehat{\omega}(u) := \lceil \frac{|V(G)|\omega(u)}{\omega(G)} \rceil \ .$$

By Lemma 14, this increases the vertex and clique label-length bounds by at most 1. If $\tau \in \mathcal{T}'_{\mathcal{R}_k,\leqslant n}$, then $\widehat{\omega}(G) \leqslant \omega(G) \leqslant 2n$ by Observation 6. Hence, by Observation 16, the full weighted scheme is strongly g_0-orientable with the same function g_0. This proves Lemma 19. □

Let $\mathcal{H}_k$ be the class of graphs of treewidth at most k. Esperet et al. [17] show that, for each fixed $k \in \mathbb{N}^+$ and each $n \in \mathbb{N}^+$, $\mathcal{H}_k$ has an n-induced-universal graph with $n(\log n)^{O(k^2)}$ vertices and edges. We finish this appendix by remarking that this can be improved using the ideas above. In particular, the use of Lemma 21 eliminates a step used in [17] that leads to the appearance of k^2 in the exponent.

Lemma 22. *There exists a constant $c > 0$ such that, for each fixed $k \in \mathbb{N}^+$, the class of graphs $\mathcal{H}_k$ admits an injective g_0-orientable $(\log n + O(k \log\log n))$-bit adjacency labelling scheme with $g_0(n) \in O((\log n)^{ck})$.*

Proof Sketch. Let G be an n-vertex graph of treewidth at most k. By Lemma 21, G has a tree-decomposition $\mathcal{T} = (T, (B_x : x \in V(T)))$ of width at most $6k+5$ where T is a binary tree of height $O(\log n)$. As in the proof of Lemma 19, define a strict partial order $<$ on $V(G)$ such that $v < w$ if the home node of v in $\mathcal{T}$ is a strict T-ancestor of the home node of w (resolving ties arbitrarily for vertices with the same home node), and for each $v \in V(G)$ define $N^+_G(v) := \{u \in N_G(v) : u < v\}$. Note that $|N^+_G(v)| \leqslant 6k+5$. Map each vertex v of G to a closed interval $f(v)$ with integer endpoints in $\{1, \ldots, 2n\}$ satisfying Properties (I1)–(I3) as described in the proof of Lemma 19. For each $v \in V(G)$, let $x_f(v)$ be the right endpoint of $f(v)$. Since each integer is an endpoint of at most one interval, the points in $S := \{x_f(v) : v \in V(G)\}$ are distinct, and $x_f(v) \in f(v) \cap S$ for every $v \in V(G)$.

Let B be a balanced binary tree whose leaves are the elements of S, ordered as in $\mathbb{R}$. Then B has height $\lceil \log_2 n \rceil \leqslant \log n + 1$. For each $v \in V(G)$, define $x_B(v) := \mathrm{lca}_B(f(v) \cap S)$. Since $x_f(v) \in f(v) \cap S$, the set $f(v) \cap S$ is non-empty, so $x_B(v)$ is well-defined. For any $u \in N^+_G(v)$, Property (I2) implies $f(v) \subset f(u)$, so $\emptyset \neq (f(v) \cap S) \subseteq (f(u) \cap S)$. Consequently, $x_B(u)$ is an ancestor of $x_B(v)$ in B.

For each node $x \in V(B)$, we upper bound the number of vertices $v \in V(G)$ with $x_B(v) = x$:

- If x is an internal node of B with left child x_1 and right child x_2, choose a real number p_x strictly between the maximum leaf in the subtree of x_1 and the minimum leaf in the subtree of x_2. If $x_B(v) = x$, then $f(v) \cap S$ contains a leaf in the subtree of x_1 and a leaf in the subtree of x_2, so the interval $f(v)$ contains p_x.
- If x is a leaf of B, then x is a point $p_x \in S$, and $x_B(v) = x$ implies $p_x \in f(v)$.

In either case, every vertex $v \in V(G)$ with $x_B(v) = x$ satisfies $p_x \in f(v)$. By Property (I3), at most $ck \log n$ vertices are mapped to x. We can therefore assign each vertex $v \in V(G)$ a colour $\varphi(v) \in [\lfloor ck \log n \rfloor]$ such that any two distinct vertices mapped to the same node of B receive different colours (for instance, using the proper colouring of the interval intersection graph G_f).

For each $v \in V(G)$, define the signature $\sigma(v)$ as the binary string describing the path $P_B(x_B(v))$ from the root of B to $x_B(v)$. Then $|\sigma(v)| \leqslant \log n + 1$. Observe that, for any $v, w \in V(G)$, $(\sigma(v), \varphi(v)) = (\sigma(w), \varphi(w))$ if and only if $v = w$. For each $v \in V(G)$ and each $u \in N_G^+(v)$, since $x_B(u)$ is an ancestor of $x_B(v)$, $\sigma(u)$ is the prefix of $\sigma(v)$ of length $|\sigma(u)|$. Define $\alpha(v, u) := \langle |\sigma(u)|, \varphi(u) \rangle$. Then $|\alpha(v, u)| \leqslant O(\log \log n + \log k)$. For each $v \in V(G)$, define

$$\alpha(v) := \langle \alpha(v, u) : u \in N_G^+(v) \rangle \ .$$

Since $|N_G^+(v)| \leqslant 6k + 5$, $|\alpha(v)| \leqslant O(k \log \log n)$. The label for a vertex $v \in V(G)$ is

$$\ell_G(v) := \langle \sigma(v), \varphi(v), \alpha(v) \rangle \ ,$$

which has length $|\ell_G(v)| \leqslant \log n + O(k \log \log n)$. Since $(\sigma(v), \varphi(v))$ uniquely identifies v, the labelling is injective.

Given the labels $\ell_G(v)$ and $\ell_G(w)$ of two vertices $v, w \in V(G)$, the adjacency tester A operates as follows:

- If $\sigma(w)$ is a prefix of $\sigma(v)$ and $\alpha(v)$ contains an entry $\langle |\sigma(w)|, \varphi(w) \rangle$, then A returns 1.
- Else, if $\sigma(v)$ is a prefix of $\sigma(w)$ and $\alpha(w)$ contains an entry $\langle |\sigma(v)|, \varphi(v) \rangle$, then A returns 1.
- Otherwise, A returns 0.

To verify correctness, suppose the first condition holds. The entry $\langle |\sigma(w)|, \varphi(w) \rangle$ in $\alpha(v)$ certifies the existence of some $u \in N_G^+(v)$ such that $|\sigma(u)| = |\sigma(w)|$ and $\varphi(u) = \varphi(w)$. Since both $\sigma(u)$ and $\sigma(w)$ are prefixes of $\sigma(v)$ of the same length, $\sigma(u) = \sigma(w)$. Together with $\varphi(u) = \varphi(w)$, this implies $u = w$, so $w \in N_G^+(v)$ and $vw \in E(G)$. The second case is symmetric. Conversely, if $vw \in E(G)$, then either $w \in N_G^+(v)$ (satisfying the first condition) or $v \in N_G^+(w)$ (satisfying the second condition).

To see that the scheme is g_0-orientable, let U_n be the n-universal graph defined by this scheme. For each edge $e = \{\beta, \beta'\} \in E(U_n)$, there exists some graph $G \in \mathcal{H}_k$ with at most n vertices and an edge $vw \in E(G)$ such that $\beta = \ell_G(v)$ and $\beta' = \ell_G(w)$. If $w \in N_G^+(v)$, orient e towards β'; if $v \in N_G^+(w)$, orient e towards β (breaking ties arbitrarily if different graphs witness opposite orientations). Now consider any vertex $\beta = \langle \sigma, \varphi, \alpha \rangle \in V(U_n)$. An edge is oriented away from β towards β' only if β' corresponds to an in-neighbour $w \in N_G^+(v)$ for some witness graph. For each such neighbour, α contains an entry $\langle s, c \rangle = \langle |\sigma(w)|, \varphi(w) \rangle$. This entry completely determines the prefix $\langle \sigma(w), \varphi(w) \rangle$ of β', where $\sigma(w)$ is the prefix of σ of length s and $\varphi(w) = c$. The only remaining component of β' is $\alpha(w)$, which has length at most $c'k \log \log n$ for some constant $c' > 0$. Thus, there are at most $2^{c'k \log \log n} = (\log n)^{c'k}$ choices for $\alpha(w)$. Since α contains at most $6k + 5$ entries, the number of edges oriented away from β in U_n is at most $(6k + 5)(\log n)^{c'k} \in O((\log n)^{ck})$ for an absolute constant $c > 0$. Therefore, the scheme is g_0-orientable with $g_0(n) \in O((\log n)^{ck})$. □

Corollary 23. *For each fixed $k \in \mathbb{N}^+$ and each $n \in \mathbb{N}^+$, the class $\mathcal{H}_k$ of graphs of treewidth at most k has an n-induced-universal graph with $n(\log n)^{O(k)}$ vertices and edges.*

Proof. By Lemma 22, $\mathcal{H}_k$ admits an injective g_0-orientable $f(n)$-bit adjacency labelling scheme with $f(n) = \log n + O(k \log\log n)$ and $g_0(n) \in O((\log n)^{ck})$. For each $n \in \mathbb{N}^+$, the n-universal graph U_n defined by this scheme has $|V(U_n)| \leqslant 2^{f(n)+1} = n(\log n)^{O(k)}$ vertices and, by orientability, at most $|V(U_n)| \cdot g_0(n) \leqslant n(\log n)^{O(k)}$ edges. By injectivity, every n-vertex graph in $\mathcal{H}_k$ is isomorphic to an induced subgraph of U_n. □